GENERATIVE AI IN ENTERPRISES: OPTIMIZING APPLICATIONS WITH LARGE LANGUAGE MODELS

DONGHAO HUANG

SINGAPORE MANAGEMENT UNIVERSITY

2026

Generative AI in Enterprises: Optimizing Applications with Large Language Models

by

Donghao HUANG

Submitted to School of Computing and Information Systems in partial fulfillment of the requirements for the Degree of Doctor of Engineering (EngD) in Computer Science

Dissertation Committee:

Zhaoxia WANG (Supervisor / Chair)
Associate Professor of Computer Science (Practice), School of Computing and Information Systems
Singapore Management University

Chong Wah NGO (Co-Supervisor / Chair)
Lee Kong Chian Professor of Computer Science, School of Computing and Information Systems
Singapore Management University

Qianru SUN
Associate Professor of Computer Science, School of Computing and Information Systems
Singapore Management University

Singapore Management University
2026

Copyright (2026) Donghao HUANG

I hereby declare that this EngD dissertation is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in this dissertation.

This EngD dissertation has also not been submitted for any degree in any university previously.

Donghao HUANG

9 April 2026

Abstract

This dissertation investigates how to deploy Large Language Models (LLMs) effectively in enterprise settings, where accuracy, reliability, cost, privacy, and operational constraints often matter more than benchmark performance alone. Drawing on seventeen peer-reviewed publications (eleven published and six accepted for publication), the work develops and validates optimization strategies across three connected themes: retrieval-augmented generation (RAG), agentic AI for workflow automation, and deployment guidelines for real-world enterprise environments.

First, we study RAG optimization through systematic evaluation of open and proprietary models, highlighting conditions under which efficient open-weight models can match or exceed proprietary alternatives. To address a pervasive failure mode in open-source LLMs—output repetition—we introduce *Repetition-Aware Performance (RAP)*, a metric that integrates repetition penalties into task performance. Across models ranging from 2B to 70B parameters, RAP-guided tuning reduces repetition by up to 93.1% with only 3.7% performance degradation, enabling more dependable RAG outputs in production.

Second, we develop multi-agent architectures for enterprise automation and propose reliability-centric evaluation for tool-using systems. An invoice reconciliation system achieves 99.9% task success with edge-deployed models at 523 Joules per task, while a structured diagnostic taxonomy identifies tool initialization as a dominant source of agent failures in smaller models. For scalable automated assessment, we introduce an LLM-as-a-Judge framework and the reliability metric *Evaluation Completion Rate (ECR@1)*, enabling principled accuracy-reliability-cost trade-offs across a 175× pricing range; GPT-4o Mini emerges as production-optimal at 78× cost reduction versus premium models. A cooperative multi-agent system for acceptance test generation (CMAS4G2) demonstrates that open-weight models approach proprietary performance (87.6% vs. 90.4% success), with reasoning configuration profoundly impacting coordination. For payment processing, we design the first LLM-based agentic payment framework (HMASP), achieving 97.0% task success, and introduce *Agentic Success Rate (ASR)*—a trajectory-level workflow fidelity metric grounded in process-mining conformance checking. Across 18 LLMs and 90,000 task instances, ASR exposes systematic procedural deviations invisible to conventional metrics: most models bypass confirmation checkpoints during payment checkout despite near-perfect task success, with regulatory implications for PCI-DSS compliance. Ongoing work is evolving the research prototype into ConvPayMAS—an agentic payment system implementing Google's AP2 three-mandate cryptographic verification with patent-pending security features —upon which an enterprise agentic

evaluation and optimization framework is being designed.

Third, we examine deployment strategies across heterogeneous hardware and tasks. Benchmarking across six hardware platforms shows that platform choices (e.g., WSL vs. native Windows) can yield up to $21\times$ inference speedups, while medium-sized models (7B–32B) achieve optimal performance-efficiency trade-offs. A central finding from 504 configurations across seven model families is that reasoning effectiveness is strongly task-dependent: reasoning degrades simple binary sentiment classification (up to -19.9 F1 pp, 100% failure rate) yet substantially improves complex emotion recognition (up to $+16.0$ F1 pp, only 14% failure rate). Entity matching evaluation across 46 model configurations and 8 benchmark datasets reveals that reasoning effects are also family-dependent—low reasoning is optimal for GPT-5, high reasoning is critical for open-weight models, and Claude models exhibit extreme asymmetric sensitivity—while open-weight models rival proprietary APIs (GPT-OSS:120b outperforming GPT-4.1 at $20\times$ lower cost). Cross-domain studies in machine translation, logical reasoning, and sports analytics further validate generalizability of the proposed metrics and deployment principles.

Overall, this dissertation demonstrates that open-source LLMs, when optimized and evaluated with reliability-aware metrics, can support privacy-preserving and cost-effective enterprise deployment. The proposed metrics, architectural patterns, and empirically grounded guidelines provide a practical foundation for building robust generative AI systems in production settings.

Keywords: Large Language Models, Retrieval-Augmented Generation, Multi-Agent Systems, Agentic Commerce, Reasoning, Edge Deployment, Enterprise AI, Model Quantization, Few-Shot Learning, Fine-Tuning, Sentiment Analysis, Entity Matching, Tool Invocation, Evaluation Metrics, LoRA, Privacy-Preserving AI, Workflow Automation

Acknowledgments

I would like to express my deepest gratitude to my supervisor, Associate Professor Zhaoxia WANG, for her invaluable guidance, continuous support, and patience throughout this research journey. Her expertise in artificial intelligence and natural language processing has been instrumental in shaping this dissertation.

I am also grateful to Professor Chong Wah NGO for his co-supervision and insightful feedback that helped refine the direction of this research. Special thanks to Associate Professor Qianru SUN for serving on my dissertation committee and providing constructive suggestions. I would also like to thank Professor Chunyan MIAO of Nanyang Technological University for her time and expertise as the external reviewer of this dissertation.

This research was supported in part by the School of Computing and Information Systems at Singapore Management University, which provided the academic environment and resources necessary for this work. I also gratefully acknowledge the support of the Singapore Digital Scholarship (Postgraduate), awarded to me in 2025 by the Infocomm Media Development Authority (IMDA).[1]

I extend my appreciation to Mastercard for their support of this research. The collaboration with colleagues in Research and Development on retrieval-augmented generation (RAG), multi-agent systems, and edge deployment projects directly contributed to several publications in this dissertation. I am also grateful for access to computational resources, including GPU infrastructure for model training and evalu-

[1] https://www.imda.gov.sg/how-we-can-help/singapore-digital-scholarship/postgraduate

ation, as well as the flexibility to present this research at international conferences.

I also thank my co-authors—Thanh-Son NGUYEN, Fiona LIAUSVIA, Joon Kiat CHUA, Samuel SAMUEL, James TAN Yong Hong, Gauri MALWE, Ritik VARSHNEY, Alvin TSE, Shila CHEW, Anna DUTKIEWICZ, Aparna CHEKURU, Erin DEUTSCHMAN, Zhenda HU, Nicole TEO, Prasan PAI, and others—for their contributions to the publications underlying this research.

Finally, I thank my family and friends for their unwavering support and encouragement throughout this doctoral journey.

Table of Contents

Abstract **2**

Acknowledgments **4**

List of Figures **xiv**

List of Tables **xviii**

1 Introduction **1**

1.1 Background and Motivations 1

1.2 Research Questions 3

1.2.1 RQ1: Optimizing Retrieval-Augmented Generation 3

1.2.2 RQ2: Agentic AI for Workflow Automation 4

1.2.3 RQ3: Enterprise Deployment and Application Strategies . . 4

1.3 Publications 5

1.3.1 Publications Supporting RAG Optimization (Chapter 3) . . 5

1.3.2 Publications Supporting Agentic AI (Chapter 4) 5

1.3.3 Publications Supporting Enterprise Deployment and Applications (Chapter 5) 6

1.3.4 Research Journey: From Open-Weight LLMs to Enterprise AI Systems 9

1.4 Research Contributions 12

1.4.1 Novel Evaluation Metrics 12

1.4.2 Architectural Innovations 15
1.4.3 Empirical Findings 16
1.4.4 Industry Impact 19
1.5 Structure of the Dissertation 20

2 Literature Review **22**
2.1 Foundational Concepts: Generative AI and Large Language Models 22
2.1.1 Definition and Evolution of Generative AI 22
2.1.2 Historical Evolution of Language Models 23
2.1.3 Architecture of Large Language Models 24
2.1.4 Training Methodologies 25
2.2 Implementation Approaches for LLMs in Enterprises 26
2.2.1 Leveraging Pre-trained Models: Zero-shot and Few-shot Learning 26
2.2.2 Fine-tuning Approaches 27
2.3 Retrieval-Augmented Generation 27
2.3.1 RAG Architecture and Components 28
2.3.2 Advantages for Enterprise Applications 28
2.3.3 RAG Evaluation Frameworks 28
2.3.4 Challenges in RAG Optimization 29
2.4 Agentic AI and Multi-Agent Systems 30
2.4.1 Foundations of Agentic AI 30
2.4.2 Multi-Agent Architectures 30
2.4.3 Orchestration Frameworks 31
2.4.4 Enterprise Applications 32
2.4.5 LLM-as-a-Judge Paradigm 32
2.4.6 Challenges in Agentic Systems 33
2.5 Edge Deployment of Large Language Models 33
2.5.1 Motivation for Edge Deployment 34

2.5.2 Model Optimization Techniques 34
2.5.3 Deployment Frameworks 35
2.5.4 Hardware Considerations 35
2.5.5 Research Gaps in Edge Deployment 36
2.6 Reasoning in Large Language Models 37
2.6.1 Reasoning Capabilities and Limitations 37
2.6.2 Chain-of-Thought Prompting 37
2.6.3 Specialized Reasoning Models 38
2.6.4 Task Complexity and Reasoning Effectiveness 39
2.7 Sentiment Analysis and Cross-Domain Applications 39
2.7.1 Evolution of Sentiment Analysis 39
2.7.2 Machine Translation 40
2.7.3 Sports Predictive Analytics 40
2.7.4 Entity Matching and Resolution 40
2.8 Summary and Research Gaps 41
2.8.1 Gaps in RAG Optimization (RQ1) 42
2.8.2 Gaps in Agentic AI (RQ2) 42
2.8.3 Gaps in Enterprise Deployment (RQ3) 43

3 Optimizing LLMs for Retrieval-Augmented Generation 46
3.1 Introduction and Motivation 46
3.2 Methodology 47
3.2.1 RAG System Architecture 47
3.2.2 Evaluation Datasets 49
3.2.3 Model Selection 50
3.2.4 Evaluation Metrics 51
3.2.5 Hardware Configuration 51
3.3 Comparative Evaluation of LLMs for RAG 52
3.3.1 Generation Quality Results 52

3.3.2 Open-Source Model Competitiveness 52
3.3.3 Inference Speed Analysis 54
3.3.4 Qualitative Analysis 55
3.4 The Repetition-Aware Performance Metric 55
3.4.1 Repetition Ratio (RR) 58
3.4.2 Defining the RAP Score 60
3.4.3 Experimental Settings 61
3.4.4 Impact of RPP on Repetition and Performance 65
3.4.5 Optimizing the Trade-Off Between Performance and Repetition 67
3.4.6 Comparing the RAP-Tuned Performance of All Models . . . 68
3.4.7 How Severe Can Repetition Be? 69
3.4.8 Ablation Study: Effects of Different Penalty Functions . . . 70
3.4.9 Summary of RAP Findings 71
3.5 Evaluation-Guided Hyperparameter Search 72
3.6 Practical Deployment Guidelines 74
3.7 Summary 75

4 Agentic AI for Workflow Automation 78
4.1 Introduction and Motivation 78
4.2 Foundations of Agentic AI 80
4.2.1 Distinguishing Characteristics 80
4.2.2 Challenges in Enterprise Deployment 81
4.3 Multi-Agent Architecture for Invoice Reconciliation 82
4.3.1 Problem Context 82
4.3.2 System Architecture 82
4.3.3 LangGraph-Based Orchestration 84
4.3.4 Experimental Setup 84
4.3.5 Results 85
4.3.6 Error Analysis 86

4.4 When Agents Fail to Act: Diagnostic Framework for Tool Invocation Reliability . . . 87
4.4.1 Motivation and Gap Analysis . . . 88
4.4.2 Diagnostic Framework Overview . . . 88
4.4.3 Structured Error Taxonomy . . . 89
4.4.4 Experimental Methodology . . . 91
4.4.5 Results and Analysis . . . 92
4.4.6 Deployment Tiers and Practical Guidance . . . 96
4.5 LLM-as-Judge for Automated Evaluation . . . 97
4.5.1 Motivation . . . 97
4.5.2 Framework Design . . . 98
4.5.3 Novel Reliability Metrics . . . 98
4.5.4 Experimental Setup . . . 99
4.5.5 Results . . . 99
4.6 Multi-Agent System for Acceptance Test Generation . . . 100
4.6.1 Problem Context . . . 101
4.6.2 Architecture . . . 101
4.6.3 Evaluation Framework . . . 103
4.6.4 Results . . . 103
4.6.5 Integration Summary . . . 105
4.7 Hierarchical Multi-Agent System for Payments . . . 105
4.7.1 Architecture Overview . . . 105
4.7.2 Evaluation . . . 107
4.7.3 Results . . . 108
4.7.4 Novelty and Contributions . . . 108
4.8 Agentic Success Rate: Measuring Workflow Fidelity in Multi-Agent Payment Systems . . . 109
4.8.1 Motivation: Why Task Success Is Insufficient . . . 110
4.8.2 Architecture Adaptations for A2A and UCP Compliance . . . 110

4.8.3 Agentic Success Rate (ASR) 112
4.8.4 Experimental Setup 114
4.8.5 Results 114
4.8.6 Deep Dive: Per-Sample Deviation Analysis 117
4.8.7 Discussion: Regulatory and Architectural Implications . . . 119
4.8.8 Ongoing Work: ConvPayMAS and Enterprise Evaluation Framework 120
4.9 Practical Deployment Guidelines 125
4.9.1 Architecture Design 125
4.9.2 Model Selection 125
4.9.3 Reliability Engineering 126
4.9.4 Evaluation Strategy 126
4.10 Summary 127

5 Enterprise Deployment and Application Strategies 130
5.1 Introduction and Background 130
5.2 Edge Deployment Strategies 132
5.2.1 LLMs at the Edge with Ollama 132
5.2.2 Deploying Reasoning LLMs on Consumer Hardware 137
5.3 Reasoning Enhancement Techniques 141
5.3.1 Logical Reasoning via Lateral Thinking Puzzles 141
5.3.2 Explainable Sentiment Analysis with DeepSeek-R1 145
5.3.3 Task Complexity and Reasoning Effectiveness 147
5.4 Cross-Domain Applications 154
5.4.1 Chinese-English Translation Optimization 154
5.4.2 LLM Sentiment for Sports Betting Prediction 156
5.4.3 Entity Matching with LLMs at Scale 160
5.5 Summary and Implications 164
5.5.1 Edge Deployment Guidelines 165

5.5.2 Reasoning Enhancement Insights 166
5.5.3 Cross-Domain Application Patterns 168

6 Conclusions and Future Work 170

6.1 Summary of Contributions 170
6.1.1 RAG Optimization Contributions (Chapter 3) 170
6.1.2 Agentic AI Contributions (Chapter 4) 171
6.1.3 Enterprise Deployment and Application Contributions (Chapter 5) 173
6.1.4 Unifying the Evaluation Metrics for Practice 175
6.2 Key Findings 175
6.2.1 Finding 1: Open-Source Viability 176
6.2.2 Finding 2: Architecture Over Scale 176
6.2.3 Finding 3: Task Complexity Determines Reasoning Effectiveness 177
6.2.4 Finding 4: Few-Shot Learning Superiority 178
6.2.5 Finding 5: Novel Metrics Address Evaluation Gaps 178
6.2.6 Finding 6: Modular Multi-Agent Architectures Enable Reliable Automation 179
6.3 Practical Implications 179
6.3.1 Model Selection 180
6.3.2 Optimization Strategy 180
6.3.3 Evaluation Framework 180
6.3.4 Architecture Design 181
6.3.5 Infrastructure Planning 181
6.3.6 Limitations and Threats to Validity 182
6.4 Future Research Directions 184
6.4.1 Advanced RAG Architectures 184
6.4.2 Adaptive Reasoning Architectures 184

6.4.3 Enhanced Multi-Agent Coordination 185
6.4.4 Multimodal Enterprise Applications 185
6.4.5 Continual Learning and Adaptation 186
6.4.6 Sustainable AI Deployment 186
6.5 Concluding Remarks 186

A Supplementary Materials 202
A.1 Complete Model Specifications 202
A.2 Hardware Specifications 205
A.2.1 Platform Selection Rationale 205
A.2.2 Key Hardware Findings Summary 205
A.3 Detailed Experimental Configurations 207
A.3.1 RAG Experiments Configuration (Chapter 3) 207
A.3.2 Fine-Tuning Configuration (Chapters 3, 5) 207
A.3.3 Multi-Agent System Configuration (Chapter 4) 208
A.3.4 Task Complexity Experiments Configuration (Section 5.3.3) 209
A.3.5 Reasoning Deployment Experiments Configuration (Section 5.2.2) 210
A.3.6 Soccer Betting Experiments Configuration (Section 5.4.2) . 211
A.4 Invoice Reconciliation Error Examples 212
A.4.1 SUCCESS Example 212
A.4.2 NO INVOICE Error Example 213
A.4.3 ERROR Example (Tool Invocation Failure) 213
A.4.4 Error Taxonomy Reference 214
A.5 Prompt Templates 215
A.5.1 RAG System Prompt (Chapter 3) 215
A.5.2 Sentiment Analysis System Prompt (Chapter 5) 215
A.5.3 Logical Reasoning System Prompt (Section 5.3.1) 215
A.5.4 Multi-Agent Data Engineering Prompt (Chapter 4) 216

A.5.5 Soccer Betting Sentiment Prompt (Section 5.4.2) 216
A.6 Novel Metrics Summary 217

List of Figures

1.1 Research progression across the 17 publications. Papers are arranged chronologically (top to bottom) and grouped by research pillar (left to right). Solid arrows indicate within-pillar build-on relationships; dashed arrows show cross-pillar connections where findings from one thread informed another. 8

3.1 Overview of the RAG system architecture used in our experiments. . 48

3.2 Comparison of generation quality metrics across LLMs on PCI DSS benchmark. 53

3.3 Inference speed comparison across LLMs on consumer and professional hardware. 54

3.4 Best performance (F1 for WebQSP, BERT-F1 for MS MARCO) when tuning RPP using original evaluation metrics (Original) and our proposed repetition-aware performance metric (RAP). RAP helps find RPP values that minimally reduce performance while significantly lowering repetition ratio (RR). Result is from Llama-3-8B (RAG-Generic Prompt). 57

3.5 Overall Methodology: RAP integrates the repetition penalty in the evaluation score. In the example, the generated text contains non-word characters (NWC) repetition and text repetition. After penalization, the accuracy is reduced from 100% to 24.63%. ReDA stands for Repetition Detection Algorithm. 58

3.6 Illustrating the impact of RPP on the repetition ratio (RR) and performance across models, datasets, and prompting strategies. 66

3.7 Original performance, repetition-aware performance (RAP), and repetition ratio (RR) for Phi-3-mini in QA (subplots (a) to (f)) and Phi-3.5-mini in MT (subplots (g) and (h)) as a function of RPP. Blue lines represent original performance (F1 for WebQSP, BERT-F1 for MS MARCO, COMET for MAC), orange lines indicate RAP scores, and green lines (right y-axis) show the RR in percentage. The blue shaded region marks the RPP for optimal original performance, the orange region highlights the best RAP, and brown indicates overlap between the two. 67

3.8 Severity of repetition. Colored squares above each point indicate the percentage of repetitive responses. The x-axis labels include the model, prompting method, and tuned RPP value. 69

3.9 Gains across different penalty functions. The *Cubic* penalty consistently achieves the highest gains across all three datasets. 70

4.1 End-to-end workflow of the multi-agent architecture-based invoice reconciliation system. 83

4.2 Architecture and operational workflow of CMAS4G2. The diagram illustrates the nine-step process involving the Supervisor, Validator, and Quality Assurance agents. 102

4.3 Overview of the proposed HMASP. An external request (e.g., "Complete my payment") is sent to the Conversational Payment Agent (CPA - first agent level); (a) The CPA processes the request and delegates it to one of the two Supervisors (b1, b2 - second agent level) for further handling; (b) The selected Supervisor routes the request to a suitable Routing agent (c1, c2, or c3 - third agent level); (c) Routing agents initiate the corresponding task-specific workflow (Task 1, Task 2, Task 3); Upon initiation, a series of task-relevant function modules are triggered to complete the request; (d) Upon task completion, the Process summary agent (d1, d2, d3 - fourth agent level) compiles the outcome and generates a response. This summary response is propagated upward the system hierarchy through Supervisor agents and CPA; (e) When user input is required to complete the task, an interrupt mechanism is triggered to obtain the necessary input before execution resumes; (f) Finally, the CPA responds to the request (e.g., "Your payment has been successfully completed"). . . 106

4.4 Standards-compliant HMASP architecture. External agents interact via UCP-compliant REST/MCP endpoints (top), which delegate to the A2A protocol layer (middle). The LangGraph agent hierarchy (centre) implements the HMASP agents: the CPA routes to domain supervisors (CS, PS) and the Off-Topic Handler (OTH); supervisors dispatch to task-specific workflows (CRW, CSW, PPW). Workflows invoke the MCP Payment Gateway (bottom) for card validation, OTP, 3DS, and tokenization. Solid arrows denote handoffs (LLM tool calls); dashed arrows denote returns. 111

4.5 Overview of the agentic payment ecosystem with ConvPayMAS. The Shopper Agent (a) creates an Intent Mandate and communicates with the E-commerce Agent (b–d) via A2A streaming; upon item selection, a Payment Mandate is created (e) and validated against the cryptographic chain; a transaction session is established (f–g) and the user is handed off to ConvPayMAS (h–i), where the CSA routes to specialized sub-agents (j) that invoke MCP payment tools (k) for card operations and authorization; confirmation flows back through the conversational loop (l). 122

5.1 Performance and Energy Efficiency Metrics on RTX A6000. (a) F1 Score (%) vs. Energy per Inference (Joules). (b) F1 Score (%) vs. Performance-to-Power Ratio (%/J). Large models achieve high F1 scores but consume significantly more energy, while small models offer superior energy efficiency with acceptable performance trade-offs. 135

5.2 Performance vs. computational cost. Circles: base/non-thinking; triangles: reasoning/thinking. Colors: IMDB (blue), Amazon (green), GoEmotions (red). Dashed lines: Pareto frontiers where no configuration dominates on both metrics. Evaluated on H100 via Ollama; DSR1/V3 excluded for consistency. Labels show top performers per dataset. 152

5.3 Betting simulation bankroll evolution across matches for the 2023-24 EPL season comparing F1-tuned baseline models with their LLM-infused variations across four different machine learning architectures. The results demonstrate that F1-tuned configurations consistently produce more stable growth trajectories and higher final returns. 159

5.4 Cost vs. F1 Pareto analysis of 46 model configurations for entity matching using the compound strategy. Each point represents a model's mean F1 (across 8 datasets) and total API-equivalent token cost. The dashed red step line traces the Pareto frontier. Marker shapes distinguish proprietary API models (circles), open-weight models deployed locally via Ollama (squares), and open-weight models accessed via the OpenRouter API (diamonds). 165

List of Tables

1.1 Mapping of Publications to Dissertation Chapters 7

2.1 Comparison of LLM Architectural Paradigms 25

3.1 MS MARCO Evaluation Dataset Composition 50

3.2 Large Language Models Evaluated 50

3.3 RAG Generation Quality Results on PCI DSS Benchmark 53

3.4 Overview of Large Language Models and Their Specifications by Task 62

3.5 Comparison of model performance across different datasets, LLMs, and prompting strategies. 68

4.1 Models Evaluated for Multi-Agent Invoice Reconciliation 85

4.2 Invoice Reconciliation Performance Across Models and Platforms . 86

4.3 Overall Invoice Reconciliation Performance Across LLM Configurations. SR = Success Rate. All metrics based on 1,980 test instances. 93

4.4 Error Counts and Rates for Vision Tasks (990 instances). NI = NOT_INITIALIZED, AM = ARGS_MISMATCH. 94

4.5 Error Counts and Rates for Non-Vision Tasks (990 instances). NI = NOT_INITIALIZED, AM = ARGS_MISMATCH. 94

4.6 LLM-as-Judge Performance: Accuracy, Reliability, and Cost 99

4.7 CMAS4G2 Performance on 200-Ticket Kill Bill Benchmark. Values are mean±std across 5 runs. 104

4.8 HMASP Task Success Rates (%) Across Models 108

4.9 A comparative overview of existing autonomous agents in payment-relevant processes with their Task-Relevant (TR) metrics. For HMASP, averages from the best-performing open-weight LLM (Qwen2.5:32b) were reported. 109

4.10 Expected trajectories per scenario. $\rightarrow$ denotes agent handoff; $[\cdots]^{\times 2}$ denotes two identical consecutive repetitions (one per remaining dialogue turn). CPA: Conversational Payment Agent; PS: Payment Supervisor; PPW: Payment Processing Workflow; CS: Cards Supervisor; CRW: Card Registration Workflow; CSW: Card Selection Workflow; OTH: Off-Topic Handler. 113

4.11 T3 (Payment Processing) trajectory detail: 11 steps across 3 dialogue turns. Each step documents the agent activated and its role in the workflow. 113

4.12 HMASP vs. Ours: TSR (%) for top HMASP-baseline models and most-improved models. H = HMASP baseline, O = Ours (mean $\pm$ std over 5 repeats), Δ = improvement. **Bold**: $\geq$20 pp gains. HMASP-top models selected by baseline TSR; top-3 improved open-weight models ranked by average Δ across all tasks. 115

4.13 T3 (Payment) results (%, mean $\pm$ std over 5 repeats) and average across T1–T3 for the top 8 models. Sorted by Avg ASR desc. Yellow: TSR = HF1 = 100% but ASR $<$ 100%. All six open-weight models outperform both proprietary GPT models. 116

4.14 Expected (11-step) vs. observed (9-step) T3 trajectory for unambiguous payment inputs. Shaded rows mark the two steps skipped by models. 117

4.15 Per-sample T3 analysis for representative models: task failures vs. hidden deviations (samples that succeed on TSR but deviate on ASR). Hidden rate = hidden deviations / successful samples. `MSmall3.2:24b` is the only model with zero hidden deviations. . 118

5.1 Ollama Models Evaluated for Edge Deployment 134
5.2 Edge Deployment Performance: Small Models (0.5B–3B) 136
5.3 Edge Deployment Performance: Medium Models (7B–32B) 137
5.4 Edge Deployment Performance: Large Models (70B–90B) 137
5.5 Cross-Architecture Reasoning Performance (5-Level Sentiment) on H100 GPU 139
5.6 Cross-Platform Temporal Robustness: Original vs. Contemporary Reviews 141
5.7 Logical Reasoning Performance on TSQA Benchmark 143
5.8 DeepSeek-R1 Performance on Sentiment Classification Tasks 146
5.9 Reasoning-Focused Distillation vs. Base Model Performance. Diff. indicates performance difference (Distilled − Base). Failure Rate (FR) shows percentage of comparisons where distilled models underperform or match base models. 149
5.10 Thinking vs. Non-Thinking Mode Performance. Diff. = F1_Thinking − F1_Non-thinking. Speed Ratio (SR) = Thinking latency / Non-thinking latency. 150
5.11 Aggregated Performance Statistics Across 504 Configurations. Statistics reported as mean ± std across 12 model pairs and 7 shot levels (0–50 shots). 151
5.12 Chinese-English Translation Performance 155
5.13 Soccer Betting Returns: Baseline vs. LLM-Enhanced Models (%) . 158
5.14 Entity Matching: Top 15 of 46 Model Configurations (Mean F1 across 8 datasets) 162

5.15 Impact of reasoning effort on entity matching across model families. F1 is the mean across 8 datasets. "Best" = recommended setting considering both F1 and cost; *medium is marginally higher (93.17 vs. 93.14) but costs 2× more. ΔF1 = F1 range (max−min); ΔCost = cost multiplier from lowest to highest setting. For models with a thinking toggle, "On" and "Off" denote thinking enabled/disabled; "Adapt" denotes adaptive thinking. 163

6.1 Summary of Novel Evaluation Metrics Introduced 179

A.1 Complete Model Specifications 202

A.2 Hardware Platforms for Model Evaluation 206

A.3 Summary of Novel Evaluation Metrics 218

Chapter 1

Introduction

1.1 Background and Motivations

The rapid advancement of artificial intelligence (AI) over recent years has positioned Large Language Models (LLMs) at the forefront of technological innovation. Models such as GPT-4 [2], Claude 3 [6], DeepSeek-R1 [54] and their successors have demonstrated unprecedented capabilities in understanding, generating, and reasoning about natural language across diverse tasks. These capabilities have profound implications for enterprises, where AI-driven automation, knowledge retrieval, and intelligent workflows are increasingly essential for maintaining competitive advantage.

Enterprise adoption of generative AI is accelerating globally. According to multiple industry surveys, organizations across sectors—including finance, healthcare, manufacturing, and retail—are actively experimenting with or deploying LLM-based solutions [26]. The transformative potential of these technologies extends across the entire business value chain: from customer service automation and document processing to strategic decision support and creative content generation.

However, significant gaps remain between laboratory-level AI performance and real-world deployment reliability. Several critical challenges impede widespread enterprise adoption:

- **Hallucinations:** LLMs frequently generate plausible but factually incorrect in-

formation, undermining trust in AI-generated outputs [173]. This limitation is particularly problematic in knowledge-intensive domains such as legal, medical, and financial services where accuracy is paramount. Retrieval-Augmented Generation (RAG) has emerged as a promising solution, integrating external knowledge to improve accuracy [106, 45].

- **Repetition and Quality Issues:** Open-source LLMs often generate repetitive content, leading to extensive empty lines or recurring sentences that undermine text quality and fluency [44, 60]. This is particularly problematic in tasks requiring coherent and contextually relevant responses, such as conversational chatbots and automated document generation [164].

- **Reasoning Limitations:** While LLMs demonstrate impressive pattern recognition, their capacity for complex logical reasoning, multi-step inference, and consistent decision-making remains constrained [83]. Recent advancements have enhanced deductive reasoning through techniques like chain-of-thought prompting [163], yet models still exhibit limitations in multi-step reasoning tasks, often leading to cumulative errors in lengthy logical chains [151].

- **Agentic Reliability:** Deploying LLMs as autonomous agents in multi-step workflows introduces compounding failure modes where small errors cascade into significant operational failures [96, 19], requiring robust reliability metrics beyond traditional accuracy measures [158].

- **Computational Constraints:** Enterprise deployment must balance model performance against latency requirements, energy consumption, hardware costs, and data privacy considerations [176, 116]. Resource efficiency is particularly critical for edge deployment scenarios, where optimization techniques such as quantization have emerged to enable efficient deployment across heterogeneous devices [32, 43].

This dissertation addresses these challenges through systematic investigation of three interconnected technical pillars: (1) RAG optimization, (2) agentic AI for

workflow automation, and (3) enterprise deployment strategies. Each pillar contributes both theoretical frameworks and practical tools for bridging the gap between LLM potential and production-ready deployment. Importantly, these pillars are not independent: insights from RAG optimization (Chapter 3) inform output quality assessment in agentic systems (Chapter 4), while deployment findings (Chapter 5) directly impact the performance characteristics of multi-agent architectures.

1.2 Research Questions

To address the challenges outlined above, this dissertation investigates the following research questions:

1.2.1 RQ1: Optimizing Retrieval-Augmented Generation

How can RAG systems be optimized to improve response quality while addressing critical issues such as content repetition in open-source LLMs?

Retrieval-Augmented Generation addresses hallucination by grounding LLM responses in retrieved factual content, but implementation choices regarding model selection, retrieval strategies, and output quality assessment significantly impact system effectiveness. This research question is addressed through:

- Comparative evaluation of open-source versus proprietary models for RAG applications
- Development of the Repetition-Aware Performance (RAP) metric and Repetition Detection Algorithm (ReDA) for quantifying and penalizing repetition in model outputs
- Empirical guidelines for optimizing RAG deployments across diverse use cases

1.2.2 RQ2: Agentic AI for Workflow Automation

How can multi-agent LLM architectures be designed to reliably automate complex enterprise workflows while providing measurable reliability guarantees?

Enterprise automation demands more than single-turn text generation; it requires orchestrated systems capable of multi-step reasoning, tool usage, and adaptive decision-making. This research question is addressed through:

- Design of multi-agent systems with specialized roles for business process automation
- Development of hierarchical architectures for complex workflow orchestration
- Introduction of novel reliability metrics (Agentic Success Rate, Evaluation Completion Rate) that capture operational reality beyond traditional accuracy measures
- Systematic error analysis through a structured diagnostic taxonomy for tool invocation failures

1.2.3 RQ3: Enterprise Deployment and Application Strategies

What strategies enable effective LLM deployment across diverse enterprise applications, hardware configurations, and domain requirements?

The transition from prototype to production introduces challenges spanning hardware optimization, cost management, reasoning enhancement, and cross-domain generalization. This research question is addressed through:

- Systematic benchmarking across diverse hardware platforms from enterprise GPUs to edge devices
- Evaluation of reasoning-capable LLMs for complex tasks requiring explainability
- Introduction of metrics (Valid Classification Ratio, Reasoning Intensity, Translation Completeness Ratio) for assessing model reliability and efficiency

- Cross-domain validation spanning sentiment analysis, logical reasoning, machine translation, and sports predictive analytics

1.3 Publications

To address these research questions, the author conducted extensive research that resulted in seventeen accepted or published papers at established international AI and NLP venues. The author was the first author or an equal-contribution first author on all works listed below. Unless otherwise marked as "Accepted," all papers have been published.

1.3.1 Publications Supporting RAG Optimization (Chapter 3)

1. **Evaluation of Orca 2 Against Other LLMs for Retrieval Augmented Generation** [66]
   PAKDD 2024 Workshops

2. **Performance Analysis of Llama 2 Among Other LLMs** [72]
   IEEE Conference on Artificial Intelligence (IEEE CAI 2024)

3. **RAP: A Metric for Balancing Repetition and Performance in Open-Source Large Language Models** [74]
   NAACL 2025

1.3.2 Publications Supporting Agentic AI (Chapter 4)

4. **Scalable Invoice Reconciliation for SMEs: Edge-Deployed LLMs in Multi-Agent Systems** [73]
   ICDSAAI 2025

5. **When Agents Fail to Act: A Diagnostic Framework for Tool Invocation Reliability in Multi-Agent LLM Systems** [81]

ICAIBD 2026 **(Accepted)**

6. **LLM as a Judge for Scalable Test Coverage Evaluation: Accuracy, Operational Reliability, and Cost** [76]
   AAAI 2026 Workshops

7. **CMAS4G2: A Locally Deployed Cooperative Multi-Agent System for Gherkin Acceptance Test Generation** [77]
   PAKDD 2026 Workshops **(Accepted)**

8. **A Novel Hierarchical Multi-Agent System for Payments Using LLMs** [80]
   PAKDD 2026 **(Accepted)**

9. **Beyond Task Success: Measuring Workflow Fidelity in LLM-Based Agentic Payment Systems** [78]
   PAKDD 2026 Workshops **(Accepted)**

1.3.3 Publications Supporting Enterprise Deployment and Applications (Chapter 5)

10. **LLMs at the Edge: Performance and Efficiency Evaluation with Ollama on Diverse Hardware** [68]
    IJCNN 2025

11. **Deploying Reasoning LLMs for Sentiment Analysis: Architecture Tradeoffs on Consumer Hardware** [75]
    IEEE ICDM 2025 Workshops

12. **Logical Reasoning with LLMs via Few-Shot Prompting and Fine-Tuning: A Case Study on Turtle Soup Puzzles** [69]
    IEEE Symposium Series on Computational Intelligence (IEEE SSCI 2025)

13. **Explainable Sentiment Analysis with DeepSeek-R1: Performance, Efficiency, and Few-Shot Learning** [67]
    IEEE Intelligent Systems

14. **Task Complexity Matters: An Empirical Study of Reasoning in LLMs for Sentiment Analysis** [71]
    PAKDD 2026 **(Accepted)**

15. **Entity Matching with LLMs at Scale: Accuracy, Cost, and Reasoning Effects** [82]
    PAKDD 2026 Workshops **(Accepted)**

16. **Optimizing Chinese to English Translation with Large Language Models** [70]
    IEEE SSCI 2025

17. **Integrating LLM Sentiment Analysis into Machine Learning for Soccer Betting** [146]
    IEEE ICDM 2025 Workshops

Table 1.1 provides a summary mapping of publications to dissertation chapters, and Figure 1.1 illustrates how these papers build on each other within and across the three research pillars.

Table 1.1: Mapping of Publications to Dissertation Chapters

Chapter	Theme	Primary Publications
3	RAG Optimization	Papers 1–3 (PAKDD Workshops, IEEE CAI, NAACL)
4	Agentic AI	Papers 4–9 (ICDSAAI, ICAIBD, AAAI Workshops, PAKDD)
5	Enterprise Deployment and Applications	Papers 10–17 (IJCNN, IEEE ICDM Workshops, IEEE SSCI, IEEE IS, PAKDD)

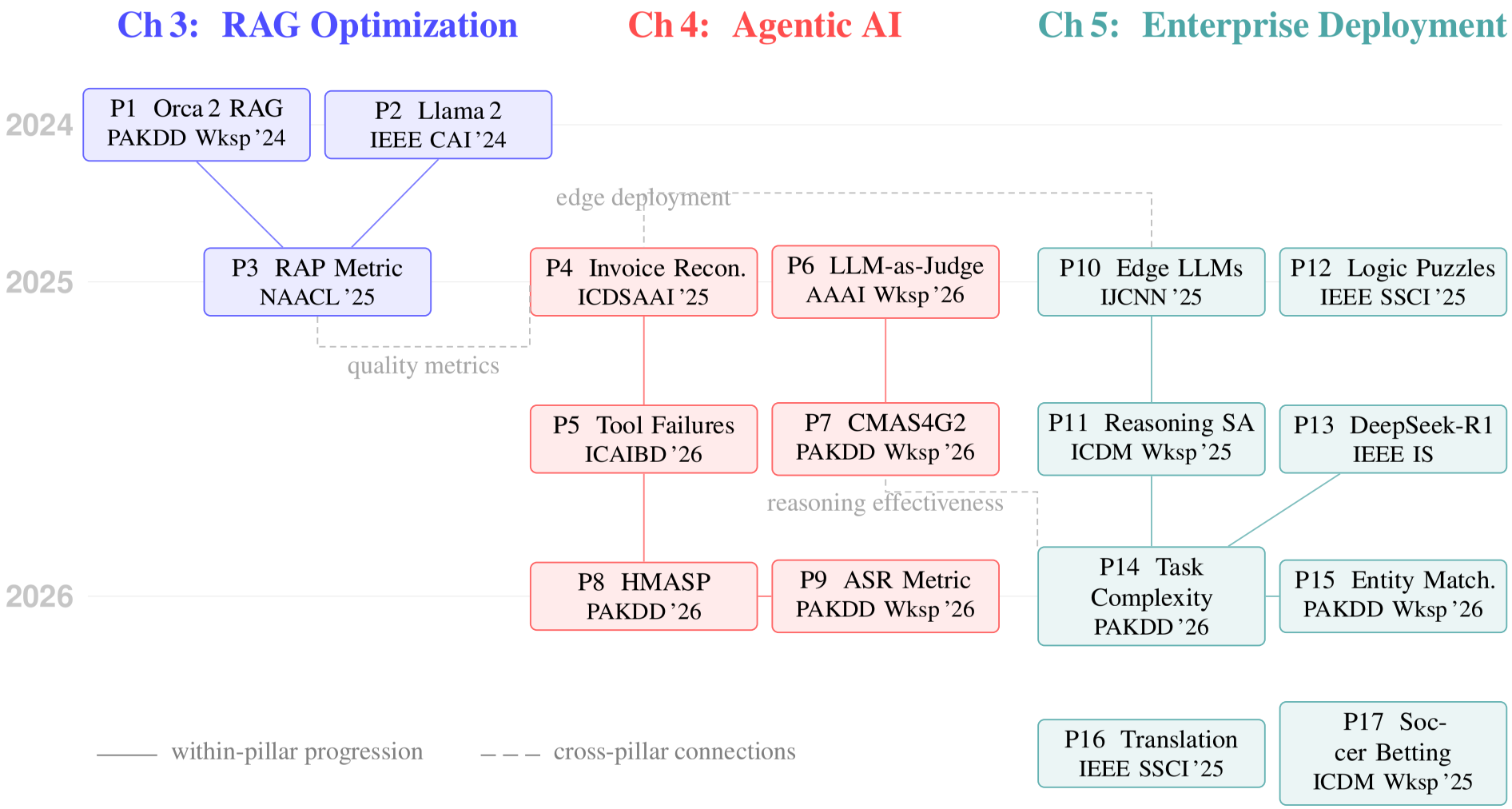


Figure 1.1: Research progression across the 17 publications. Papers are arranged chronologically (top to bottom) and grouped by research pillar (left to right). Solid arrows indicate within-pillar build-on relationships; dashed arrows show cross-pillar connections where findings from one thread informed another.

1.3.4 Research Journey: From Open-Weight LLMs to Enterprise AI Systems

The research presented in this dissertation was shaped by the practical demands of deploying generative AI within a large financial services enterprise. Figure 1.1 traces the chronological progression; the narrative below explains the motivations and industrial context that drove each phase.

Phase 1: Securing Enterprise LLM Adoption through RAG (2023–2024). When this research began in mid-2023, transmitting proprietary data to third-party API endpoints—including those of OpenAI—was deemed an unacceptable security risk for financial institutions. The release of Llama 2 in July 2023 opened a new pathway: enterprises could deploy open-weight models on their own infrastructure, retaining full control over data residency and access. Evaluating whether these models were usable for enterprise workloads therefore became the immediate research priority.

Retrieval-Augmented Generation was the first production-ready pattern to emerge, because it allows LLMs to be grounded in proprietary knowledge bases without fine-tuning. Papers 1 and 2 benchmarked leading open-weight models (Orca-2 and Llama-2) on RAG tasks and demonstrated that, with proper configuration, they could match or exceed proprietary alternatives on consumer-grade hardware. A critical gap, however, was the absence of metrics that captured operational quality: models that scored well on standard benchmarks often produced repetitive or degenerate outputs in production. Paper 3 addressed this by introducing the Repetition-Aware Performance (RAP) metric and the ReDA tuning algorithm, which reduced repetition by up to 93.1% while preserving downstream accuracy. RAP was subsequently adopted in the Mastercard Assistant Platform (MAP), the company's internal RAG-powered knowledge assistant, where it continues to serve as the primary output-quality metric in production.

Phase 2: Pivoting to Agentic AI and Edge Deployment (2024–2026). By late 2024, RAG applications such as MAP had matured sufficiently to transition from the R&D team to commercialisation. Two concurrent developments shifted the research focus. First, the broader industry began exploring agentic architectures—systems in which LLMs autonomously orchestrate multi-step workflows by invoking tools and coordinating with other agents. Second, edge deployment emerged as a viable strategy: Paper 10 confirmed that models up to 14B parameters could run on 16 GB consumer GPUs with practical inference speeds, enabling privacy-preserving, cloud-independent operation. This edge feasibility finding directly informed the hardware configuration for subsequent agentic prototypes.

Three progressively complex agentic use cases were investigated, each motivated by a real operational workflow. The first was invoice reconciliation (Papers 4 and 5): a three-agent system that automates email parsing, database lookup, and financial matching. Deploying this system on edge hardware exposed a rich taxonomy of tool-invocation failures—twelve categories spanning OCR, database query, and database update stages—whose severity scaled inversely with model size. The second use case was automated acceptance testing (Papers 6 and 7): an LLM-as-a-Judge framework for evaluating Gherkin test scenarios, extended into CMAS4G2, a cooperative multi-agent system for generating those tests. The third was end-to-end payment processing (Papers 8 and 9): the Hierarchical Multi-Agent System for Payments (HMASP), whose four-level agent hierarchy required a new trajectory-level metric—the Agentic Success Rate (ASR)—to capture workflow fidelity beyond simple task success.

The cross-pillar arrows in Figure 1.1 reflect how insights flowed across these streams. Quality metrics developed for RAG evaluation (P3) informed the rubric design for agentic output assessment (P4). Edge deployment benchmarks (P10) directly shaped the hardware strategy for the invoice reconciliation system (P4). And findings on task-complexity-dependent reasoning effectiveness (P14) were validated in the agentic context of CMAS4G2 (P7), where structured reasoning enabled transformative gains for smaller models but excessive reasoning depth caused catastrophic

degradation through context exhaustion. Collectively, these six papers—together with their evaluation frameworks—are now feeding into an *Agentic Evaluation and Optimisation* initiative within the enterprise, aimed at establishing systematic methods for assessing and improving multi-agent systems before production deployment.

Phase 3: Reasoning, Fine-Tuning, and Cross-Domain Applications (2025–2026). Two key techniques emerged as essential for leveraging open-weight models in enterprise settings: few-shot prompting and fine-tuning. Few-shot prompting proved remarkably efficient for capable reasoning models—DeepSeek-R1 achieved 99.31% accuracy on binary sentiment with just five examples (Paper 13), while Qwen2.5-72B surpassed GPT-4o on logical reasoning with only ten shots (Paper 12). However, few-shot prompting has inherent scalability limitations: each additional example lengthens the input context and increases compute cost, and Paper 12 found that several models could not complete evaluation at higher shot counts due to prohibitive inference times. Fine-tuning addresses both bottlenecks—reliability *and* efficiency. Paper 12 showed that fine-tuning improved the Valid Classification Ratio from as low as 1.17% to 100% within 0.2 training epochs, delivering output conformance that few-shot prompting could not guarantee across all model families, while eliminating the context-length overhead of in-context examples entirely.

As reasoning-augmented models (OpenAI o1, DeepSeek-R1, Qwen3) gained prominence, understanding how reasoning affects performance, efficiency, and cost across enterprise tasks became critical. Papers 11–14 systematically investigated these trade-offs: Paper 11 introduced the Reasoning Intensity metric, Paper 13 demonstrated that architecture matters more than scale (DeepSeek-R1:32b outperforming its 70b variant), and Paper 14 established the task-complexity hypothesis—reasoning helps complex tasks but can degrade simple ones—across 504 model–task configurations. Papers 12, 16, and 17 applied these insights to diverse domains (logical reasoning, machine translation, and sports predictive analytics), validating the generalisability of the findings.

Paper 15 (Entity Matching with LLMs at Scale) bridges the research back to immediate production impact. Mastercard's Generative AI Merchant Enrichment (GAIME) product uses LLM-based entity matching to map raw transaction descriptors to structured merchant records at scale. The accuracy–cost–reasoning trade-offs characterised in Paper 15 directly inform model selection and configuration decisions for GAIME's matching pipeline. Ongoing work extends these findings through a two-stage optimisation strategy: large frontier models are first used to iteratively refine extraction prompts, and the best-performing prompt is then used to generate training data for parameter-efficient fine-tuning (LoRA/QLoRA) of sub-4B-parameter models. Early results show that models as small as 0.8 B parameters, after targeted fine-tuning, can approach the extraction accuracy of models orders of magnitude larger—opening a path to deployment on low-cost hardware and latency-sensitive production pipelines, and closing the loop between academic investigation and deployed enterprise AI.

1.4 Research Contributions

This dissertation makes the following primary contributions to the field of enterprise AI:

1.4.1 Novel Evaluation Metrics

A key finding across this research is that traditional accuracy metrics are insufficient for enterprise deployment—operational reliability, output quality, and efficiency require dedicated measurement frameworks. This dissertation introduces seven complementary metrics:

1. **Repetition-Aware Performance (RAP):** A metric that quantifies and integrates repetition penalty into model evaluation, computed as $\text{RAP} = P \times F(\text{RR})$ where P is standard performance (F1, BERT-F1, COMET) and $F(\text{RR}) = (1 - \text{RR})^3$

is the cubic penalty function. RAP enables effective tuning of the repetition penalty parameter, achieving optimal balance between repetition reduction (up to 93.1%) and performance preservation. This addresses a critical gap in evaluating open-source LLMs, where repetition issues are prevalent but previously unquantified [74].

2. **Valid Classification Ratio (VCR):** A reliability metric defined as VCR $= \frac{\text{Valid Classifications}}{\text{Total Classifications}} \times 100\%$, measuring the proportion of model outputs that match predefined labels. This metric reveals that fine-tuning dramatically improves VCR from as low as 1.17% to 100% within 0.2 epochs, providing practitioners with a concrete measure of deployment readiness [69].

3. **Evaluation Completion Rate (ECR@1):** An operational reliability metric measuring first-attempt evaluation success, defined as ECR@1 $= \frac{\text{\# successful first attempts}}{N} \times 100\%$ where N is the total number of evaluations. ECR@1 reveals reliability variations from 85.4% to 100% across models, with direct cost implications via retry overhead—a consideration invisible to accuracy-only evaluation. Models with low ECR@1 incur substantial operational costs; for example, GPT-OSS 20B (high) at 85.4% ECR@1 requires an average of 1.17 attempts per evaluation [76].

4. **Agentic Success Rate (ASR):** A trajectory-level metric for multi-agent systems that evaluates workflow fidelity by comparing observed and expected agent execution sequences at the *transition* (bigram) level, rather than focusing only on final task outcomes. Drawing on process-mining conformance checking, ASR represents each consecutive agent handoff as a transition and decomposes performance into Transition Recall (TR), which measures completeness, and Transition Precision (TP), which measures efficiency. Given expected trajectory $E = [e_1, \ldots, e_n]$ and observed trajectory $O = [o_1, \ldots, o_m]$, let B_E and B_O denote their respective transition multisets:

$$\text{TR} = \frac{|B_E \cap B_O|}{|B_E|}, \quad \text{TP} = \frac{|B_E \cap B_O|}{|B_O|}, \quad \text{ASR} = F_1(\text{TR}, \text{TP})$$

An ASR of 100% indicates perfect transition alignment. Low Transition Recall signals skipped handoffs; low Transition Precision signals redundant ones. Applied to HMASP across 18 LLMs and 90,000 task instances (5 repeats per model–scenario pair), ASR reveals that most evaluated models systematically skip a CPA-mediated confirmation checkpoint during payment checkout—a procedural deviation invisible to both TSR and HF1, with ASR gaps ranging from 0.01% to 2.9%. The root cause is consistent: for unambiguous payment intents, models infer user consent and bypass the explicit confirmation step, executing 9 agent hops instead of the expected 11 (TR $= 80\%$, TP $= 100\%$, ASR $= 88.9\%$ per affected sample). This has regulatory significance: PCI-DSS requires auditable workflows, and bypassing confirmation checkpoints may violate compliance logging even when task outcomes are correct. Prompt refinements and deterministic routing guards guided by ASR diagnostics yield $+24.2$ pp average TSR improvement across 11 models, demonstrating that trajectory-level evaluation is essential in regulated domains. Although introduced for multi-agent payment workflows, ASR generalises to arbitrary agentic architectures wherever expected execution trajectories can be specified [78, 80].

5. **Reasoning Intensity (RI):** A metric quantifying the proportion of computational effort devoted to reasoning, computed as RI $= \frac{\text{MRT}}{\text{MRT}+\text{MOT}}$ where MRT is Mean Reasoning Tokens and MOT is Mean Output Tokens. RI identifies three operational regimes—minimal (RI < 0.5), balanced ($0.5 \leq$ RI ≤ 0.85), and intensive (RI > 0.85)—providing deployment guidance for latency-sensitive applications. Models in the balanced regime achieve optimal accuracy-efficiency trade-offs, with gpt-oss:120b (medium) achieving 86.2% F1 with RI = 0.728 [75].

6. **Translation Completeness Ratio (TCR):** A reliability metric for machine translation defined as TCR $= \frac{\text{Complete Translations}}{\text{Total Entries}} \times 100\%$, measuring the proportion of outputs fully rendered in the target language. This metric addresses the observation that LLMs sometimes produce outputs with untranslated segments, enabling

quantification of translation reliability beyond quality scores [70].

7. **Characters per Second (CPS):** An efficiency metric for translation tasks measuring CPS $= \frac{\text{Source Characters}}{\text{Translation Time}}$, enabling comparison of translation throughput across models and configurations. CPS reveals that smaller fine-tuned models (Phi-3.5-mini, InternLM2.5-7B) offer 2–3$\times$ speed improvements with acceptable quality trade-offs [70].

1.4.2 Architectural Innovations

This research contributes reusable architectural patterns for enterprise workflow automation with quantified reliability guarantees:

1. **Multi-Agent Invoice Reconciliation System:** A three-agent architecture (Email Agent, Data Engineering Agent, Reconciliation Agent) leveraging edge-deployed LLMs with the LangGraph framework. The system achieves 99.9% success rate while consuming only 523 J per task on RTX 4090 for smaller models, demonstrating that complex business processes can be reliably automated with accessible hardware. A comprehensive structured error taxonomy—spanning OCR tool failures (`OCR_TOOL_NOT_INITIALIZED`, `OCR_TOOL_ARGS_MISMATCH`, `OCR_TOOL_ERROR`, `OCR_TOOL_RESULT_MISMATCH`), database query failures (`DB_QUERY_TOOL_*`), and database update failures (`DB_UPDATE_TOOL_*`)—enables systematic diagnosis of tool invocation reliability, revealing that tool initialization failures constitute the primary reliability bottleneck with catastrophic rates in small models (89% errors at 3B) absent at 32B scale [73, 81].

2. **Hierarchical Multi-Agent System for Payments (HMASP):** The first LLM-based multi-agent system for end-to-end agentic payment workflows, featuring a four-level hierarchy: (1) Conversational Payment Agent (CPA) as the entry point handling all external requests, (2) Supervisor Agents for domain-specific workflow orchestration, (3) Routing Agents for final decision-making on workflow

triggering, and (4) Process Summary Agents for outcome reporting. HMASP incorporates architectural patterns including shared state variables for information isolation, decoupled message states preventing hallucination of critical payment data, and structured handoff protocols enabling seamless agent coordination. The system achieves 97.0% task success rate with Qwen2.5:32b matching GPT-4.1 performance, establishing a template for financial process automation [80].

3. **Cooperative Multi-Agent System for Gherkin Generation (CMAS4G2):** A multi-agent framework for automated acceptance test generation using Microsoft's AutoGen Swarm, coordinating Supervisor, Validator, and Quality Assurance agents. CMAS4G2 demonstrates that open-weight models approach proprietary performance (GPT-OSS 20B: 87.6% overall success, 85.4% coverage vs. Claude Sonnet 4: 90.4%, 91.6%—a gap of only 2.8 percentage points). Critically, reasoning configuration profoundly impacts coordination: structured reasoning enables transformative gains for smaller models (Qwen3-8B: 49.9% $\rightarrow$ 77.5% overall success) while excessive reasoning depth causes catastrophic degradation through context exhaustion [**?**].

4. **LLM-as-a-Judge (LAJ) Framework:** A production-ready, rubric-driven framework for evaluating Gherkin acceptance tests with structured JSON outputs. Systematic evaluation across 20 model configurations over 500 runs reveals cost spans of 175$\times$ ($0.45–$78.96 per 1K evaluations), enabling informed model selection based on budget constraints. GPT-4o Mini emerges as production-optimal, achieving best-in-class accuracy (6.07 MAAE), high reliability (96.6% ECR@1), and 78$\times$ cost reduction versus GPT-5 (high reasoning) [76].

1.4.3 Empirical Findings

Systematic benchmarking across models, tasks, and hardware configurations yields actionable insights for enterprise deployment:

1. **Open-Source Competitiveness:** Properly optimized open-source models can match or exceed proprietary alternatives. Orca-2-13b achieves superior overall scores in RAG applications compared to larger proprietary models while operating efficiently on consumer-grade GPUs, reducing barriers to enterprise LLM adoption. On logical reasoning tasks, Qwen2.5-72B achieves 80.88% F1 with only 10-shot prompting, surpassing GPT-4o's 80.36% [66, 72, 69].

2. **Architecture Over Scale:** Contrary to conventional wisdom, larger models do not always outperform smaller counterparts. DeepSeek-R1:32b (81.7% F1) substantially outperforms DeepSeek-R1:70b (74.2% F1), and a 32B Qwen2.5-based variant outperforms a 70B Llama-based model by 6.69 percentage points on sentiment classification. Similarly, qwen3:8b achieves the highest performance (84.8% F1) in its family, surpassing larger 14B, 30B, and 32B variants. This finding has significant cost and deployment implications [67, 75].

3. **Task Complexity Determines Reasoning Effectiveness:** Comprehensive evaluation across 504 configurations reveals that reasoning effectiveness is systematically task-dependent. Binary sentiment classification shows degradation up to -19.9 F1 percentage points when reasoning is applied, while complex 27-class emotion recognition benefits with gains up to +16.0 pp. This task-complexity hypothesis—validated across seven model families including DeepSeek-R1, Qwen3, Granite, and Magistral—provides principled guidance for model selection: base/non-thinking models dominate for simple tasks, while reasoning justifies its 2.1×–54× computational overhead only for complex multi-class problems. This finding directly informs the reasoning configuration choices in agentic systems (Chapter 4), where CMAS4G2 demonstrates that structured reasoning enables transformative gains for smaller models while excessive depth causes catastrophic degradation [71].

4. **Quantization and Platform Optimization:** Ollama's Q4_K_M quantization outperforms bitsandbytes 4-bit implementation, with Llama3.1:8b achieving

3.28% higher F1 score (92.87% vs. 89.50% baseline) while enabling deployment on consumer hardware. Additionally, WSL provides up to 21× speedup over native Windows for smaller models (qwen2.5:0.5b: 16,933 vs. 816 tokens/s), offering concrete deployment guidance for Windows-based enterprise environments. These platform optimization findings directly impact the edge deployment strategies employed in the invoice reconciliation system (Chapter 4) [68].

5. **Energy Efficiency Trade-offs:** Large models (70B+) achieve high F1 scores (96–97%) but consume 1,283–1,876 Joules per inference. Small models (1.5B–3B) achieve the highest performance-to-power ratios (0.161–0.163 %/J), with qwen2.5:3b reaching 91.00% F1 while consuming only 558 Joules. Medium-sized models (7B–32B) offer optimal balance, with qwen2.5:32b achieving 96.75% (±0.11%) F1 with practical inference speeds [68].

6. **Consumer Hardware Viability:** Models up to 14B parameters are feasible on 16GB consumer GPUs. For reasoning tasks, qwen3:8b offers optimal accuracy-efficiency trade-offs (84.8% F1, 13GB VRAM, 8.35s inference), enabling privacy-preserving local deployment without significant performance penalties compared to cloud alternatives [75].

7. **Fine-Tuning Superiority for Reliability:** Fine-tuning consistently outperforms few-shot prompting for deployment reliability, achieving 100% VCR within 0.2 epochs while few-shot VCR varies dramatically (1.17%–100% depending on model). However, for capable reasoning models, few-shot learning can be remarkably efficient—DeepSeek-R1 achieves 99.31% accuracy on binary sentiment with just 5 shots, an 8× improvement over GPT-4o's 40-shot requirement [69, 67].

8. **Temporal Robustness Challenges:** All evaluated models exhibit consistent 10–17 percentage point F1 degradation on contemporary data compared to original benchmarks, though reasoning intensity remains stable (±0.02). This suggests that while models' reasoning processes are robust, their learned patterns require

periodic updating to maintain accuracy on evolving content [75].

9. **Reasoning Effort Has Opposite Effects Across Model Families for Entity Matching:** The largest-scale LLM entity matching evaluation to date—46 model configurations across 8 benchmark datasets—reveals that the optimal reasoning level is family-dependent rather than universal. For GPT-5, low reasoning is optimal (93.14 F1 at \$6.25 vs. 92.14 F1 at \$24.50 for high); for open-weight models, high reasoning is critical (GPT-OSS:120b improves from 90.11 to 92.10 F1 with increasing effort); and for Claude models, thinking produces asymmetric effects (+10.13 F1 for Sonnet 4.5 but -0.25 for Opus 4.5). Open-weight models rival proprietary APIs: GPT-OSS:120b (92.10 F1) outperforms prior-generation proprietary baselines (GPT-4.1: 90.36, GPT-4o: 89.58) at 20$\times$ lower cost than GPT-5(high). Pareto frontier analysis identifies sharp diminishing returns beyond \$3 total cost, with the ultra-budget tier (<\$0.10) already achieving 87–89 F1—providing actionable model selection guidance across budget constraints [82].

10. **Cross-Domain Optimization Patterns:** Task-specific optimization dramatically improves practical outcomes. F1 tuning for sports betting prediction produces 2–3$\times$ higher returns than accuracy tuning (e.g., Random Forest + GPT-4.1 achieves 279.48% return with F1 tuning vs. 112.34% with accuracy tuning), demonstrating that metric selection must align with downstream objectives [146].

1.4.4 Industry Impact

Several contributions have already transitioned from research to production or are actively informing enterprise systems. The RAP metric and ReDA algorithm are deployed in the Mastercard Assistant Platform (MAP), where they serve as the primary output-quality gate for the company's internal RAG-powered knowledge assistant. The entity matching accuracy–cost–reasoning trade-offs characterised in Paper 15 directly inform model selection for Mastercard's Generative AI Merchant Enrichment (GAIME) product, and ongoing LoRA/QLoRA fine-tuning of sub-4B-

parameter models is extending these findings toward low-cost, latency-sensitive production serving. The ASR metric, by exposing compliance-relevant workflow deviations invisible to conventional metrics, provides the trajectory-level visibility that regulated industries require as they adopt agentic payment capabilities such as Mastercard's Agent Pay; ASR-guided engineering has already demonstrated $+24.2$ pp average task success improvement across 11 models in HMASP.

More broadly, this research reduces barriers to enterprise LLM adoption in three ways. First, systematic benchmarking across 17 papers consistently demonstrates that properly configured open-weight models can match or exceed proprietary alternatives—from RAG (Orca-2 surpassing larger models), to entity matching (GPT-OSS:120b outperforming GPT-4.1 at 20× lower cost), to multi-agent coordination (Qwen2.5:32b matching GPT-4.1 on HMASP)—enabling enterprises to retain full control over data residency and access. Second, the edge deployment findings confirm that models up to 14B parameters are viable on consumer GPUs ($5,000–$10,000 hardware), with platform optimisations (Ollama quantisation, WSL) delivering up to 21× speedups, making privacy-preserving local deployment practical. Third, the seven complementary metrics introduced—RAP, VCR, ECR@1, ASR, RI, TCR, and CPS—together with the cost-performance Pareto analysis spanning 175× cost ranges, equip practitioners with a comprehensive toolkit for evaluating AI system reliability across operational dimensions invisible to traditional accuracy measures.

1.5 Structure of the Dissertation

The remainder of this dissertation is organized as follows:

Chapter 2: Literature Review provides a comprehensive survey of foundational concepts in generative AI and LLMs, implementation approaches for enterprise deployment, and existing research on RAG optimization, agentic AI systems, and diverse enterprise applications.

Chapter 3: Optimizing LLMs for Retrieval-Augmented Generation presents

our research on RAG system optimization, including comparative evaluation of LLMs demonstrating open-source competitiveness [66, 72], the RAP metric and ReDA algorithm for output quality assessment with up to 93.1% repetition reduction [74], and practical deployment guidelines.

Chapter 4: Agentic AI for Business Automation introduces our multi-agent system architectures for workflow automation, with detailed analysis of the invoice reconciliation system achieving 99.9% success rate and the structured error taxonomy for systematic reliability diagnosis [73, 81], the HMASP four-level hierarchy for payment processing with 97.0% task success [80], the CMAS4G2 framework demonstrating reasoning configuration impacts [**?**], and the LLM-as-a-Judge framework with novel reliability metrics spanning 175× cost range [76].

Chapter 5: Enterprise Deployment and Application Strategies examines three interconnected themes: (1) edge deployment strategies demonstrating WSL speedups of up to 21× and optimal model sizing (7B–32B) for performance-efficiency balance [68, 75]; (2) reasoning enhancement techniques including few-shot prompting achieving 91.39% F1 with transparent reasoning traces, fine-tuning for reliability [67, 69], and task-complexity-dependent reasoning effectiveness across 504 configurations [71]; and (3) cross-domain applications spanning machine translation with novel TCR and CPS metrics [70], sports predictive analytics demonstrating 279.48% betting returns through LLM-enhanced sentiment features [146], and the largest-scale LLM entity matching evaluation to date (46 configurations, 8 datasets) revealing family-dependent reasoning effects and cost-performance Pareto guidance [82].

Chapter 6: Conclusions and Future Work synthesizes key findings across all three pillars, discusses limitations, and outlines directions for future research including extensions to additional enterprise domains and emerging model architectures.

Chapter 2

Literature Review

This chapter provides a comprehensive review of the foundational concepts, techniques, and prior research relevant to this dissertation. We begin with an overview of generative AI and large language models, then examine implementation approaches for enterprise deployment, and finally review existing work on the three core research pillars: RAG optimization, agentic AI systems, and enterprise deployment and application strategies including edge computing, reasoning enhancement, and cross-domain applications. Throughout, we identify specific research gaps that motivate the contributions presented in subsequent chapters.

2.1 Foundational Concepts: Generative AI and Large Language Models

2.1.1 Definition and Evolution of Generative AI

Generative AI represents a class of artificial intelligence models distinguished by their ability to generate new, realistic data instances that bear resemblance to the data on which they were trained [48]. This capability extends across various modalities, encompassing text, images, audio, video, and computer code. The evolution of generative AI has been significantly shaped by advancements in machine learning

techniques, with deep learning and neural networks playing a pivotal role in its progress.

Early milestones in generative modeling include Generative Adversarial Networks (GANs) [48] and Variational Autoencoders (VAEs) [99], which laid foundational principles for machine-generated content. These approaches demonstrated the potential for neural networks to learn complex data distributions and generate novel samples.

The emergence of transformer architectures [157] marked a paradigm shift in natural language processing and generative AI. The self-attention mechanism central to transformers enables models to capture long-range dependencies in sequential data more effectively than recurrent architectures. This innovation paved the way for the development of increasingly capable language models.

2.1.2 Historical Evolution of Language Models

The development of language models has progressed through several distinct phases:

Statistical Language Models (1980s–2000s): Early approaches relied on n-gram statistics to estimate the probability of word sequences. While computationally tractable, these models suffered from data sparsity and limited ability to capture semantic relationships [114].

Neural Language Models (2000s–2017): The introduction of neural network-based language models, beginning with Bengio et al.'s [12] feedforward architecture and later recurrent variants including LSTMs [58], enabled distributed representations of words and improved handling of long-range dependencies.

Transformer Era (2017–present): The transformer architecture [157] fundamentally changed language modeling by enabling efficient parallel processing and attention-based context modeling. Key developments include:

- **GPT Series:** OpenAI's Generative Pre-trained Transformer models demonstrated the power of unsupervised pre-training followed by task-specific fine-tuning [137,

138, 16].

- **BERT:** Bidirectional encoder representations enabled superior performance on understanding tasks through masked language modeling [34].
- **T5:** The text-to-text transfer transformer unified diverse NLP tasks under a common framework [140].
- **Modern LLMs:** GPT-4 [2], Claude [6], Llama [152, 51], DeepSeek-R1 [54], and others represent the current state of the art, with capabilities spanning reasoning, code generation, and multimodal understanding.

2.1.3 Architecture of Large Language Models

Modern LLMs are characterized by several architectural components and design choices:

Transformer Architecture

The transformer [157] employs multi-head self-attention and position-wise feed-forward networks. The self-attention mechanism computes attention weights as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where Q, K, and V represent query, key, and value matrices, and d_k is the dimension of the keys.

Multi-head attention extends this by learning multiple attention patterns in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2.2}$$

Architectural Variants

Table 2.1 summarizes the major architectural paradigms for LLMs.

Table 2.1: Comparison of LLM Architectural Paradigms

Type	Examples	Strengths	Primary Use Cases
Encoder-Only	BERT, RoBERTa	Strong understanding, bidirectional context	Classification, NER, QA
Decoder-Only	GPT, Llama, Claude	Excellent generation, scalable	Text generation, chat, code
Encoder-Decoder	T5, BART	Flexible, good for seq2seq	Translation, summarization
Mixture of Experts	Mixtral, DeepSeek-V3	Efficient scaling, specialization	Large-scale deployment

Scaling Laws and Their Limitations

Research has established empirical scaling laws relating model performance to compute, parameters, and training data [95, 59]. Key findings include:

- Performance improves predictably with increased compute, following power-law relationships
- Optimal training requires balanced scaling of model size and training data
- Emergent capabilities appear at certain scale thresholds

However, recent research challenges conventional assumptions about scale, revealing that performance does not always scale monotonically with model size. Architectural design, training strategies, and optimization techniques can outweigh parameter count. The Qwen2.5 technical report demonstrates that 32B Qwen2.5-based distillations can outperform 70B Llama-based variants [167], while evaluation studies show DeepSeek-R1:32b (81.7% F1) substantially outperforming DeepSeek-R1:70b (74.2% F1) on sentiment classification tasks [54].

2.1.4 Training Methodologies

Modern LLM training typically involves multiple phases:

Pre-training: Self-supervised learning on large text corpora using objectives like next-token prediction (causal language modeling) or masked language modeling. Pre-training datasets often comprise trillions of tokens from diverse sources [152].

Instruction Tuning: Fine-tuning on datasets pairing instructions with desired responses improves model ability to follow user intent [162, 127].

Alignment: Techniques including Reinforcement Learning from Human Feedback (RLHF) [23, 127] and Direct Preference Optimization (DPO) [139] align model behavior with human preferences.

2.2 Implementation Approaches for LLMs in Enterprises

2.2.1 Leveraging Pre-trained Models: Zero-shot and Few-shot Learning

Pre-trained LLMs can be applied to downstream tasks through various prompting strategies without modifying model weights:

Zero-shot Learning: The model is prompted with a task description and expected to generate appropriate responses without examples. This approach is effective for well-understood tasks but may struggle with domain-specific requirements [16].

Few-shot Learning: Providing several input-output examples in the prompt enables models to infer the desired behavior through in-context learning. Research has shown few-shot prompting can substantially improve performance on diverse tasks [16]. However, the optimal number of examples varies depending on model architecture and task complexity—larger models often approach peak performance with fewer examples, especially in simpler tasks, while smaller models benefit more from added context [16].

Chain-of-Thought Prompting: Including intermediate reasoning steps in exam-

ples encourages models to generate explicit reasoning chains, improving performance on complex tasks requiring multi-step inference [163].

2.2.2 Fine-tuning Approaches

When prompting proves insufficient, fine-tuning adapts model weights to specific tasks or domains:

Full Fine-tuning: Updating all model parameters on task-specific data. While effective, this approach requires substantial computational resources and may lead to catastrophic forgetting [115].

Parameter-Efficient Fine-Tuning (PEFT): Methods that update only a small subset of parameters:

- **LoRA (Low-Rank Adaptation):** Freezes pre-trained weights and injects trainable rank decomposition matrices into transformer layers [65].
- **QLoRA:** Combines LoRA with 4-bit quantization for memory-efficient fine-tuning [33].
- **Adapter Methods:** Insert small trainable modules between frozen layers [63].

These parameter-efficient approaches are particularly relevant for enterprise deployment, enabling domain adaptation without the computational overhead of full fine-tuning.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) addresses LLM limitations by incorporating external knowledge during inference [106]. This section reviews the RAG paradigm, which forms the foundation for Chapter 3.

2.3.1 RAG Architecture and Components

The RAG paradigm combines retrieval and generation through several stages:

1. **Document Processing:** Ingesting and chunking source documents into manageable units
2. **Embedding Generation:** Creating vector representations using models like Sentence-BERT [143] or Instructor [150]
3. **Indexing:** Storing embeddings in vector databases for efficient similarity search
4. **Retrieval:** Identifying relevant documents through similarity search at query time
5. **Augmented Generation:** Conditioning LLM outputs on retrieved context

2.3.2 Advantages for Enterprise Applications

RAG offers several advantages for enterprise applications:

- **Knowledge Currency:** External knowledge bases can be updated without model retraining
- **Reduced Hallucination:** Grounding responses in retrieved evidence improves factual accuracy [45]
- **Domain Adaptation:** Organization-specific knowledge can be incorporated without fine-tuning
- **Traceability:** Retrieved sources provide citations for generated content

2.3.3 RAG Evaluation Frameworks

Evaluating RAG systems requires metrics addressing both retrieval and generation quality:

RAGAS [38] provides automated evaluation through:

- Faithfulness: Consistency of generated responses with retrieved context
- Answer Relevancy: Alignment of responses with user queries
- Context Relevancy: Quality of retrieved documents

ARES [145] uses synthetic training data to fine-tune lightweight evaluation models, enabling assessment with minimal human annotation.

However, existing RAG evaluation frameworks focus primarily on semantic accuracy while overlooking critical user experience factors such as output verbosity, repetition, and formatting quality. Traditional metrics like BLEU [130], ROUGE [113], and even neural metrics like BERTScore [171] do not capture these dimensions [44]. This gap motivates the development of new evaluation approaches that integrate output quality assessment into performance measurement, as addressed in Chapter 3.

2.3.4 Challenges in RAG Optimization

Despite its advantages, RAG presents several optimization challenges:

Output Quality: Open-source LLMs frequently produce repetitive, verbose, or poorly structured outputs that degrade user experience [44, 60]. The repetition problem can be particularly severe—analysis has shown that some models generate responses where the vast majority of content is repetitive, with repeated text constituting over 90% of overall answer length in extreme cases [44]. Repetition occurs in two primary forms: non-word character (NWC) repetition (repeated whitespaces, punctuation, and symbols) and text repetition (repeated phrases or sentences). Existing metrics inadequately capture these user experience factors, necessitating novel approaches that quantify repetition and integrate it into performance assessment.

Hyperparameter Sensitivity: RAG performance depends critically on parameters including chunk size, retrieval count, and generation settings. The repetition penalty parameter (RPP) [97] significantly impacts output quality by penalizing previously seen tokens during sampling, but traditional metrics like precision and

BLEU overlook repetition, making optimal tuning challenging. Research has shown that while higher RPP values generally reduce repetition, excessive application can result in incomplete or fragmented responses, highlighting the need for metrics that enable balanced optimization.

These gaps motivate research on RAG optimization presented in Chapter 3, including the development of the Repetition-Aware Performance (RAP) metric and Repetition Detection Algorithm (ReDA) that explicitly account for repetition in model outputs.

2.4 Agentic AI and Multi-Agent Systems

Agentic AI extends LLM capabilities beyond single-turn interactions to autonomous, goal-directed behavior. This section reviews foundations relevant to Chapter 4.

2.4.1 Foundations of Agentic AI

Agentic AI systems are characterized by several distinguishing capabilities [20, 1]:

- **Tool Use:** Ability to interact with external tools, APIs, and databases [147, 148]
- **Planning:** Decomposing complex goals into executable sub-tasks
- **Memory:** Maintaining state across interactions for long-term objectives
- **Reflection:** Self-evaluation and strategy adjustment

These capabilities enable LLMs to move beyond passive question-answering toward active problem-solving and workflow execution.

2.4.2 Multi-Agent Architectures

Multi-agent systems leverage specialized agents for distinct subtasks, coordinated through various paradigms [158, 153, 55]:

Hierarchical Systems: Supervisor agents delegate tasks to specialized workers, enabling complex workflow automation [62]. This architecture is particularly suited for business processes with clear task decomposition, such as payment processing workflows requiring card management, transaction handling, and authorization coordination.

Collaborative Systems: Agents engage in peer-to-peer communication to solve problems collectively [107]. Collaboration enables diverse expertise to be brought to bear on complex problems, with structured handoff mechanisms enabling seamless agent transitions while preserving contextual information.

Competitive Systems: Adversarial dynamics between agents can improve output quality through debate and critique [37].

2.4.3 Orchestration Frameworks

Several frameworks support multi-agent system development:

LangGraph [87, 88]: Graph-based workflow orchestration enabling flexible agent coordination with support for cycles, branching, and state management. LangGraph's architecture supports modular and adaptive task pipelines, making it particularly well-suited for complex applications requiring customizable and context-aware processing. The framework enables architectural patterns including shared state variables for information isolation, decoupled message states preventing data hallucination, and structured handoff protocols facilitating agent coordination.

AutoGen [166, 36]: Microsoft's framework for multi-agent conversations and task execution with human-in-the-loop capabilities, supporting swarm patterns and structured handoffs [118, 117]. AutoGen's Swarm pattern enables coordination among specialized agents through conditional handoff logic, where agents decide handoff targets based on input characteristics.

CrewAI [27]: Role-based agent coordination with support for hierarchical and sequential workflows.

2.4.4 Enterprise Applications

Agentic AI has demonstrated effectiveness across enterprise applications:

- **Software Development:** Automated coding, testing, and debugging workflows [92, 89]
- **Data Analysis:** Autonomous data science pipelines [61]
- **Document Processing:** Automated extraction, validation, and workflow routing
- **Process Automation:** Intelligent virtual assistants for business process workflows [53]
- **E-Commerce:** Conversational agents with grounded responses [170]

Financial services—including payment processing, fraud detection, and transaction management—represent emerging applications for agentic AI. However, end-to-end agentic payment workflows remain largely unexplored, with no established reference framework for implementing such systems using LLM-based multi-agent architectures.

2.4.5 LLM-as-a-Judge Paradigm

A related development is the emergence of LLM-based evaluation, where language models assess the quality of outputs from other models or systems. The "LLM-as-a-Judge" paradigm [174] has gained traction across various evaluation contexts, including natural language generation [112], creative writing assessment, and software testing [5]. However, systematic investigation of LLM judges for software testing—particularly regarding accuracy, cost, and operational reliability—remains limited.

2.4.6 Challenges in Agentic Systems

Enterprise deployment of agentic systems presents unique challenges:

Reliability and Failure Diagnosis: Agent failures may stem from poor system design rather than model limitations [96, 19]. Understanding when and why agents fail to invoke tools correctly is critical for production deployment. Recent work by Kokane et al. [102] demonstrates the value of granular error taxonomies for single-agent tool use, yet these frameworks remain insufficient for multi-agent contexts where failures cascade through coordination protocols.

Safety: Autonomous actions require careful guardrails and human oversight mechanisms [144].

Cost: Multi-agent workflows can accumulate significant API costs through extensive LLM calls.

Latency: Sequential agent operations may introduce unacceptable delays for real-time applications.

Evaluation: Standard accuracy metrics may be insufficient for assessing end-to-end workflow automation [91, 134], necessitating new metrics that capture pipeline-level success rates rather than individual task accuracy.

These challenges motivate research on agentic AI for business automation presented in Chapter 4, including novel reliability metrics such as the Agentic Success Rate (ASR), which decomposes tasks into atomic steps, and Evaluation Completion Rate (ECR@1), which measures first-attempt success rates with direct cost implications.

2.5 Edge Deployment of Large Language Models

Edge deployment—executing LLMs locally on consumer-grade or enterprise hardware—addresses critical constraints around privacy, latency, and cost. This section reviews foundations relevant to the edge deployment strategies discussed in

Chapter 5.

2.5.1 Motivation for Edge Deployment

Enterprise requirements often necessitate local LLM deployment [176, 116]:

- **Data Privacy:** Sensitive data cannot leave organizational boundaries, particularly in regulated industries such as finance and healthcare
- **Latency:** Real-time applications require low-latency inference unavailable through cloud APIs
- **Cost:** Cloud API costs may be prohibitive for high-volume applications
- **Reliability:** Local deployment eliminates external dependencies and ensures availability
- **Sustainability:** Edge deployment can reduce the environmental footprint of AI systems by optimizing energy consumption [35]

The democratization of LLMs through edge deployment holds significant potential for social good, enhancing privacy in sensitive domains, reducing environmental footprint, and expanding access to AI in regions with limited cloud infrastructure [176].

2.5.2 Model Optimization Techniques

Several techniques enable LLM deployment on resource-constrained hardware:

Quantization: Reducing precision of model weights from FP32 to lower bit-widths (INT8, INT4) substantially reduces memory requirements and improves inference speed [32, 43]. Recent advances in quantization-aware training and post-training quantization enable aggressive compression with minimal accuracy loss.

Model Pruning: Removing less important weights or structures reduces model size while maintaining performance [42, 98]. Both structured and unstructured pruning approaches have been applied to transformer architectures.

Knowledge Distillation: Training smaller "student" models to mimic larger "teacher" models transfers capabilities to more deployable architectures [57, 52, 64, 54].

Efficient Attention: Techniques like Flash Attention [31] reduce memory overhead of attention computation, enabling longer context lengths on limited hardware.

2.5.3 Deployment Frameworks

Frameworks supporting edge LLM deployment include:

Ollama [125]: Simplified local deployment with automatic optimization, cross-platform support (macOS, Linux, Windows), and an intuitive API for model management.

llama.cpp [46]: CPU-optimized inference for quantized models with broad hardware support.

vLLM [103]: High-throughput serving with PagedAttention for efficient memory management.

MLX [7]: Apple Silicon-optimized inference framework leveraging unified memory architecture.

2.5.4 Hardware Considerations

Edge deployment requires careful hardware selection balancing performance, cost, and power consumption [22, 109]:

- **Enterprise GPUs:** NVIDIA H100 (96GB HBM3) provides maximum performance for enterprise workloads; NVIDIA RTX A6000 Ada (48GB VRAM) provides headroom for larger models at approximately $10,000 [123]

- **Consumer GPUs:** NVIDIA RTX 4090 (24GB VRAM) enables deployment of models up to 70B parameters with quantization at approximately $5,000 [124]
- **Apple Silicon:** M3/M4 Max chips with unified memory provide efficient inference for medium-sized models at approximately $4,000, representing viable edge deployment options [41, 85, 29]
- **Edge Devices:** NVIDIA Jetson platforms enable deployment in IoT and embedded contexts
- **CPU-Only:** Quantized models can run on commodity hardware, albeit with reduced throughput

2.5.5 Research Gaps in Edge Deployment

Despite advances in optimization techniques, several gaps remain:

- Systematic benchmarking across diverse hardware platforms is limited, particularly for energy efficiency metrics
- Guidelines for model selection based on hardware constraints and task requirements are lacking
- Trade-offs between quantization methods require further investigation across different model families
- Cross-platform consistency of quantized models needs better characterization
- The impact of platform configuration (e.g., WSL vs. native Windows) on inference performance requires more comprehensive study

These gaps motivate research on edge deployment strategies presented in Chapter 5, including comprehensive evaluation across six hardware platforms and the introduction of energy efficiency metrics.

2.6 Reasoning in Large Language Models

Reasoning capabilities distinguish advanced AI systems from simple pattern matching. This section reviews reasoning enhancement techniques relevant to Chapter 5.

2.6.1 Reasoning Capabilities and Limitations

While LLMs demonstrate impressive language generation capabilities, their reasoning abilities remain an active area of research. Key challenges include:

Multi-step Reasoning: LLMs often struggle with problems requiring extended chains of logical inference, exhibiting error accumulation across reasoning steps [83, 151].

Mathematical Reasoning: Arithmetic and mathematical problem-solving expose limitations in systematic computation [25].

Logical Consistency: Models may generate superficially coherent but logically inconsistent responses [93]. Logical fallacy detection remains challenging, requiring models to identify subtle reasoning errors in natural language arguments [**?**].

Lateral Thinking: Problems requiring unconventional reasoning approaches, such as lateral thinking puzzles (e.g., "Turtle Soup" puzzles requiring deductive reasoning from partial information), present particular challenges for current models [4].

2.6.2 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting [163] has emerged as a pivotal technique for enhancing LLM reasoning. By including intermediate reasoning steps in prompts, models are encouraged to generate explicit reasoning chains before final answers.

Few-shot CoT: Providing examples with detailed reasoning solutions instructs models to emulate step-by-step reasoning patterns.

Zero-shot CoT: Simple instructions like "Let's think step by step" can trigger improved reasoning without examples [101].

Empirical studies demonstrate marked improvements on arithmetic, commonsense, and symbolic reasoning tasks when using CoT prompting [163]. However, effectiveness varies with model scale and task complexity—a relationship explored further in Section 5.3.3.

2.6.3 Specialized Reasoning Models

Recent developments have produced models specifically optimized for reasoning tasks:

OpenAI o1 Series: Employs reinforcement learning to develop deliberative reasoning capabilities, allocating variable computational effort based on problem complexity [126].

DeepSeek-R1: Features explicit reasoning traces generated through reinforcement learning, providing step-by-step reasoning processes that distinguish it from conventional LLMs [54]. The model family includes distilled variants (8B–70B) enabling deployment on diverse hardware.

Qwen3: Alibaba's reasoning-enhanced model family with support for "thinking mode" that generates explicit reasoning traces through adaptive compute based on query difficulty [168, 136]. Thinking mode can be enabled or disabled via system prompts, enabling evaluation of reasoning impact on specific tasks.

Granite3.3: IBM's lightweight models with conditional reasoning that separates intermediate reasoning from final answers [86, 30].

Magistral: Mistral's reasoning model using reinforcement learning to shape reasoning paths [141, 3].

These models represent a shift toward "thinking before responding," with explicit reasoning phases prior to answer generation.

2.6.4 Task Complexity and Reasoning Effectiveness

Recent research has begun to question whether reasoning capabilities universally improve LLM performance across all NLP tasks. Studies on financial sentiment analysis demonstrate that chain-of-thought reasoning can lead to "overthinking"—generating elaborate reasoning chains that introduce errors rather than clarifying decisions [108].

2.7 Sentiment Analysis and Cross-Domain Applications

LLMs have demonstrated broad applicability across diverse NLP tasks. This section reviews sentiment analysis and related applications relevant to Chapter 5.

2.7.1 Evolution of Sentiment Analysis

Sentiment analysis has evolved through several paradigms [161]:

Lexicon-Based Methods: Using predefined sentiment dictionaries to classify text polarity [120]. While interpretable, these methods struggle with context-dependent sentiment.

Machine Learning Approaches: Supervised classifiers (SVM, Naive Bayes) trained on labeled sentiment data [128]. Feature engineering was critical for performance.

Deep Learning Methods: Neural architectures (CNN, LSTM, Transformers) learning sentiment features from data [149]. Pre-trained representations substantially improved performance.

LLM-Based Analysis: Leveraging pre-trained language models for zero-shot or few-shot sentiment classification [172]. Modern LLMs achieve competitive performance without task-specific training, with reasoning-capable models providing transparent reasoning traces that enhance interpretability—particularly valuable

for fine-grained sentiment tasks requiring nuanced distinctions between adjacent categories (e.g., “Strongly Positive” vs. “Positive”).

2.7.2 Machine Translation

Machine translation between typologically distant languages such as Chinese and English presents particular challenges:

Traditional Approaches: Rule-based and statistical machine translation relied on handcrafted rules or patterns from parallel corpora [100].

Neural Machine Translation: The transformer architecture enabled end-to-end learning of translation patterns [157].

LLM-Based Translation: Large language models demonstrate competitive translation performance through few-shot prompting, with fine-tuning further improving quality. Evaluation metrics have evolved from n-gram overlap (BLEU [130]) to neural-based metrics like COMET [142] that better correlate with human judgments. However, these metrics may not capture all aspects of translation quality—LLMs sometimes produce outputs with untranslated segments, motivating the introduction of complementary metrics that measure the proportion of fully rendered translations.

2.7.3 Sports Predictive Analytics

The integration of LLMs with traditional machine learning for predictive analytics represents an emerging application area [17, 160]. Sentiment features extracted from social media and news sources can enhance prediction accuracy when combined with conventional statistical models, demonstrating that LLM-derived features provide complementary signal to historical data.

2.7.4 Entity Matching and Resolution

Entity matching (also called entity resolution or record linkage) is the task of identifying records across one or more data sources that refer to the same real-world

entity. The standard pipeline comprises a *blocking* stage that filters obvious non-matches for scalability, followed by a *matching* stage that makes fine-grained decisions [40, 47, 129, 15].

Traditional matching approaches range from rule-based methods [13, 110] and adaptive string-distance techniques [14] to deep learning models that learn record representations for comparison [121, 10]. Pre-trained language models have recently advanced the field: Li et al. [111] demonstrated that pre-trained transformers substantially improve matching accuracy, and Tu et al. [154] proposed a unified multi-tasking approach.

The advent of LLMs has introduced zero-shot and few-shot paradigms for entity matching. Narayan et al. [122] investigated whether foundation models can perform data wrangling tasks including entity matching; Peeters and Bizer [132, 133] evaluated ChatGPT and subsequent LLMs on standard benchmarks; and Fan et al. [39] explored cost-effective in-context learning designs. Wang et al. [159] conducted the most comprehensive investigation to date, comparing three strategies (matching, comparing, and selecting) and proposing the Compound Entity Matching (ComEM) framework that composes strategies and models for improved effectiveness and cost-efficiency. However, the impact of configurable reasoning effort—a defining feature of recent models such as GPT-5, DeepSeek-R1, and Qwen3—on entity matching accuracy and cost remains largely unexplored, motivating the evaluation in Chapter 5.

2.8 Summary and Research Gaps

This literature review has surveyed foundational concepts in LLMs, implementation approaches for enterprise deployment, and existing research on the three core research pillars. Several research gaps motivate this dissertation:

2.8.1 Gaps in RAG Optimization (RQ1)

1. **Evaluation Metrics:** Existing metrics inadequately capture user experience factors like repetition and verbosity in LLM outputs. Traditional metrics (BLEU, ROUGE, F1) and even neural metrics (BERTScore, COMET) overlook repetitive content that degrades output quality [44, 60]. Novel metrics are needed that balance performance with output quality, integrating repetition penalty into model evaluation through mechanisms such as a repetition ratio computed from detected non-word character and text repetitions.

2. **Hyperparameter Optimization:** The impact of generation parameters, particularly repetition penalty [97], on RAG output quality is underexplored. While the repetition penalty parameter (RPP) aims to reduce redundancy, traditional metrics fail to capture this dimension, making optimal tuning challenging. Metrics that enable identification of trade-off values—significantly reducing repetition while minimizing performance loss—are needed.

3. **Open-Source Benchmarking:** Systematic comparison of open-source models against proprietary alternatives for RAG-specific tasks remains limited, particularly regarding the trade-offs between model size, architecture, and performance across diverse datasets and prompting strategies.

2.8.2 Gaps in Agentic AI (RQ2)

1. **Workflow Automation:** Real-world evaluation of agentic systems for complex business processes such as invoice reconciliation and payment processing is lacking. Most existing work focuses on simpler single-task automation rather than end-to-end multi-agent workflows [158]. No established reference framework exists for implementing end-to-end agentic payment workflows using LLM-based multi-agent systems.

2. **Reliability Metrics:** Standard accuracy metrics are insufficient for assessing

multi-agent pipeline reliability [102, 96]. Metrics that capture end-to-end success rates, first-attempt completion rates, and cost implications of failures are needed for production deployment decisions. Fine-grained metrics that decompose workflows into atomic steps can reveal that smaller models fail primarily at tool orchestration rather than downstream processing—insights invisible to binary success metrics.

3. **Error Taxonomy:** Systematic diagnostic frameworks for characterizing failure modes in multi-agent systems are underdeveloped. A comprehensive taxonomy covering tool initialization failures, parameter mismatches, execution errors, and result interpretation failures across different tool categories is needed to enable targeted reliability improvements.

4. **Edge Deployment:** Practical guidelines for deploying multi-agent systems on resource-constrained enterprise hardware are needed, including model selection, energy efficiency considerations, and cost-performance trade-offs [35].

2.8.3 Gaps in Enterprise Deployment (RQ3)

1. **Hardware Benchmarking:** Systematic evaluation across diverse consumer and enterprise hardware platforms is limited, particularly for energy efficiency metrics [22]. Cross-platform comparisons including enterprise GPUs (H100, RTX A6000), consumer GPUs (RTX 4090), Apple Silicon (M3/M4 Max), and edge devices are needed.

2. **Reasoning Reliability:** Methods for assessing the reliability of LLM reasoning outputs require further development. Beyond accuracy, metrics capturing the proportion of valid classifications and reasoning efficiency are needed for deployment decisions.

3. **Task-Complexity-Dependent Reasoning:** The relationship between task complexity and reasoning effectiveness remains underexplored. Understanding when

reasoning mechanisms improve versus degrade performance across tasks of varying complexity—from binary classification to fine-grained multi-class emotion recognition—is critical for model selection [108]. Comprehensive evaluation across 500+ configurations spanning multiple model families is needed to establish principled guidelines.

4. **Model Selection Guidelines:** Evidence-based recommendations for model selection based on task requirements, hardware constraints, and efficiency trade-offs are lacking. The relationship between model architecture, parameter count, and performance requires nuanced investigation, including Pareto frontier analysis of cost-performance trade-offs [54, 167].

5. **Cross-Domain Validation:** Demonstration that optimization techniques generalize across diverse enterprise applications (translation, sentiment analysis, predictive analytics, entity matching) is needed to establish the broader applicability of LLM deployment strategies.

6. **Entity Matching at Scale:** While LLM-based entity matching has shown promise, no study has systematically evaluated the impact of configurable reasoning effort across a broad range of model families (proprietary and open-weight) and budget tiers. Understanding family-dependent reasoning effects and cost-performance trade-offs is essential for production deployment in data-intensive enterprise applications.

The following chapters address these gaps through systematic empirical research and methodological contributions:

- Chapter 3 addresses RAG optimization gaps through comparative evaluation, the RAP metric and ReDA algorithm for repetition detection, and deployment guidelines enabling up to 93.1% repetition reduction with only 3.7% performance degradation

- Chapter 4 addresses agentic AI gaps through multi-agent architectures for business automation (invoice reconciliation achieving 99.9% success, payment processing achieving 97.0% task success), a structured error taxonomy for systematic failure diagnosis, and novel evaluation frameworks including ASR and ECR@1 reliability metrics

- Chapter 5 addresses enterprise deployment gaps through edge benchmarking across six hardware platforms, reasoning enhancement evaluation across 504 configurations demonstrating task-complexity-dependent effectiveness, and cross-domain applications including translation and predictive analytics demonstrating technique generalizability

Chapter 3

Optimizing LLMs for Retrieval-Augmented Generation

3.1 Introduction and Motivation

Retrieval-Augmented Generation (RAG) has emerged as a critical paradigm for enterprise knowledge management, enabling Large Language Models to provide accurate, contextually grounded responses by incorporating external knowledge sources [106]. By combining dense retrieval mechanisms with generative models, RAG addresses fundamental limitations of parametric LLMs—namely, their tendency to hallucinate [84] and inability to access current or organization-specific information.

However, deploying RAG systems effectively requires careful optimization across multiple dimensions: model selection, retrieval configuration, generation parameters, and evaluation methodology. This chapter synthesizes findings from three accepted publications to establish comprehensive optimization guidelines for enterprise RAG deployment:

1. **Evaluation of Orca 2 for RAG** [66]: Comparative assessment of model performance on RAG benchmarks (PAKDD 2024 Workshops)

2. **Performance Analysis of Llama 2** [72]: In-context learning evaluation across

multiple LLMs (IEEE CAI 2024)

3. **RAP Metric** [74]: Novel evaluation metric addressing repetition in open-source LLM outputs (NAACL 2025)

The research presented in this chapter addresses the first research question (RQ1): *How can RAG systems be optimized to improve response quality while addressing critical issues such as content repetition in open-source LLMs?* The findings establish foundations that inform subsequent chapters: the evaluation methodology developed here informs the reliability metrics for agentic systems (Chapter 4), while the model selection insights guide edge deployment strategies (Chapter 5).

The chapter is organized as follows: Section 3.2 presents our experimental methodology including datasets, models, and evaluation frameworks. Section 3.3 details comparative evaluation results demonstrating open-source model competitiveness. Section 3.4 introduces the Repetition-Aware Performance metric and its application to hyperparameter optimization. Section 3.6 synthesizes practical deployment guidelines.

3.2 Methodology

3.2.1 RAG System Architecture

Our experimental RAG system follows a standard architecture comprising document processing, retrieval, and generation components (Figure 3.1), consistent with the framework established by Gao et al. [45].

Document Processing

Documents are processed through the following pipeline:

1. **Text Extraction:** PDF documents are parsed using specialized libraries to extract text while preserving structural information.

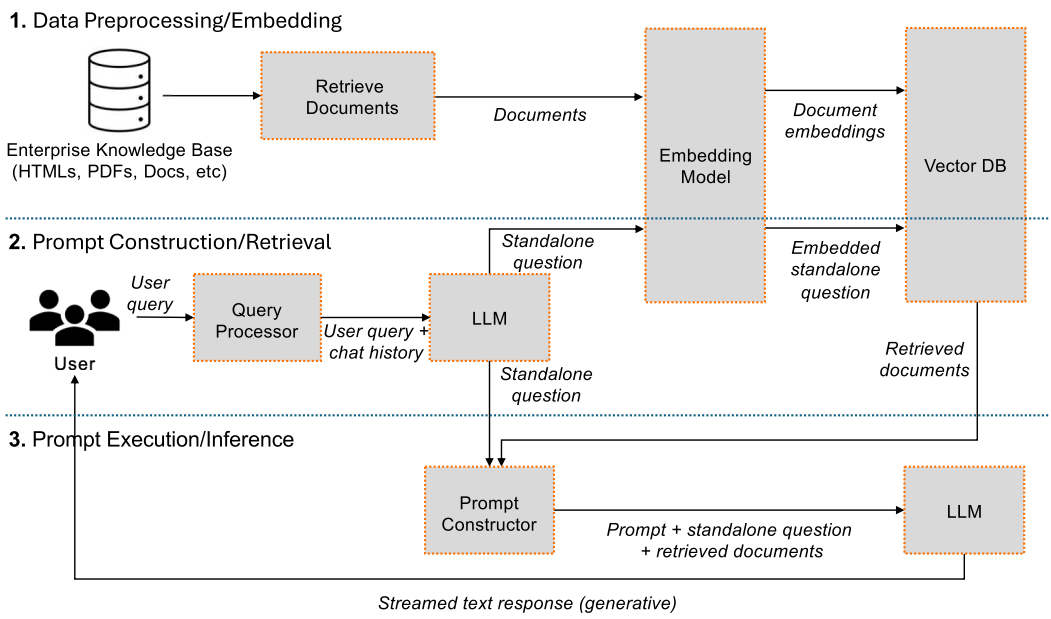


Figure 3.1: Overview of the RAG system architecture used in our experiments.

2. **Chunking:** Documents are segmented into chunks of approximately 500 tokens with 50-token overlap to maintain context continuity.

3. **Embedding:** Text chunks are encoded using the HuggingFace Instructor model [150], generating 768-dimensional dense vectors.

4. **Indexing:** Embeddings are stored in FAISS [94] for efficient similarity search.

Retrieval Configuration

For each user query, the retrieval component:

- Encodes the query using the same embedding model
- Performs approximate nearest neighbor search to identify top-k relevant chunks ($k = 10$ in our experiments)
- Returns retrieved chunks as context for generation

Conversational Retrieval Chain

We employ LangChain's ConversationalRetrievalChain [104] for multi-turn interactions. The chain operates in three phases:

1. **Standalone Question Generation:** Synthesizes a context-independent question from chat history and current query
2. **Document Retrieval:** Retrieves relevant context using the standalone question
3. **Response Generation:** Generates the final response conditioned on retrieved context

This architecture serves as the foundation for more complex multi-agent systems presented in Chapter 4, where retrieval and generation are orchestrated across specialized agents.

3.2.2 Evaluation Datasets

PCI DSS Dataset

For RAG-focused experiments, we utilize documents from the Payment Card Industry Data Security Standard (PCI DSS) version 4.0. This dataset comprises 13 PDF documents obtained from the PCI Security Standards Council, covering security requirements for organizations handling payment card data.

The domain-specific nature of PCI DSS demands accurate technical retrieval, concise synthesis of requirements, and context-aware compliance guidance.

Our evaluation queries focus on:

1. Basic definitional questions ("What's PCI DSS?")
2. Comparative analysis ("Changes from version 3.2.1 to 4.0")
3. Technical deep-dives ("New vulnerability assessment requirements")
4. Follow-up clarifications ("More on penetration testing")

MS MARCO Dataset

For broader in-context learning evaluation, we utilize a subset of the MS MARCO dataset [9]. We curated 500 queries with well-formed answers across five categories:

Table 3.1: MS MARCO Evaluation Dataset Composition

Category	Queries	Example
LOCATION	100	Where is the Eiffel Tower?
NUMERIC	100	How many states in the US?
PERSON	100	Who invented the telephone?
DESCRIPTION	100	What is photosynthesis?
ENTITY	100	What is the capital of France?
Total	**500**	

WebQSP Dataset

For knowledge-base question answering (KBQA) evaluation, we utilize the WebQSP dataset [169], comprising 1,008 questions with entity-based answers from Wikidata. This dataset enables evaluation of LLM performance on structured knowledge retrieval tasks.

3.2.3 Model Selection

Table 3.2 summarizes the LLMs evaluated in our experiments, spanning proprietary and open-source alternatives across a range of parameter counts from 2B to 70B.

Table 3.2: Large Language Models Evaluated

Model	Organization	Parameters	Type
GPT-4	OpenAI	Unknown	Proprietary
GPT-3.5-Turbo	OpenAI	Unknown	Proprietary
Orca-2-13b	Microsoft	13B	Open-source
Orca-2-7b	Microsoft	7B	Open-source
Llama-2-70b	Meta	70B	Open-source
Llama-2-13b	Meta	13B	Open-source
Llama-2-7b	Meta	7B	Open-source
Llama-3-70b	Meta	70B	Open-source
Llama-3-8b	Meta	8B	Open-source
Gemma-1.1-7b	Google	7B	Open-source
Gemma-1.1-2b	Google	2B	Open-source
Phi-3-mini	Microsoft	3.8B	Open-source
Mistral-7b	Mistral AI	7B	Open-source

3.2.4 Evaluation Metrics

Generation Quality Metrics

We employ the RAGAS framework [38] for generation quality assessment:

Faithfulness Score (FS): Measures factual consistency between generated responses and retrieved context:

$$FS = \frac{\text{Number of contextually supported claims}}{\text{Total number of claims in response}} \tag{3.1}$$

Answer Relevancy (AR): Assesses alignment between responses and user queries, computed as the cosine similarity between the original query and questions that the response would answer.

Overall Score: Harmonic mean of Faithfulness and Answer Relevancy:

$$\text{Overall} = \frac{2 \times FS \times AR}{FS + AR} \tag{3.2}$$

Inference Speed

Measured as tokens generated per second, capturing practical deployment considerations for real-time applications.

Answer Quality for In-Context Learning

For MS MARCO evaluation, we employ:

- **ROUGE-L:** Longest common subsequence-based similarity [113]
- **BLEU-1:** Unigram precision with brevity penalty [130]

3.2.5 Hardware Configuration

Experiments were conducted on multiple hardware configurations:

- **Consumer Desktop:** NVIDIA GeForce RTX 4090 (24GB VRAM)
- **Professional Workstation:** NVIDIA A40 (48GB VRAM)

- **High-Performance Server:** NVIDIA L40 (48GB VRAM)

This diversity enables evaluation of model performance across typical enterprise hardware tiers, informing the comprehensive hardware benchmarking presented in Chapter 5.

3.3 Comparative Evaluation of LLMs for RAG

3.3.1 Generation Quality Results

Figure 3.2 presents generation quality metrics across evaluated models on the PCI DSS benchmark.

Key observations include:

Faithfulness: All models except GPT-3.5-Turbo and Llama-2-13b achieved perfect faithfulness scores (100%), indicating strong factual consistency with retrieved context. The lower scores for GPT-3.5-Turbo appear attributable to its tendency to rely on parametric knowledge rather than retrieved context.

Answer Relevancy: Orca-2-13b and Orca-2-7b achieved top performance, with scores approaching 99%. This demonstrates their effectiveness in providing contextually relevant responses aligned with user queries.

Overall Score: Orca-2-13b achieved the highest overall score (99.3%), followed closely by Orca-2-7b (99.2%), indicating balanced performance across quality dimensions.

Table 3.3 summarizes the quantitative results.

3.3.2 Open-Source Model Competitiveness

A central finding of our evaluation is that properly configured open-source models can match or exceed proprietary alternatives for RAG applications. Orca-2-13b achieved a 99.3% overall RAGAS score—surpassing GPT-4 (97.5%) while operating

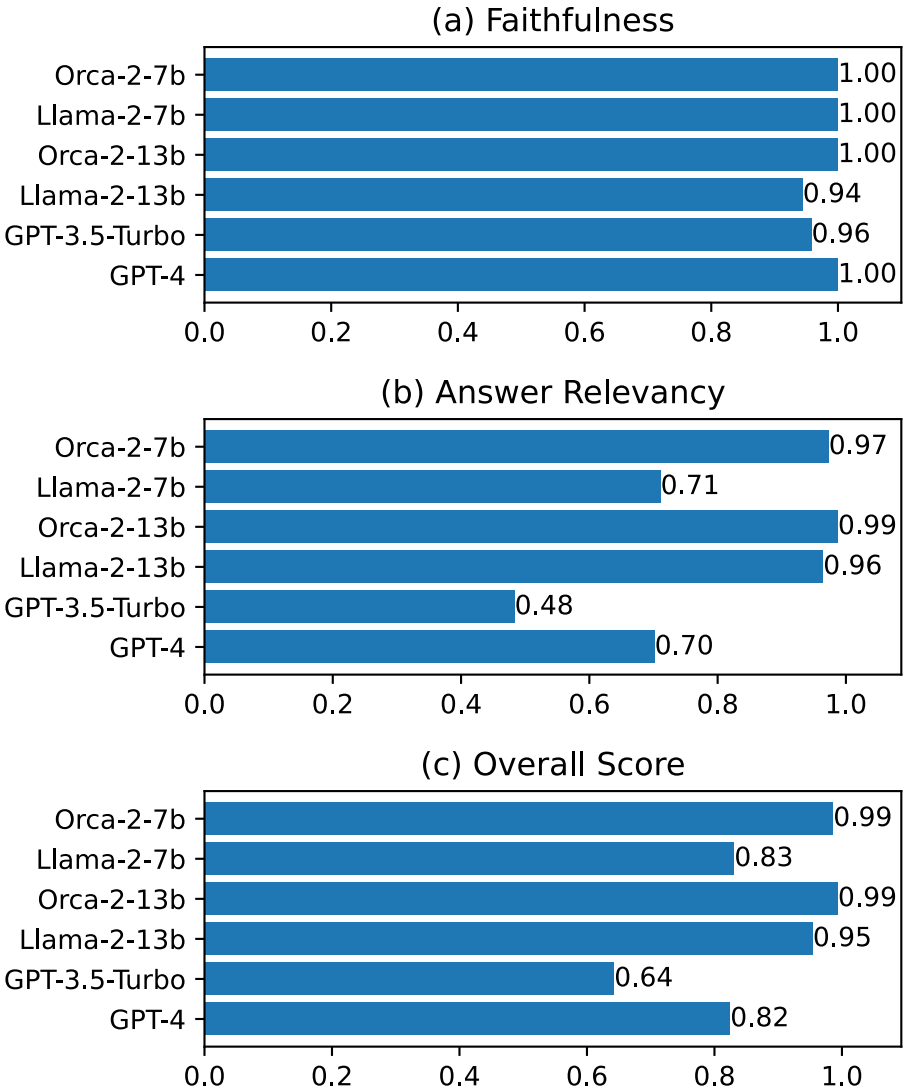


Figure 3.2: Comparison of generation quality metrics across LLMs on PCI DSS benchmark.

Table 3.3: RAG Generation Quality Results on PCI DSS Benchmark

Model	**Faithfulness (%)**	**Answer Relevancy (%)**	**Overall (%)**
Orca-2-13b	100.0	98.7	99.3
Orca-2-7b	100.0	98.5	99.2
GPT-4	100.0	95.2	97.5
Llama-2-7b	100.0	92.1	95.9
GPT-3.5-Turbo	87.3	89.8	88.5
Llama-2-13b	91.2	85.4	88.2

on consumer-grade hardware. This finding challenges the assumption that proprietary models are necessary for high-quality RAG deployment and has significant implications for enterprise cost management and data privacy.

The performance advantage of Orca-2 models appears attributable to their training methodology, which emphasizes learning from complex explanation traces rather than simply imitating larger models [119]. This training approach produces models that more reliably follow retrieved context rather than defaulting to parametric knowledge—a critical capability for enterprise RAG applications where accuracy must be grounded in organizational documents.

This finding—that smaller, specialized models can outperform larger general-purpose alternatives—recurs throughout this dissertation. In Chapter 4, we observe similar patterns where qwen2.5:32b matches GPT-4.1 performance for invoice reconciliation, and in Chapter 5, DeepSeek-R1:32b substantially outperforms DeepSeek-R1:70b on sentiment classification tasks.

3.3.3 Inference Speed Analysis

Figure 3.3 presents inference speed comparisons.

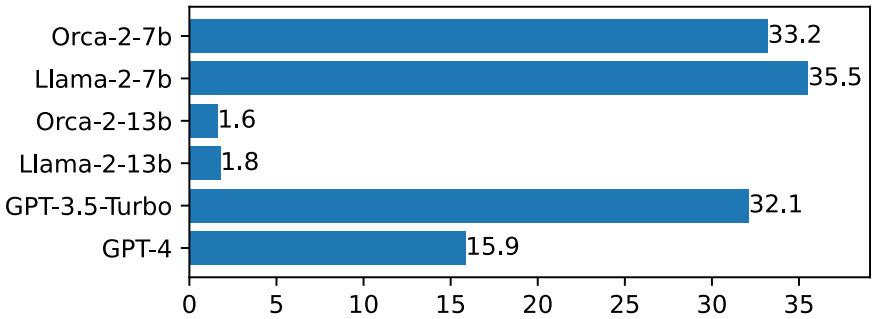


Figure 3.3: Inference speed comparison across LLMs on consumer and professional hardware.

The Orca-2-7b model achieved approximately 33 tokens per second on the RTX 4090, closely matching GPT-3.5-Turbo (32 tokens/sec) and significantly outperforming GPT-4 (16 tokens/sec). This demonstrates that smaller, efficient models can

provide competitive throughput on consumer-grade hardware.

Notable observations:

- 13B parameter models exhibited slower speeds due to GPU memory constraints
- Running Orca-2-13b on the A40 (48GB) improved speed to approximately 15 tokens/sec
- Hardware memory capacity significantly impacts larger model performance

3.3.4 Qualitative Analysis

Examination of generated responses revealed several patterns:

Language Switching: Orca-2-13b occasionally switched to Spanish after the first question while maintaining high quality scores. This indicates that RAGAS evaluation—utilizing GPT-4-Turbo—assesses quality based on semantics rather than language consistency.

Standalone Question Generation: Models exhibited varying quality in generating standalone questions for retrieval. Orca-2-7b sometimes produced generic questions, while other models maintained query specificity.

Context Utilization: GPT-3.5-Turbo and GPT-4 occasionally ignored retrieved context in favor of parametric knowledge, explaining their lower faithfulness scores in some scenarios. This behavior is particularly problematic for enterprise applications where responses must be grounded in organizational documents.

3.4 The Repetition-Aware Performance Metric

When optimizing a RAG system, it is crucial to employ evaluation metrics that can effectively guide both system development and automated hyperparameter tuning. Standard metrics for QA or text generation (e.g., accuracy, F1, BLEU) capture how well the generated content aligns with the ground truth but fail to account

for issues like redundancy or verbosity. In open-source LLMs—particularly those not rigorously RLHF-tuned for conciseness—repetitive outputs are common [44]. For instance, a model might repeat a sentence from the context multiple times or generate unnecessarily long sequences of non-word characters, such as repeated newlines. Such behavior not only degrades user experience but can also indirectly reduce accuracy if key information becomes obscured by repetitive content. This is particularly problematic in tasks requiring coherent and contextually relevant responses, such as conversational chatbots and—as we demonstrate in Chapter 4—multi-agent systems where output quality directly impacts downstream component reliability.

To address this, the *repetition penalty parameter* (RPP) is employed during the sampling process to reduce redundant outputs by penalizing previously seen tokens [97]. While effective, excessive application of RPP can result in incomplete or fragmented responses, ultimately diminishing user satisfaction.

To address this issue, we introduced the *Repetition-Aware Performance* (RAP) metric in our NAACL 2025 paper [74]. That work investigates the relationship between the repetition penalty parameter (RPP) and its effect on generated text. While parameters like *temperature* and *top-k sampling* are typically optimized via random or grid search, these approaches are inadequate for tuning RPP because conventional metrics, such as precision and BLEU scores [130], fail to account for repetition.

To address this, we propose a metric called **R**epetition-**A**ware **P**erformance (RAP), which quantifies repetition and penalizes it in the model's performance evaluation, allowing for the tuning of RPP. Figure 3.4 illustrates tuning RPP for Llama-3-8B using the original evaluation metrics (Original) and our proposed metric. For the WebQSP dataset [169], repetition ratio (RR) is reduced by 93.1% but the performance only drops by 3.7%. For the MS MARCO dataset [9], RR drops by 73.9% with only 3.7% of performance drop. Tuning RPP using RAP can significantly reduce the RR with minimal performance loss.

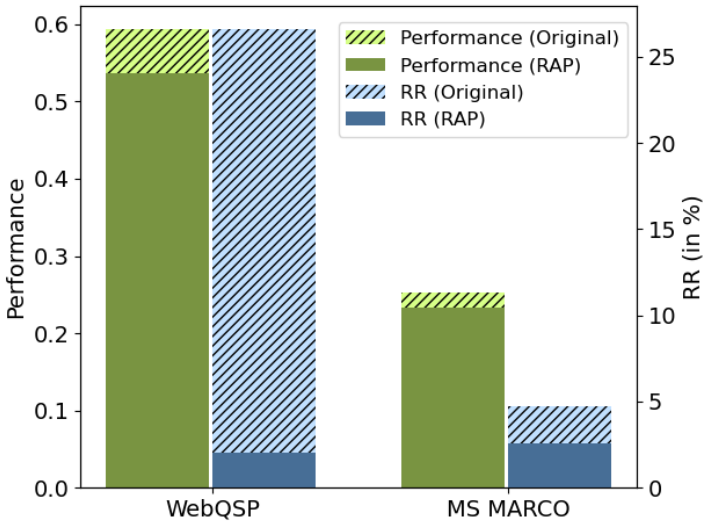


Figure 3.4: Best performance (F1 for WebQSP, BERT-F1 for MS MARCO) when tuning RPP using original evaluation metrics (Original) and our proposed repetition-aware performance metric (RAP). RAP helps find RPP values that minimally reduce performance while significantly lowering repetition ratio (RR). Result is from Llama-3-8B (RAG-Generic Prompt).

To compute RAP, we introduce an efficient algorithm called ReDA—Repetition Detection Algorithm—designed to identify all repeating patterns, including non-word character (NWC) and text repetitions, along with their corresponding repetition lengths. Additionally, we define the repetition ratio (RR), which measures the proportion of repeated content relative to the total text length. RR is applied to penalize the model's performance using a cubic penalty formula, as detailed in Section 3.4.2. Figure 3.5 illustrates computing RAP with a running example. Although the accuracy is 100%, the content contains many repeated elements, resulting in a low RAP score of 24.63%.

We conduct experiments on three datasets encompassing question answering (including knowledge base question answering—KBQA—and open-ended QA) and machine translation tasks. We evaluate twelve open-source LLMs ranging from 2 to 70 billion parameters. We also examine the effect of repetition suppression for different prompting strategies including Retrieval-Augmented Generation (RAG) [106] and non-RAG, combining with generic prompt and chat template.

Our study reveals that increasing RPP typically reduces repetition but often

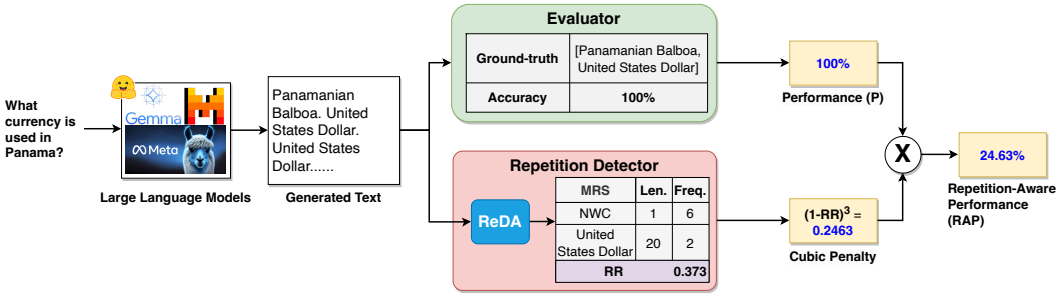


Figure 3.5: Overall Methodology: RAP integrates the repetition penalty in the evaluation score. In the example, the generated text contains non-word characters (NWC) repetition and text repetition. After penalization, the accuracy is reduced from 100% to 24.63%. ReDA stands for Repetition Detection Algorithm.

results in performance degradation, highlighting the importance of tuning RPP. The effects of RPP differ across various models, datasets, and prompting techniques. Additionally, we find that using chat templates is effective in reducing repetition for QA tasks. Our experimental results demonstrate that RAP can effectively tune RPP, helping to identify a trade-off value that significantly reduces repetition while minimizing performance loss. These findings offer insights into optimizing RPP for performance and repetition suppression in open-source LLMs.

3.4.1 Repetition Ratio (RR)

First, we define the **Repetition Ratio (RR)** for a set of outputs (e.g., all answers in an evaluation set). The repetition ratio measures the portion of the generated text that is composed of repetitive content. To compute RR, repetition must be detected at both the character level (e.g., excessive whitespace or punctuation) and the word/phrase level. We employ a repetition detection algorithm (ReDA) that identifies two types of repetition:

- **Non-word Character (NWC) Repetition:** Sequences of spaces or other non-alphanumeric characters that are overly long (e.g., multiple newline characters or dashes), indicating that the model is stalling or outputting blank lines.
- **Textual Repetition:** Longer sequences of words that repeat. We search for any substring of length $\geq L$ (a threshold such as 5 characters) that appears at least

twice in the output.

For each generated output S, let $R(S)$ denote the total length (in characters) of all repeated segments identified in S (summing lengths from both NWC and textual repetition). Let $|S|$ be the total length of S. Then, the repetition ratio for a single output is defined as:

$$r(S) = \frac{R(S)}{|S|}.$$

For an entire set D of outputs, the aggregate repetition ratio is given by:

$$\text{RR}_D = \frac{\sum_{S \in D} R(S)}{\sum_{S \in D} |S|}. \tag{3.3}$$

Here, $\text{RR}_D = 0$ indicates no repetition, while a value of 0.10 indicates that 10% of the generated text is redundant.

In practice, $R(S)$ is implemented using regular expressions to detect runs of whitespace, punctuation, and repeated substrings efficiently [74]. Algorithm 1 provides pseudocode for the ReDA algorithm, which iteratively removes detected repetitions and sums their lengths.

Algorithm 1 Repetition Detection (ReDA) [74]

Require: Generated text S
Ensure: Repetition lengths $L_{\text{NWC}}, L_{\text{text}}, L_{\text{total}}$

```
L_NWC ← 0, L_text ← 0
Use regex to find any occurrence of 5 or more non-word characters in a row in S
if matches found then
    Remove these sequences from S and increment L_NWC by the total number of removed characters
end if
Use regex to find repeated textual patterns of length ≥ 5 in S
for each match of a repeated substring of length ℓ in S do
    L_text ← L_text + ℓ                    ▷ Accumulate lengths of repeats
end for
L_total ← L_NWC + L_text
return (L_NWC, L_text, L_total)
```

3.4.2 Defining the RAP Score

With RR_D computed, we define the RAP score for the model's outputs. Let P be the base performance metric of interest (e.g., average F1 score, exact match accuracy, BERT-F1, or COMET, depending on the task). We incorporate a repetition penalty through a function $F(\text{RR}_D)$ that decreases from 1 as RR_D increases. In our work, we use a cubic penalty for smoothness:

$$F(\text{RR}) = (1 - \text{RR})^3 \,.$$

The RAP score is then defined as:

$$\text{RAP} = P \times F(\text{RR}_D) \,. \tag{3.4}$$

If the model produces no repetition ($\text{RR}_D = 0$, so $F = 1$), then $\text{RAP} = P$. If there is any repetition, then $0 < F < 1$, leading to $\text{RAP} < P$, effectively lowering the performance score in proportion to the repetition. For example, if $\text{RR}_D = 0.1$ (10% repetition), then $F(0.1) = 0.9^3 \approx 0.729$, meaning the model retains roughly 73% of its base score under RAP. This approach incentivizes reducing repetition.

RAP is valuable in two ways:

1. It can guide hyperparameter tuning (see Section 3.5). For instance, when tuning the repetition penalty parameter (RPP), using only accuracy might select a configuration with higher accuracy but excessive redundancy. RAP leads to choosing an RPP that balances performance and conciseness.

2. It allows for holistic model comparisons. In our experiments, while larger models had higher raw performance, some smaller models achieved comparable RAP scores due to reduced redundancy—an important insight for enterprise deployments.

It is important to ensure that RAP does not excessively penalize necessary repetitions. In some contexts, repeating critical instructions—such as "Please read carefully. Please read carefully."—is essential for ensuring emphasis and clarity. Our

ReDA algorithm is designed to detect only excessive or verbose repetitions; small amounts of repetition yield a penalty factor close to 1. In practice, we found that RAP correlates well with human judgments of answer quality when redundancy is taken into account.

3.4.3 Experimental Settings

Datasets. We conduct our experiments on three datasets: WebQSP [169] and MS MARCO [9] for question answering (QA), and MAC [70] for Chinese-English machine translation (MT). The test sets contain 1,008, 500, and 1,133 questions, respectively. For WebQSP, we crawl relevant Wikipedia articles and retrieve the 8 most pertinent document chunks per question to form RAG prompts. We also adopt Gemini-1.0-Pro to evaluate the knowledge base QA (KBQA) task, where ground-truth answers consist of lists of entities rather than free-form text.

Cross-Task Validation with Machine Translation. While this chapter focuses primarily on RAG optimization for question answering, we include machine translation (MT) experiments to demonstrate the generalizability of the RAP metric. The repetition problem is not unique to RAG-based QA—it manifests across diverse generation tasks. Validating RAP on MT establishes its applicability as a general-purpose evaluation metric for open-source LLMs, regardless of whether retrieved context is involved. This cross-task validation strengthens confidence in RAP's utility for enterprise deployments spanning multiple use cases. The MT findings presented here lay groundwork for the comprehensive translation evaluation in Chapter 5, where we introduce additional metrics (Translation Completeness Ratio, Characters per Second) for deployment-oriented assessment.

Open-Source LLMs. This study evaluates a diverse set of open-source large language models (LLMs) for question answering (QA) and machine translation (MT) tasks. For QA, we assess nine models: Llama-2 (7B, 13B, 70B), Llama-3 (8B, 70B), Gemma-1.1 (2B, 7B), Phi-3-mini (3.8B), and Mistral-7B. For MT, we evaluate three

models: Phi-3.5-mini (3.8B), Mistral-7B, and Llama-3.1-70B. Table 3.4 provides an overview of these models—including their companies, model names, Hugging Face identifiers, and the tasks on which they were evaluated. This selection spans a broad range of model sizes and architectures, representing the state of the art in open-source LLM development and commercial use.

Table 3.4: Overview of Large Language Models and Their Specifications by Task

Company	Model Name	Hugging Face Model ID	Task
Meta	Llama-2-7B	meta-llama/Llama-2-7b-chat	QA
	Llama-2-13B	meta-llama/Llama-2-13b-chat	QA
	Llama-2-70B	meta-llama/Llama-2-70b-chat	QA
	Llama-3-8B	meta-llama/Meta-Llama-3-8B-Instruct	QA
	Llama-3-70B	meta-llama/Meta-Llama-3-70B-Instruct	QA
	Llama-3.1-70B	shenzhi-wang/Llama3.1-70B-Chinese-Chat	MT
Microsoft	Phi-3-mini	microsoft/Phi-3-mini-128k-instruct	QA
	Phi-3.5-mini	microsoft/Phi-3.5-mini-instruct	MT
Google	Gemma-1.1-2B	google/gemma-1.1-2b-it	QA
	Gemma-1.1-7B	google/gemma-1.1-7b-it	QA
Mistral AI	Mistral-7B-v0.2	mistralai/Mistral-7B-Instruct-v0.2	QA
	Mistral-7B-v0.3	shenzhi-wang/Mistral-7B-v0.3-Chinese-Chat	MT

Automating QA and MT Tasks with LLMs. We developed a Python script leveraging the HuggingFace Transformers library [165] to automate evaluation across all models and datasets. For QA, a repetition penalty (RP) ranging from 1.0 to 1.3 (in increments of 0.02) was applied; for MT, an RP range of 1.0 to 1.1 was used. To ensure consistency and reproducibility, the temperature was set to 0 and `top_p` was fixed at 0.95.

Prompt Templates. To explore the impact of prompting strategies, we implemented three QA prompt templates: (1) *RAG - Generic Prompt* (RAG-GP): a uniform template applied to all models; (2) *RAG - Chat Template* (RAG-CT): LangChain's default template adapted to each LLM's chat format, preserving core content; and (3) *Non-RAG*: a template simulating a single-turn conversation in the LLM's native chat format. For MT, we employed two templates: (1) *MT - Generic Prompt* (MT-GP): a basic template applied uniformly; and (2) *MT - Chat Template* (MT-CT): a template customized to each LLM's chat format.

The following listings describe the prompt templates used in our experiments.

Listing 3.1: RAG - Generic Prompt template for question answering. The placeholders {question} and {context} are filled with the question and retrieved data, respectively.

```
Use the following pieces of context to answer the question
   at the end. If you don't know the answer, just say that
   you don't know, don't try to make up an answer.
{context}
Question: {question}
Helpful Answer:
```

Listing 3.2: Non-RAG prompt template for question answering with Llama-3 models. The placeholder {question} is filled with the question.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
You are a chatbot having a conversation with a human.
<|eot_id|>
<|start_header_id|>user<|end_header_id|>
{question}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Listing 3.3: RAG - Chat Template for Llama-3 models. The placeholders {question} and {context} are filled with the question and retrieved data, respectively.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
You are a chatbot having a conversation with a human.
<|eot_id|>
<|start_header_id|>user<|end_header_id|>
```

```
Use the following pieces of context to answer the question
   at the end. If you don't know the answer, just say that
   you don't know, don't try to make up an answer.
{context}
Question: {question}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Listing 3.4: Machine Translation - Generic Prompt (MT-GP) for all models. The placeholder {input} is filled with the Chinese sentence from the MAC testing set.

```
You will be given a Chinese sentence to translate. If it is
   an incomplete sentence, or if you are unsure about the
   meaning, simply copy the input text as your output. Do
   not output any additional sentences, such as explanations
    or reasoning.
Chinese: {input}
```

Listing 3.5: Machine Translation - Chat Template (MT-CT) for Llama-3 models. The placeholder {input} is filled with the Chinese sentence from the MAC testing set.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
You are a helpful assistant that translates Chinese to
   English.
<|eot_id|>
<|start_header_id|>user<|end_header_id|>
You will be given a Chinese sentence to translate. If it is
   an incomplete sentence, or if you are unsure about the
   meaning, simply copy the input text as your output. Do
   not output any additional sentences, such as explanations
    or reasoning.
```

```
Chinese: {input}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Evaluation Metrics. For WebQSP, we report precision, recall, and F1 scores. For MS MARCO, we use BERTScore [171] (with the deberta-xlarge-mnli model) and report the average F1 score (*BERT-F1*). For MAC, we employ the Crosslingual Optimized Metric for Evaluation of Translation (COMET) [142], which leverages contextual embeddings from pretrained language models to assess translation quality. In addition, we report the repetition ratio (Equation 3.3) and the RAP scores (Section 3.4.2), which incorporate a penalty for repetition into the performance metrics.

3.4.4 Impact of RPP on Repetition and Performance

Figure 3.6 shows the effect of varying RPP on the repetition and performance of different open-source LLMs across three datasets and multiple prompting strategies. The results indicate that increasing RPP generally reduces repetition; however, this often comes at the expense of performance. The trends and severity vary by model. For example, Phi-3-mini's performance on the MS MARCO dataset drops significantly when RPP exceeds 1.14, whereas Llama-3-8B improves under higher RPP settings. Thus, selecting an optimal RPP is crucial to balance repetition reduction with minimal performance loss.

The datasets also behave differently. WebQSP tends to exhibit higher baseline repetition (e.g., at RPP = 1.0) than MS MARCO, particularly when using the RAG-GP strategy. In the MAC dataset for machine translation, repetition decreases with increasing RPP, even though a smaller range is tested. Notably, for both QA datasets, the Chat Template strategy consistently produces lower RR values compared to the Generic Prompt approach, indicating its effectiveness in minimizing repetition. However, this advantage is not as pronounced in the MT task, where increasing RPP

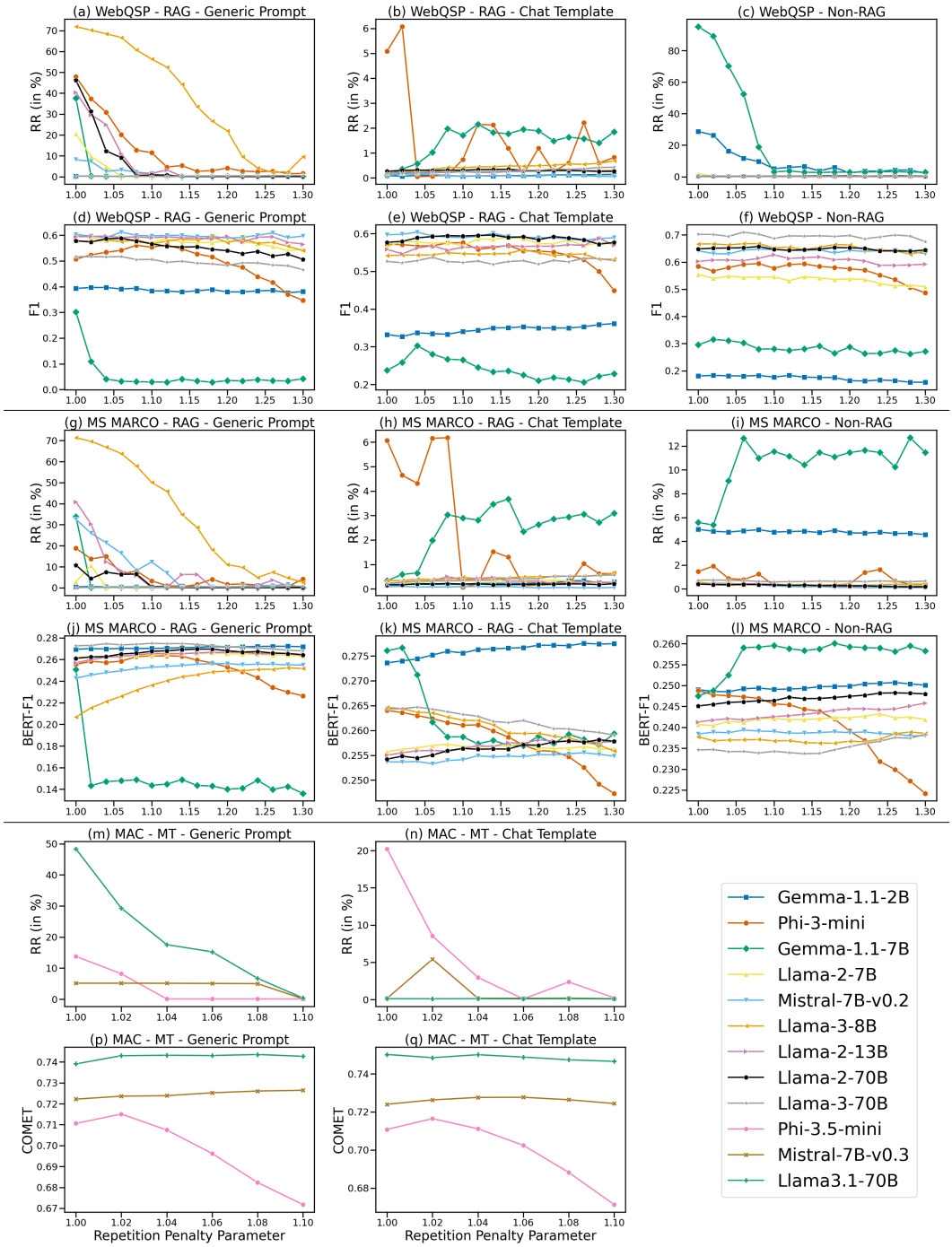


Figure 3.6: Illustrating the impact of RPP on the repetition ratio (RR) and performance across models, datasets, and prompting strategies.

effectively reduces repetition for both prompting strategies.

3.4.5 Optimizing the Trade-Off Between Performance and Repetition

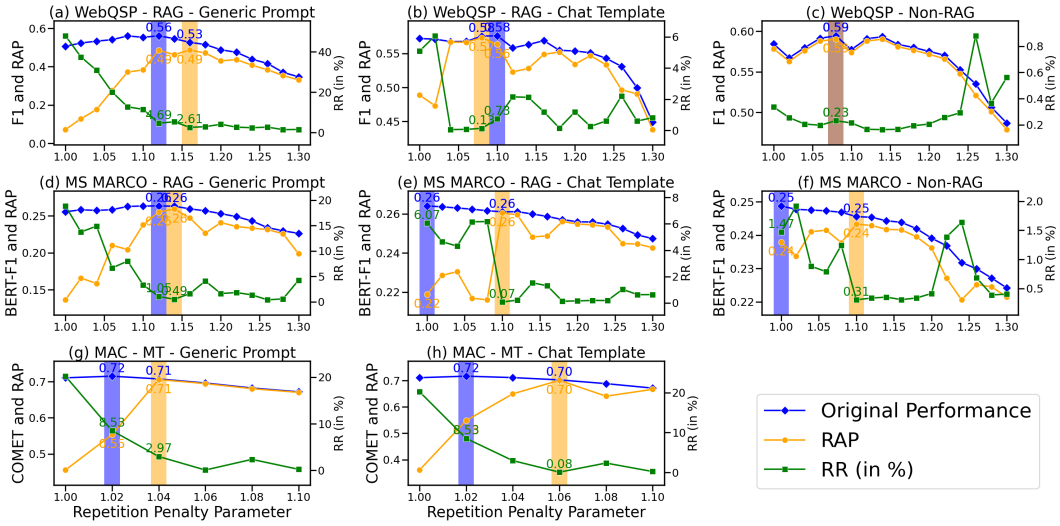


Figure 3.7: Original performance, repetition-aware performance (RAP), and repetition ratio (RR) for Phi-3-mini in QA (subplots (a) to (f)) and Phi-3.5-mini in MT (subplots (g) and (h)) as a function of RPP. Blue lines represent original performance (F1 for WebQSP, BERT-F1 for MS MARCO, COMET for MAC), orange lines indicate RAP scores, and green lines (right y-axis) show the RR in percentage. The blue shaded region marks the RPP for optimal original performance, the orange region highlights the best RAP, and brown indicates overlap between the two.

This section demonstrates the use of RAP for tuning RPP by comparing conventional performance metrics and repetition-aware performance. Figure 3.7 presents results for Phi-3-mini on WebQSP and MS MARCO, and for Phi-3.5-mini on MAC. The blue line shows the original performance, the orange line the RAP score, and the green line indicates the repetition ratio (RR). The shaded regions indicate the optimal RPP according to conventional metrics (blue) and RAP (orange); brown shading marks overlapping regions. In most cases (except Figure 3.7c), the optimal RPP differs when tuned for original performance versus RAP. For instance, in Figure 3.7e, tuning with BERT-F1 yields an optimal RPP of 1.0 with a performance of 0.264 and a high repetition ratio of 6.07%. With RAP, the optimal RPP shifts to 1.1,

where performance drops by only 1.1% (to 0.261), but the repetition ratio decreases significantly (by about 98.8%). These results highlight that tuning RPP with RAP better balances performance and repetition reduction.

3.4.6 Comparing the RAP-Tuned Performance of All Models

Table 3.5: Comparison of model performance across different datasets, LLMs, and prompting strategies.

Dataset	LLM	RAG-GP	RAG-CT	Non-RAG
WebQSP	Gemma-1.1-2B	0.397	0.361	0.175
	Phi-3-mini	0.527	0.575	0.595
	Gemma-1.1-7B	0.109	0.302	0.291
	Mistral-7B-v0.2	**0.609**	**0.604**	0.648
	Llama-2-7B	0.589	0.590	0.548
	Llama-3-8B	0.572	0.558	0.668
	Llama-2-13B	0.595	0.588	0.627
	Llama-2-70B	0.577	0.596	0.657
	Llama-3-70B	0.517	0.537	**0.710**
MS MARCO	Gemma-1.1-2B	0.272	**0.277**	**0.250**
	Phi-3-mini	0.263	0.261	0.246
	Gemma-1.1-7B	0.149	0.276	0.249
	Mistral-7B-v0.2	0.256	0.256	0.239
	Llama-2-7B	0.266	0.257	0.243
	Llama-3-8B	0.252	0.265	0.239
	Llama-2-13B	0.265	0.258	0.246
	Llama-2-70B	0.270	0.258	0.248
	Llama-3-70B	**0.275**	0.264	0.238
	LLM	**MT-GP**	**MT-CT**	
MAC	Phi-3.5-mini	0.707	0.702	
	Mistral-7B-v0.3	0.726	0.728	
	Llama3.1-70B	**0.743**	**0.750**	

Table 3.5 presents the performance of various open-source LLMs after tuning RPP using RAP. On **WebQSP**, Mistral-7B leads for both RAG-GP and RAG-CT, scoring 0.609 and 0.604, respectively, while Llama-3-70B achieves the highest F1 of 0.710 in the Non-RAG condition. For **MS MARCO**, Gemma-1.1-2B excels in both RAG-CT and Non-RAG, with scores of 0.277 and 0.250, respectively, whereas

Llama-3-70B attains the best score of 0.275 under RAG-GP. In the MT task (**MAC**), Llama-3.1-70B outperforms others in both the generic and chat template categories, scoring 0.743 and 0.750. These results demonstrate that larger models like Llama-3-70B and Mistral-7B generally deliver superior performance across datasets and prompting strategies, while mid-sized models like Gemma-1.1-2B can perform competitively on specific tasks. Model choice should therefore be task-dependent rather than based solely on size.

3.4.7 How Severe Can Repetition Be?

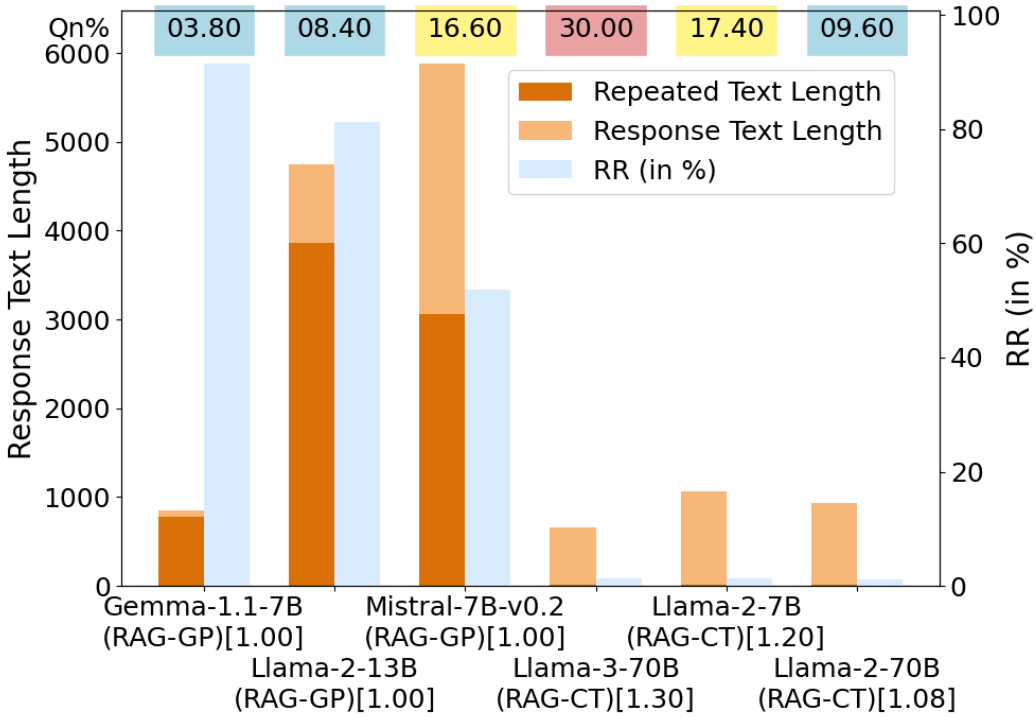


Figure 3.8: Severity of repetition. Colored squares above each point indicate the percentage of repetitive responses. The x-axis labels include the model, prompting method, and tuned RPP value.

In our experiments, repeated text can account for up to 97.85% of the overall output, and up to 41.27% of responses exhibit repetition. Figure 3.8 shows the three highest and three lowest repetition ratios (RR) for the MS MARCO dataset. Colored squares above each point indicate the percentage of repeated responses (blue: below 10%, yellow: 10–20%, red: above 20%). Notably, while Gemma-1.1-7B (RAG-GP)

shows only 3.8% of responses with repetition, in those cases the repeated text makes up 91.53% of the total output length. In contrast, although 30% of Llama-3-70B (RAG-CT)'s responses contain repetition, the repeated text comprises only 1.29% of the overall length. For example, Llama-2-13B generated repeated content averaging around 4,000 characters.

These severity findings underscore the importance of addressing repetition not only for user experience but also for downstream system reliability. In multi-agent systems (Chapter 4), repetitive outputs from one agent can propagate errors through the pipeline, degrading overall workflow success rates. The RAP metric thus serves as both an evaluation tool and a quality gate for production deployments.

3.4.8 Ablation Study: Effects of Different Penalty Functions

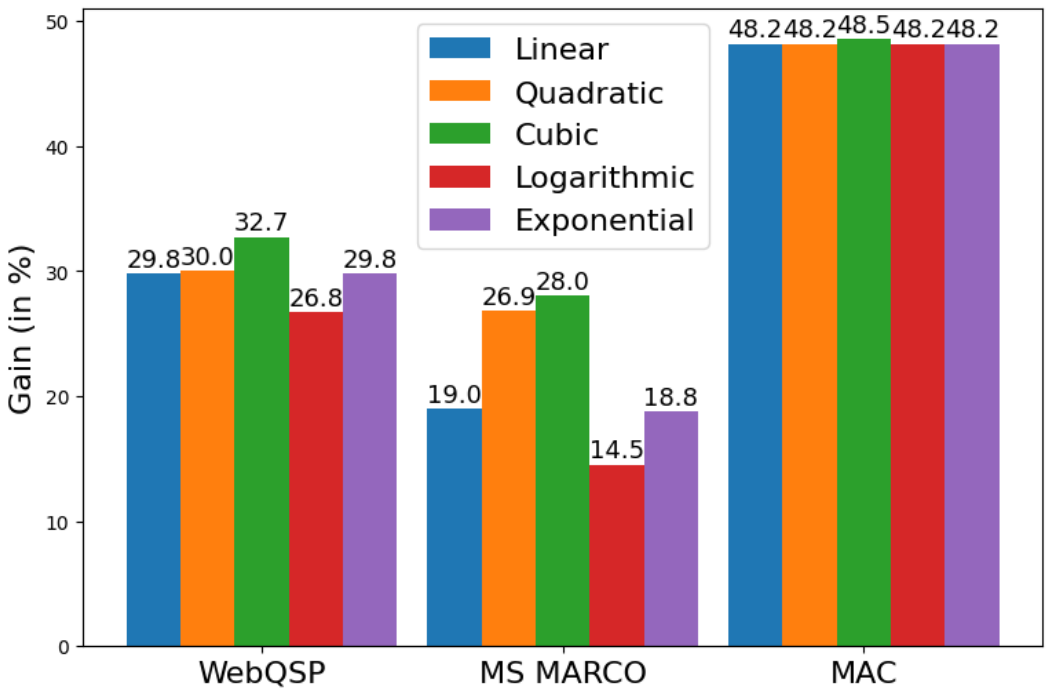


Figure 3.9: Gains across different penalty functions. The *Cubic* penalty consistently achieves the highest gains across all three datasets.

We present an ablation study to evaluate the impact of different penalty functions $F(\text{RR})$ on the RAP metric (Equation 3.4). We consider five options:

- *Linear*: $F(\text{RR}) = 1 - \text{RR}$,

- *Quadratic*: $F(\text{RR}) = (1 - \text{RR})^2$,
- *Cubic*: $F(\text{RR}) = (1 - \text{RR})^3$,
- *Logarithmic*: $F(\text{RR}) = \log_2(2 - \text{RR})$,
- *Exponential*: $F(\text{RR}) = e^{-\text{RR}}$.

We compare the *Gain* achieved with each penalty formula. For a given model, *Gain* is defined as the difference between the reduction in repetition and the performance drop when tuning the repetition penalty parameter (RPP) using original performance metrics versus RAP. In other words, it reflects the relative decrease in repetition compared to the loss in performance; a higher *Gain* indicates a more favorable trade-off.

Figure 3.9 shows the average *Gain* scores for each penalty function across the three datasets, computed by averaging the gains from all models and prompting strategies. The results indicate that the *Cubic* penalty consistently yields the highest gains (32.7% for WebQSP, 28.0% for MS MARCO, and 48.5% for MAC), offering the best balance between reducing repetition and maintaining performance. These findings recommend the *Cubic* penalty for the RAP metric across diverse datasets and tasks.

3.4.9 Summary of RAP Findings

This research tackles the challenge of reducing repeated content generation while preserving overall performance in open-source LLMs. We propose the repetition-aware performance metric, RAP, which quantifies and integrates repetition penalty into model evaluation. Conventional metrics often overlook repetition, making it difficult to tune the repetition penalty parameter (RPP). In contrast, RAP enables effective tuning of RPP and can be easily applied to other repetition methods as well.

We evaluated RAP on twelve open-source LLMs using three datasets across QA and MT tasks, with different prompting strategies. Results show that higher RPP

values generally reduce repetition but can significantly impact performance. This phenomenon emphasizes the need for tuning RPP. The experiments also demonstrate the capability of using RAP for identifying the RPP value that efficiently reduces repetition while minimizing performance loss. We also highlight the severity of the repetition issues in the generated content, calling for greater attention from the research community on this matter. Our experiments offer valuable insights into managing repetition in open-source LLMs, including a recommendation to use Chat Templates for RAG prompting.

The RAP methodology—combining detection, quantification, and penalty-based optimization—exemplifies the evaluation framework philosophy that permeates this dissertation. Just as RAP integrates output quality into traditional performance metrics, the Agentic Success Rate (Chapter 4) integrates pipeline reliability into task completion metrics, and the Reasoning Intensity metric (Chapter 5) integrates computational efficiency into accuracy assessment. This pattern of multi-dimensional evaluation addresses the fundamental insight that single-metric optimization yields systems that perform well on benchmarks but poorly in production.

3.5 Evaluation-Guided Hyperparameter Search

Building on our prior work—including our NAACL 2025 paper on the RAP metric—we propose an evaluation-guided hyperparameter search loop for optimizing Retrieval-Augmented Generation (RAG) systems. The workflow is detailed in Algorithm 2.

In this procedure, the search space Λ may be explored exhaustively or using an intelligent algorithm. The metric M can be defined as the standard performance metric P if accuracy is the only concern, but it is often augmented to incorporate aspects such as brevity or non-repetition. Our proposed RAP metric (see Section 3.4.2) is one such example. By using RAP as the optimization target, configurations that yield correct yet non-repetitive outputs are inherently favored.

Algorithm 2 Hyperparameter Search for RAG Optimization

Require: Validation set Q of queries (with ground-truth answers), initial retriever and LLM, and search space Λ for hyperparameters (e.g., retrieval and generation settings).

Ensure: Optimal hyperparameter setting λ^*.

1: **for** each candidate setting $\lambda \in \Lambda$ (or using an intelligent search strategy) **do**
2: Configure the retriever/LLM with λ (e.g., set k, τ, RPP, etc.)
3: Generate answers $\hat{Y} = \{\hat{y}_q : q \in Q\}$ using the RAG pipeline
4: Evaluate $\hat{Y}$ against ground truth Y to obtain a performance score $P(\hat{Y}, Y)$ (e.g., average accuracy or F1)
5: Compute repetition or other penalties $R(\hat{Y})$ (e.g., repetition ratio)
6: Compute the overall metric $M(\hat{Y})$, which combines performance and penalties (e.g., the RAP score as in Equation 3.4)
7: Record $(\lambda, M(\hat{Y}))$
8: **end for**
9: **return** $\lambda^* = \arg\max_\lambda M(\hat{Y}_\lambda)$

In our work, this approach is used to tune the repetition penalty parameter (RPP). Traditional grid search using only accuracy would not differentiate between two configurations with similar accuracy but differing levels of verbosity. By contrast, an augmented metric like RAP can prioritize configurations with lower redundancy. We also verify that the final chosen λ^* is robust, using a separate test set or cross-validation.

It is important to note that hyperparameter choices may interact. For example, extensive fine-tuning of the LLM may alter its sensitivity to decoding parameters, and a stronger retriever (providing higher quality documents) may permit a lower repetition penalty. Thus, hyperparameter optimization is an ongoing process that evolves with model development.

In conclusion, the RAP metric offers a nuanced evaluation of RAG systems, particularly when optimizing open-source LLMs prone to repetition. By integrating RAP into our development cycle, we ensure that our optimized RAG system retrieves and generates correct information in a succinct and user-friendly manner. Future work may extend RAP to include additional factors (e.g., factuality or coherence), but as a first step, it significantly enhances our ability to mitigate a common failure mode in open-source LLM generation.

3.6 Practical Deployment Guidelines

Based on our comprehensive evaluation, we synthesize the following guidelines for enterprise RAG deployment:

Model Selection. Open-source models such as Orca-2 and Mistral-7B can achieve generation quality competitive with proprietary alternatives while offering significant advantages in deployment flexibility, cost, and data privacy. Orca-2-13b achieved 99.3% overall RAGAS score, surpassing GPT-4 (97.5%) while operating on consumer-grade hardware. For RAG applications requiring high faithfulness, models specifically trained to follow retrieved context (rather than relying on parametric knowledge) should be prioritized.

Hardware Planning. Model size should be matched to available hardware:

- 7B-parameter models perform efficiently on consumer GPUs (RTX 4090, 24GB VRAM), achieving approximately 33 tokens/second
- 13B+ models benefit substantially from professional-grade hardware (A40, L40 with 48GB VRAM) or cloud deployment
- Memory capacity is the primary constraint for larger models; insufficient VRAM leads to significant performance degradation

These hardware findings inform the comprehensive edge deployment evaluation in Chapter 5, where we extend benchmarking across six hardware platforms.

Repetition Management. The repetition penalty parameter (RPP) should be systematically tuned using RAP rather than default values:

- RAP-guided tuning achieves up to 93.1% repetition reduction with only 3.7% performance degradation
- The cubic penalty function $F(\text{RR}) = (1 - \text{RR})^3$ provides optimal balance across datasets

- Chat templates consistently reduce repetition compared to generic prompts in RAG configurations
- Optimal RPP varies substantially across models and datasets—Phi-3-mini degrades sharply above RPP 1.14, while Llama-3-8B improves

Evaluation Strategy. Employ multi-dimensional evaluation combining faithfulness, relevance, and repetition metrics. Single-metric optimization may yield configurations that perform well on benchmarks but poorly in practice. The RAP framework demonstrates this principle: optimizing for accuracy alone selects configurations with high redundancy, while RAP-guided optimization achieves comparable accuracy with dramatically reduced repetition.

Prompt Engineering. Prompt template selection significantly impacts both retrieval quality and generation coherence:

- Chat templates reduce repetition and improve context utilization
- Model-specific templates (respecting each model's expected format) outperform generic prompts
- Standalone question generation quality affects retrieval relevance—models producing generic questions may retrieve suboptimal context

3.7 Summary

This chapter presented a comprehensive framework for optimizing Retrieval-Augmented Generation (RAG) systems built on open-source Large Language Models. Drawing from three accepted publications, we addressed the critical challenges of model selection, evaluation methodology, and generation quality that enterprises face when deploying RAG solutions.

Our comparative evaluation of LLMs on the PCI DSS benchmark revealed that smaller, efficient models such as Orca-2-13b can achieve generation quality compet-

itive with—or exceeding—proprietary alternatives like GPT-4, while maintaining practical inference speeds on consumer-grade hardware. Specifically, Orca-2-13b achieved a 99.3% overall RAGAS score with perfect faithfulness, demonstrating that model size alone does not determine RAG effectiveness. This finding—that architecture and training methodology matter more than scale—recurs throughout this dissertation and informs model selection strategies in subsequent chapters.

Central to our contribution is the Repetition-Aware Performance (RAP) metric, which addresses a pervasive yet overlooked problem in open-source LLM outputs: excessive repetition. Through systematic evaluation of twelve models across question answering and machine translation tasks, we demonstrated that repeated content can comprise over 90% of generated text in severe cases, with up to 41% of responses exhibiting redundancy. The RAP metric integrates a cubic penalty function that enables principled tuning of the repetition penalty parameter (RPP), achieving up to 93.1% reduction in repetition with minimal performance degradation (typically under 4%). Cross-task validation on machine translation confirmed RAP's generalizability beyond RAG-specific applications.

Our experimental analysis yielded several practical insights for enterprise deployment. Chat templates significantly reduce repetition compared to generic prompts in RAG configurations, providing a simple yet effective mitigation strategy. The optimal RPP varies substantially across models and datasets, underscoring the importance of systematic hyperparameter tuning rather than default settings. Additionally, hardware memory capacity critically impacts performance for larger models, with the RTX 4090 supporting efficient inference for 7B-parameter models while 13B models benefit substantially from professional-grade GPUs like the A40.

The evaluation-guided hyperparameter search procedure presented in this chapter operationalizes these findings, using RAP as the optimization target to identify configurations that balance factual accuracy, relevance, and output conciseness. This holistic approach ensures that optimized RAG systems not only retrieve and generate correct information but deliver it in a manner suitable for enterprise applications

where user experience and response clarity are paramount.

The methodological contributions of this chapter—particularly the philosophy of multi-dimensional evaluation that integrates quality considerations beyond accuracy—establish foundations for subsequent chapters. Chapter 4 extends this approach to multi-agent systems with the Agentic Success Rate (ASR) and Evaluation Completion Rate (ECR@1), while Chapter 5 applies similar principles through Reasoning Intensity (RI) and domain-specific metrics for translation and predictive analytics.

Chapter 4

Agentic AI for Workflow Automation

4.1 Introduction and Motivation

Agentic AI extends LLM capabilities beyond single-turn interactions to autonomous, goal-directed behavior. Unlike traditional AI systems with narrowly defined tasks and explicit instructions, agentic AI dynamically reconfigures its strategies based on feedback, uncertainty, and evolving objectives. This makes it particularly valuable for automating complex business workflows that traditionally require human judgment and coordination.

This chapter addresses the second research question (RQ2): *How can multi-agent LLM systems be designed and evaluated for reliable, scalable enterprise automation?* The research presented here builds upon the evaluation methodology established in Chapter 3—particularly the philosophy of multi-dimensional evaluation that integrates quality considerations beyond accuracy—and extends it to the unique challenges of multi-agent coordination. The findings inform deployment strategies examined in Chapter 5, where reasoning configuration and hardware selection interact to determine system effectiveness.

This chapter presents the design and evaluation of autonomous, LLM-driven multi-agent systems for enterprise workflow automation, synthesizing findings from six first-authored publications and one collaborative demonstration paper:

1. **Invoice Reconciliation System** [73]: Edge-deployed LLMs in multi-agent systems for SME automation (ICDSAAI 2025)

2. **Tool Invocation Reliability Framework** [81]: Diagnostic framework for multi-agent systems introducing the structured error taxonomy, validated on invoice reconciliation (ICAIBD 2026, submitted)

3. **LLM-as-Judge Framework** [76]: Scalable test coverage evaluation with novel Evaluation Completion Rate (ECR@1) metric for operational reliability (AAAI 2026 Workshops, accepted)

4. **Automated Acceptance Test Generation** [**?**]: Cooperative multi-agent system for Gherkin test generation using AutoGen Swarm, evaluated using the LLM-as-Judge framework (PAKDD 2026 Workshops, submitted)

5. **Hierarchical Multi-Agent System for Payments** [80]: End-to-end agentic payment processing with modular architecture (PAKDD 2026, submitted)

6. **Agentic Success Rate for Payment Systems** [78, 79]: Trajectory-level workflow fidelity metric for multi-agent payment systems, exposing systematic procedural deviations invisible to conventional metrics (AI4DF 2026, accepted; JAAMAS, under review)

7. **ConvPayMAS: Conversational Payment Multi-Agent System** [24]: Ongoing evolution of the research prototype toward a production agentic payment system with AP2 three-mandate verification and patent-pending security features (AAMAS 2026 demonstration, work in progress)

The chapter is organized to highlight methodological progressions: Sections 4.3–4.4 present invoice reconciliation and its evolution into a comprehensive diagnostic framework with the structured error taxonomy; Sections 4.5–4.6 demonstrate how LLM-as-Judge enables automated acceptance test generation; Sections 4.7–4.8 extend these methodologies to payment processing, introducing trajectory-level

workflow fidelity evaluation; and Section 4.8.8 describes ongoing work evolving the research prototype toward a production system with enterprise-grade security and an agentic evaluation framework.

A key contribution of this chapter is the introduction of two novel metrics that complement the Repetition-Aware Performance (RAP) metric from Chapter 3: the **Agentic Success Rate (ASR)**, introduced in Section 4.8, which measures trajectory-level workflow fidelity by comparing observed and expected agent execution sequences, and the **Evaluation Completion Rate (ECR@1)**, which measures first-attempt success rates in automated evaluation systems—both addressing critical gaps in agentic AI evaluation methodology. Together with the structured error taxonomy, these metrics enable systematic diagnosis of failure modes impossible with traditional binary success/failure evaluation.

4.2 Foundations of Agentic AI

4.2.1 Distinguishing Characteristics

Agentic AI systems are characterized by several key capabilities that distinguish them from traditional AI applications:

- **Tool Use:** Ability to interact with external tools, APIs, databases, and services
- **Planning:** Decomposing complex goals into executable sub-tasks
- **Memory:** Maintaining state across interactions for long-term objectives
- **Reflection:** Self-evaluation and strategy adjustment based on outcomes
- **Collaboration:** Coordinating with other agents and human operators

These capabilities introduce complexity beyond single-turn generation tasks. While Chapter 3 addressed optimization challenges for RAG systems involving

retrieval and generation, agentic systems must orchestrate multiple tools across multiple steps, with errors in early stages cascading through subsequent operations.

4.2.2 Challenges in Enterprise Deployment

Implementing multi-agent AI systems in enterprise settings presents several challenges [20]:

1. **Coordination Complexity:** Managing multiple LLM-powered agents is computationally intensive, raising concerns about latency and resource utilization
2. **Cost Management:** Costs escalate significantly when relying on proprietary cloud-based LLMs or high-performance infrastructure
3. **Data Privacy:** Enterprise requirements often necessitate edge deployment, introducing technical complexities
4. **Reliability:** Agent failures may stem from poor system design rather than model limitations—recent research demonstrates that LLM-based multi-agent system failures often originate from architectural decisions rather than model quality
5. **Framework Maturity:** Lack of mature orchestration frameworks complicates inter-agent communication

These challenges motivate the systematic evaluation methodology developed in this chapter, where we distinguish between orchestration failures (addressable through better prompting and architecture) and fundamental model limitations (requiring larger models or different approaches).

4.3 Multi-Agent Architecture for Invoice Reconciliation

This section presents the first of two related studies on invoice reconciliation. The initial system [73] establishes the multi-agent architecture and demonstrates feasibility, while Section 4.4 extends this work with a comprehensive diagnostic framework [81] introducing the structured error taxonomy for systematic failure characterization.

4.3.1 Problem Context

Invoice reconciliation is a critical yet labor-intensive process for businesses, particularly small and medium-sized enterprises (SMEs). The process involves matching incoming payments to outstanding invoices, cross-verifying databases, emails, and transaction documents, updating associated records and status fields, and handling exceptions and discrepancies.

Traditional automation approaches frequently fall short in handling unstructured data and accommodating complex business rules. Our multi-agent system addresses these challenges through specialized, coordinated LLM agents deployed on edge devices, ensuring data privacy while maintaining high accuracy.

4.3.2 System Architecture

Figure 4.1 presents the overall architecture. The system comprises three specialized agents operating sequentially:

Data Engineering Agent

The Data Engineering Agent serves as an intelligent interface between natural language requests and database operations, utilizing SQLite as an in-memory database for efficient business invoicing data management. Key capabilities include:

- Translating natural language queries into SQL commands

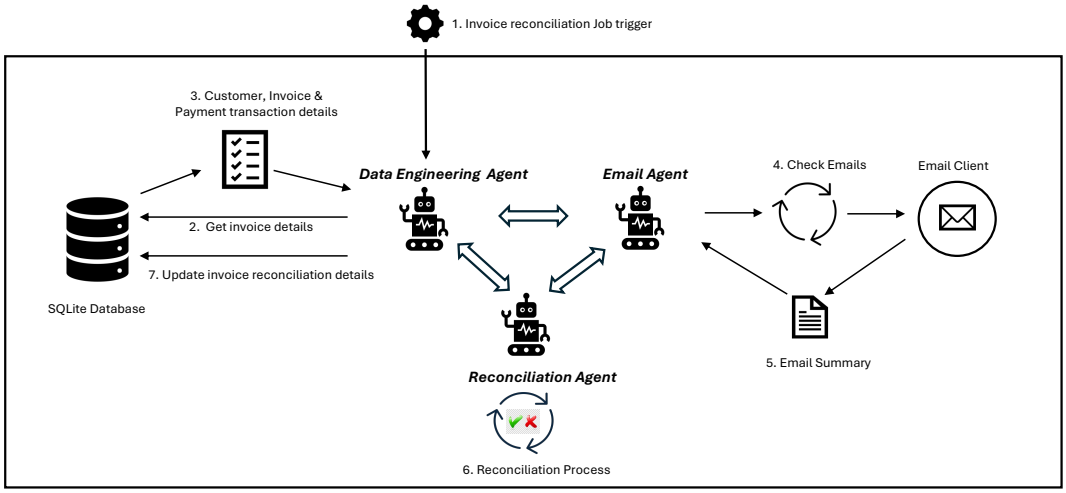


Figure 4.1: End-to-end workflow of the multi-agent architecture-based invoice reconciliation system.

- Performing schema validation and query verification
- Executing queries against SQLite databases
- Converting results back to natural language summaries

The agent's architecture incorporates two primary tools: an SQL query builder and an SQL query executor, both enhanced by few-shot prompting techniques. These prompts provide essential schema information and example queries, significantly improving the accuracy and reliability of database interactions.

Email Agent

The Email Agent processes client communications and attachments through a systematic workflow:

1. Establishes live connection with email client via API
2. Scans inbox for recent emails from customers with "Unpaid" status
3. Checks for attachments and processes them using vision-based OCR
4. Generates structured summaries of email content and extracted invoice details

Unlike proprietary models that integrate vision capabilities, open-source models such as Functionary V3.1 and Qwen2.5 excel in function calling but lack native vision capabilities. To address this limitation, we supplemented these models with Llama3.2-Vision-11B for robust image processing functionality.

Reconciliation Agent

The Reconciliation Agent integrates outputs from other agents and performs the core matching logic:

- Combines structured data from Data Engineering and Email Agents
- Applies business rules for payment-invoice matching
- Updates database records with reconciliation results
- Generates status reports and exception notifications

4.3.3 LangGraph-Based Orchestration

The system is implemented using the LangGraph framework [105], which provides graph-based architecture enabling flexible workflows, dynamic execution paths based on intermediate results, modular agent definitions for maintainability, and state management for multi-step processes.

4.3.4 Experimental Setup

Hardware Configurations

Testing was conducted on two hardware systems representing typical SME environments:

1. **RTX A6000 Desktop:** Ubuntu via WSL, 48GB VRAM, approximately $10,000 USD

2. **RTX 4090 Laptop:** Ubuntu via WSL, 16GB VRAM, approximately $5,000 USD

Due to memory constraints, only smaller models (qwen2.5:7b, functionary-small, llama3.2-vision:11b) were evaluated on the RTX 4090, while all open-source models were tested on the RTX A6000. This hardware diversity informs the comprehensive deployment evaluation in Chapter 5, where we extend benchmarking across six platforms.

Model Selection

Table 4.1 lists evaluated models.

Table 4.1: Models Evaluated for Multi-Agent Invoice Reconciliation

Model	Type	Parameters	Hardware
GPT-4o	Proprietary (API)	Unknown	Cloud
GPT-4o-mini	Proprietary (API)	Unknown	Cloud
Qwen2.5-72B	Open-source	72B	RTX A6000
Qwen2.5-32B	Open-source	32B	RTX A6000
Qwen2.5-7B	Open-source	7B	Both
Functionary-medium	Open-source	70B	RTX A6000
Functionary-small	Open-source	8B	Both
Llama3.2-vision:11B	Open-source	11B	RTX A6000

Evaluation Metrics

- **Success Rate:** Proportion of correctly completed reconciliation tasks
- **Mean Execution Time:** Average time per reconciliation workflow
- **Energy per Task:** Joules consumed per completed task
- **Performance-to-Power Ratio:** Success rate normalized by energy consumption

4.3.5 Results

Table 4.2 presents comprehensive evaluation results.

Key findings:

Table 4.2: Invoice Reconciliation Performance Across Models and Platforms

Model	Platform	SUCCESS	NO INVOICE	ERROR
GPT-4o (API)	Cloud	93.13%	–	–
GPT-4o-mini (API)	Cloud	97.68%	–	–
qwen2.5:7b	RTX 4090	865	2	123
qwen2.5:7b	RTX A6000	850	4	136
functionary-small	RTX 4090	879	101	10
functionary-small	RTX A6000	876	108	6
qwen2.5:32b	RTX A6000	**989**	0	1
functionary-medium	RTX A6000	932	47	11
qwen2.5:72b	RTX A6000	985	1	4

1. **Open-source superiority:** Qwen2.5-32B achieved 99.9% success rate (989/990), outperforming all proprietary alternatives including GPT-4o-mini (97.68%). This finding parallels the Orca-2 results from Chapter 3, where smaller open-source models outperformed GPT-4 on RAG tasks, reinforcing the theme that architecture and training methodology matter more than parameter count or proprietary status.

2. **Execution time parity:** Edge-deployed Qwen2.5-32B execution times were comparable to cloud APIs, making it a superior candidate due to combined accuracy and speed advantages

3. **Resource efficiency:** Functionary-small achieved optimal performance-to-power ratio, consuming only 523J per task on RTX 4090 while maintaining 88.78% success rate

4. **Energy trade-offs:** Larger models consume substantially more energy (19,394J for qwen2.5:72b vs 523J for functionary-small) with diminishing accuracy returns

5. **Platform variations:** RTX 4090 achieved higher success rates for qwen2.5:7b and functionary-small compared to RTX A6000, underscoring its suitability for edge applications

4.3.6 Error Analysis

Workflow outputs were categorized into three groups:

SUCCESS: Correctly located, identified, and processed invoice data. The workflow performed as expected, handling data extraction and reconciliation seamlessly.

NO INVOICE: Workflow failures due to OCR task failures (LLM hallucinations extracting non-existent information from images) or reconciliation task failures (LLM misinterpreting natural language queries and generating invalid SQL queries).

ERROR: Complete task failures from fundamental context misunderstanding, where the LLM completely misinterpreted workflow requirements.

Error trends across model families revealed that functionary models recorded higher NO INVOICE cases, indicating potential limitations in handling certain data reconciliation scenarios. Conversely, qwen2.5 family models exhibited greater ERROR occurrences, suggesting challenges in interpreting task context accurately.

While this initial study demonstrated the feasibility of edge-deployed multi-agent systems for invoice reconciliation, the coarse-grained error categories (SUCCESS, NO INVOICE, ERROR) limited our ability to diagnose specific failure modes. This motivated the development of the comprehensive diagnostic framework presented in the following section.

4.4 When Agents Fail to Act: Diagnostic Framework for Tool Invocation Reliability

Building upon the invoice reconciliation system presented in Section 4.3, this section introduces a comprehensive diagnostic framework [81] that addresses a critical limitation of the initial study: the inability to systematically diagnose *where* and *why* workflows fail. Traditional binary success/failure metrics obscure critical diagnostic information, limiting opportunities for targeted improvements.

This diagnostic approach parallels the philosophy underlying the RAP metric from Chapter 3: just as RAP integrates repetition quality into performance evaluation to enable principled hyperparameter tuning, our error taxonomy integrates

failure mode characterization into system evaluation to enable principled architecture decisions.

4.4.1 Motivation and Gap Analysis

Current evaluation approaches for multi-agent LLM systems predominantly report aggregate task success rates [56, 175], obscuring underlying failure causes. While recent work by Kokane et al. [102] demonstrates the value of granular error taxonomies for single-agent tool use, these frameworks remain insufficient for multi-agent contexts where failures cascade through coordination protocols.

Consider the invoice reconciliation pipeline from Section 4.3: a model might correctly invoke the OCR tool but fail to initiate the subsequent database update, resulting in pipeline failure. Binary metrics would score this as complete failure, obscuring that the model successfully executed partial workflow steps—a fundamentally different (and more easily addressable) problem than complete workflow incomprehension.

This diagnostic gap has significant practical implications:

- **Model selection:** Organizations cannot determine whether failures stem from model capacity limitations or prompt design issues
- **System improvement:** Without granular failure characterization, developers cannot prioritize optimization efforts
- **Deployment decisions:** Binary metrics provide insufficient information for evidence-based deployment planning across hardware configurations

4.4.2 Diagnostic Framework Overview

Our diagnostic framework provides systematic methodology for evaluating procedural reliability in multi-agent LLM systems through two core components:

1. **Structured Error Taxonomy:** Comprehensive failure characterization spanning four error types—tool initialization, parameter handling, execution, and result interpretation—applicable per tool call stage
2. **Hardware-Performance Characterization:** Systematic evaluation across deployment configurations for deployment guidance

The framework is designed for broad applicability across multi-agent scenarios while maintaining diagnostic precision through domain-specific instantiation. We validate this approach using the invoice reconciliation system from Section 4.3, demonstrating how the framework transforms coarse error categories into actionable diagnostics.

4.4.3 Structured Error Taxonomy

For comprehensive failure characterization in multi-agent coordination scenarios, we develop a systematic error taxonomy based on four generalizable error types. When applied per tool call stage, the taxonomy scales with system complexity: in the three-tool invoice reconciliation system, the four types yield twelve distinct failure modes; systems with more tool calls would produce correspondingly more categories.

Error Type Categories.

- **Not Initialized:** Agent fails to properly invoke tool due to malformed calls, incorrect tool names, or invalid parameter structures
- **Args Mismatch:** Tool invocation succeeds but contains incorrect or invalid arguments leading to tool-specific failures
- **Error:** Tool executes with valid parameters but returns runtime errors or exceptions
- **Result Mismatch:** Tool execution succeeds but produces outputs that cause downstream task failure

Tool Categories. For invoice reconciliation workflows, we define three tool categories representing common primitives in structured business automation:

- **OCR Tool:** Multi-modal document processing for receipt and invoice image analysis
- **DB Query Tool:** Database query operations for invoice lookup and verification
- **DB Update Tool:** Database modification operations for status updates

Complete Taxonomy. The cross-product produces twelve diagnostic categories:

1. `OCR_TOOL_NOT_INITIALIZED`: Failure to invoke OCR tool
2. `OCR_TOOL_ARGS_MISMATCH`: OCR tool called with invalid arguments
3. `OCR_TOOL_ERROR`: OCR tool execution returns runtime errors
4. `OCR_TOOL_RESULT_MISMATCH`: OCR output causes downstream failures
5. `DB_QUERY_TOOL_NOT_INITIALIZED`: Failure to invoke database query tool
6. `DB_QUERY_TOOL_ARGS_MISMATCH`: Query tool called with invalid arguments
7. `DB_QUERY_TOOL_ERROR`: Query execution returns runtime errors
8. `DB_QUERY_TOOL_RESULT_MISMATCH`: Query results cause downstream failures
9. `DB_UPDATE_TOOL_NOT_INITIALIZED`: Failure to invoke database update tool
10. `DB_UPDATE_TOOL_ARGS_MISMATCH`: Update tool called with invalid arguments
11. `DB_UPDATE_TOOL_ERROR`: Update execution returns runtime errors

12. `DB_UPDATE_TOOL_RESULT_MISMATCH`: Update results cause downstream failures

This taxonomy enables systematic identification of reliability bottlenecks and provides actionable diagnostic information for system improvement beyond aggregate success metrics.

4.4.4 Experimental Methodology

Enhanced Multi-Agent Architecture

We validate the diagnostic framework using the invoice reconciliation architecture from Section 4.3, enhanced with detailed instrumentation for failure diagnosis. Each agent is augmented with logging mechanisms to capture initialization failures, parameter mismatches, and execution errors at each tool invocation.

Expanded Model Selection

To establish comprehensive reliability benchmarks across the performance spectrum, we expand evaluation to include:

- **Closed-Source Models:** OpenAI's GPT-4 series (gpt-4o, gpt-4o-mini, gpt-4.1-nano, gpt-4.1-mini, gpt-4.1) and Anthropic's Claude models (3.5 Sonnet, 3.7 Sonnet) serve as reference baselines representing state-of-the-art proprietary systems

- **Open-Weight Models:** The Qwen2.5 series (3B, 7B, 14B, 32B, 72B) and Functionary V3.1 (8B, 70B) provide systematic coverage across model scales, with local deployment ensuring data sovereignty

All open-weight models were executed locally using Ollama v0.6.8 with 4-bit quantization (Q4_K_M format) to simulate realistic edge deployment constraints.

Extended Hardware Infrastructure

We extend testing to three representative edge platforms:

- **NVIDIA RTX A6000 Ada GPU:** High-end workstation (48GB VRAM, Ubuntu 24.04, ~$10K)
- **NVIDIA GeForce RTX 4090 Laptop GPU:** Mobile workstation (16GB VRAM, Ubuntu 24.04 via WSL, ~$5K)
- **Apple M3 Max:** Consumer-grade ARM platform (96GB unified memory, macOS 15.4.1, ~$4K)

Evaluation Dataset and Protocol

We employ standardized evaluation using a synthetic dataset of 1,980 test instances balanced between vision-enabled (990) and text-only (990) tasks. All experiments employ deterministic execution (temperature=0, fixed prompts), eliminating sampling variance and ensuring reproducible diagnostic assessment.

4.4.5 Results and Analysis

Performance and Reliability Trade-offs

Table 4.3 summarizes overall performance metrics across all evaluated LLM configurations.

Reliability Thresholds. Our analysis reveals clear performance thresholds informing deployment decisions:

- **Flawless performance:** qwen2.5:32b achieves perfect 100% success matching GPT-4.1, demonstrating that open-weight models can achieve closed-source reliability at sufficient scale

Table 4.3: Overall Invoice Reconciliation Performance Across LLM Configurations. SR = Success Rate. All metrics based on 1,980 test instances.

Model	Platform	SR (%)	Time (s)	Steps
Closed-source models				
gpt-4o-mini	OpenAI	99.3	7.6 ± 1.6	8.6 ± 1.2
gpt-4o	OpenAI	98.8	12.4 ± 13.2	8.5 ± 0.6
gpt-4.1-nano	OpenAI	85.7	8.6 ± 12.3	18.5 ± 18.6
gpt-4.1-mini	OpenAI	99.5	8.6 ± 2.7	9.6 ± 1.9
gpt-4.1	OpenAI	**100.0**	8.3 ± 7.7	9.9 ± 9.7
claude-3-5-sonnet	Anthropic	97.8	18.7 ± 6.3	8.5 ± 0.5
claude-3-7-sonnet	Anthropic	99.3	16.8 ± 12.3	8.5 ± 0.7
Open-weight models				
qwen2.5:3b	RTX A6000	13.9	8.1 ± 19.0	18.8 ± 14.7
qwen2.5:7b	RTX A6000	57.3	9.9 ± 10.2	20.7 ± 22.0
qwen2.5:14b	RTX A6000	96.6	$\mathbf{7.3 \pm 8.0}$	8.6 ± 2.2
qwen2.5:32b	RTX A6000	**100.0**	13.9 ± 16.9	8.5 ± 0.5
qwen2.5:72b	RTX A6000	95.1	39.3 ± 24.8	8.7 ± 4.2
functionary-small	RTX A6000	7.5	4.0 ± 21.6	5.6 ± 3.7
functionary-medium	RTX A6000	53.2	30.8 ± 242.6	7.8 ± 2.8

- **Production threshold:** qwen2.5:14b maintains 96.6–97.4% success across all hardware platforms, establishing a practical reliability threshold for cost-sensitive deployments

- **Diminishing returns:** qwen2.5:72b exhibits slightly lower success (95.1%) than its 32B counterpart, echoing the scaling law findings from Chapter 3 where larger models do not always outperform smaller alternatives

- **Fundamental limits:** Smaller models exhibit significant degradation—qwen2.5:7b achieves 49.3–58.5%, while qwen2.5:3b drops to 13.1–14.9%—highlighting fundamental capacity constraints in complex multi-agent coordination

Hardware Impact on Deployment Viability. Platform selection critically affects both latency and reliability. qwen2.5:14b executes in 7.3 seconds on RTX A6000 versus 60.0 seconds on M3 Max—an 8.2× difference highlighting the importance of optimized hardware. However, hardware alone is insufficient: qwen2.5:7b on M3 Max averages 85.9 seconds despite smaller size, requiring 20.7 steps per task

versus 8.6 steps for the larger 14B model. Execution logs show that qwen2.5:7b often enters cyclical decision loops, occasionally exceeding LangGraph's recursion limit (25 steps) and triggering forced termination—suggesting that planning efficiency improves with model capacity.

Diagnostic Error Analysis

Tables 4.4 and 4.5 present detailed error distributions across all evaluated models, categorized according to our four-type taxonomy (yielding twelve categories across the three tool stages).

Table 4.4: Error Counts and Rates for Vision Tasks (990 instances). NI = NOT_INITIALIZED, AM = ARGS_MISMATCH.

Platform	Model	OCR NI	OCR AM	DB-Q NI	DB-Q AM	DB-U NI	DB-U AM
RTX A6000	qwen2.5:3b	0	1	0	7	881 (89%)	30
	qwen2.5:7b	369 (37%)	0	151 (15%)	21	30	2
	qwen2.5:14b	0	0	38 (4%)	0	15 (2%)	0
	qwen2.5:32b	0	0	0	0	0	0
	qwen2.5:72b	0	0	28 (3%)	0	12 (1%)	0
OpenAI	gpt-4.1-nano	0	0	0	60 (6%)	20 (2%)	9
	gpt-4.1-mini	0	0	0	0	9 (1%)	0
	gpt-4.1	0	0	0	0	0	0
Anthropic	claude-3-5-sonnet	0	0	0	1	0	29 (3%)
	claude-3-7-sonnet	1	0	1	0	5	0

Table 4.5: Error Counts and Rates for Non-Vision Tasks (990 instances). NI = NOT_INITIALIZED, AM = ARGS_MISMATCH.

Platform	Model	DB-Q NI	DB-Q AM	DB-U NI	DB-U AM
RTX A6000	qwen2.5:3b	7 (1%)	17 (2%)	756 (76%)	5
	qwen2.5:7b	264 (27%)	0	7 (1%)	1
	qwen2.5:14b	8 (1%)	1	4	1
	qwen2.5:32b	0	0	0	0
	qwen2.5:72b	57 (6%)	0	0	0
OpenAI	gpt-4.1-nano	55 (6%)	24 (2%)	114 (12%)	1
	gpt-4.1-mini	0	0	0	0
	gpt-4.1	0	0	0	0
Anthropic	claude-3-5-sonnet	0	0	0	9 (1%)
	claude-3-7-sonnet	0	0	1	0

Note: ERROR and RESULT_MISMATCH categories (4 of 12 taxonomy categories) recorded zero occurrences across all configurations, likely reflecting the

deterministic nature of local database operations.

Systematic Failure Patterns. Tool initialization failures (`NOT_INITIALIZED`) dominate error distributions across all model scales. This pattern holds consistently for both vision and non-vision tasks, indicating that procedural invocation—not parameter handling or result interpretation—represents the primary reliability bottleneck. This suggests fundamental challenges in multi-agent orchestration rather than modality-specific limitations.

Model Scale Impact. Error rates demonstrate clear inverse correlation with model parameters:

- **Catastrophic failure:** qwen2.5:3b exhibits 881 database update initialization errors (89% of vision tasks)
- **Moderate improvement:** qwen2.5:14b reduces this to 15–38 errors (1.5–3.8%)
- **Flawless performance:** qwen2.5:32b achieves zero errors across all categories, matching GPT-4.1

This dramatic improvement establishes clear capacity thresholds for production deployment.

Framework Validation. The systematic patterns revealed by our taxonomy demonstrate its diagnostic value. Traditional aggregate success metrics would obscure the critical insight that tool initialization—rather than execution complexity or result interpretation—represents the primary reliability challenge. This granular understanding enables evidence-based decisions about model selection, system architecture, and deployment strategies impossible with coarse-grained evaluation alone.

Qualitative Failure Analysis

To complement quantitative taxonomy results, we conducted systematic qualitative analysis of 200 randomly sampled initialization failures from qwen2.5:3b, qwen2.5:7b, and Functionary-Small execution logs.

Failure Mode Classification. Our qualitative review identifies two dominant failure patterns:

Omission Failures (≈68% of sampled cases). Agents fail to recognize that tool invocation is required, producing natural-language responses instead. This reflects inadequate procedural reasoning—the model conceptually resolves the task but omits the necessary operational step, revealing weak coupling between semantic understanding and action execution.

Malformed Call Failures (≈32% of sampled cases). Models correctly identify tool-use needs but construct invalid function calls through incorrect tool names, invalid JSON structure, or hallucinated parameters. These patterns indicate insufficient schema grounding and weak syntactic discipline during tool invocation.

4.4.6 Deployment Tiers and Practical Guidance

Results enable clear deployment guidance based on diagnostic analysis:

Tier 1: Maximum Reliability (≥32B parameters). Models demonstrate production-ready error profiles with zero errors across all taxonomy categories. qwen2.5:32b on RTX A6000 (~$10K) achieves flawless performance comparable to GPT-4.1 while maintaining data sovereignty. Appropriate for autonomous workflows requiring high reliability.

Tier 2: Balanced Performance (14B parameters). qwen2.5:14b represents the minimum viable production configuration, achieving 96.6–97.4% success across all hardware platforms. On RTX 4090 (~$5K), it provides 96.6% reliability with 7.3s

latency—the optimal choice for most SME deployments balancing cost, performance, and accuracy.

Tier 3: Budget-Constrained (<14B parameters). Models below 14B require substantial architectural augmentation to achieve acceptable reliability, including validation layers, automated retry mechanisms, and human-in-the-loop escalation.

4.5 LLM-as-Judge for Automated Evaluation

This section presents the first of two related studies on automated software quality assurance. The LLM-as-Judge framework [76] establishes methodology for scalable test coverage evaluation, introducing the Evaluation Completion Rate (ECR@1) metric for operational reliability. Section 4.6 then demonstrates how this framework enables practical automated acceptance test generation [**?**].

4.5.1 Motivation

As agentic systems scale, manual evaluation becomes impractical. Assessing software test coverage at scale remains a bottleneck in QA pipelines. Traditional static analysis tools (JaCoCo [8], coverage.py [11], and Istanbul [90]) provide quantitative metrics but cannot assess semantic completeness—whether tests adequately capture business requirements, edge cases, and error conditions. Manual expert evaluation is accurate but prohibitively expensive at scale.

LLMs offer potential for automated semantic evaluation, but systematic investigation of accuracy, cost, and operational reliability for software testing remains limited. Our LLM-as-Judge (LAJ) framework addresses this gap with rubric-driven assessment aligned with industry best practices.

4.5.2 Framework Design

Design Principles

The LAJ framework emphasizes three core principles:

1. **Agreement with humans:** Rubric-grounded scoring aligning with expert judgment
2. **Structured outputs:** JSON-formatted responses enabling programmatic processing
3. **Deployment awareness:** Metrics capturing operational reality beyond accuracy

Evaluation Task

Given a software requirement specification R (e.g., a Jira ticket) and corresponding Gherkin acceptance test script T, the LAJ model M produces assessment $A_M(R, T) \in [0, 100]$ representing estimated coverage percentage. We evaluate against expert-provided ground truth $A^*(R, T)$.

4.5.3 Novel Reliability Metrics

Beyond traditional accuracy, we introduce metrics capturing operational reliability:

Evaluation Completion Rate (ECR@1)

ECR@1 measures the percentage of evaluations producing valid, parseable output on the first attempt:

$$\text{ECR@1} = \frac{\text{Number of successful first attempts}}{N} \times 100\% \tag{4.1}$$

Reliability failures include API timeouts, malformed JSON, and schema violations. ECR@1 reveals operational reliability variations from 85.4% to 100% across models—with direct cost implications through retry overhead.

This metric complements the diagnostic framework from Section 4.4: while the structured error taxonomy diagnoses *what* fails in multi-agent workflows, ECR@1 quantifies *how often* evaluation systems produce usable outputs—both essential dimensions for production deployment.

Adjusted Cost

To account for retry overhead during deployment:

$$C_M^{1K,adj} = C_M^{1K} \times \frac{100}{\text{ECR@1}_M} \tag{4.2}$$

This captures effective deployment cost when retries are required due to failed evaluations.

4.5.4 Experimental Setup

We evaluated 20 model configurations spanning GPT-4, GPT-5 variants with varying reasoning effort, and open-weight models on 100 expert-annotated Gherkin scripts over 5 runs (500 total evaluations).

4.5.5 Results

Table 4.6 presents multi-dimensional analysis.

Table 4.6: LLM-as-Judge Performance: Accuracy, Reliability, and Cost

Model	MAAE	ECR@1 (%)	Cost/1K	Adj. Cost
GPT-4o Mini	**6.07**	96.6	$1.01	$1.05
GPT-4o	8.34	**100.0**	$16.76	$16.76
GPT-4.1	8.14	**100.0**	$15.23	$15.23
GPT-5 (low)	7.69	**100.0**	$23.57	$23.57
GPT-5 (high)	6.16	99.8	$78.96	$79.12
GPT-OSS 120B (low)	14.00	96.6	$0.75	$0.78
GPT-OSS 20B (high)	18.78	85.4	$0.63	$0.74

Note: MAAE = Mean Absolute Assessment Error (lower is better). Costs in USD per 1,000 evaluations.

Key Findings

Accuracy and reliability are independent dimensions: GPT-5 (low) achieves 100% ECR@1 but moderate accuracy (7.69 MAAE), while GPT-4o Mini excels in both (6.07 MAAE, 96.6% ECR@1). Multi-dimensional evaluation is essential for production deployment.

Smaller models can outperform larger ones: GPT-4o Mini (6.07 MAAE) surpasses GPT-4o (8.34 MAAE) and GPT-4.1 (8.14 MAAE), indicating that model optimization matters more than parameter count. This finding reinforces the scaling law nuances observed across all chapters of this dissertation.

Reliability failures have material cost impact: Low ECR@1 models incur retry overhead substantially increasing operational costs. GPT-OSS 20B (high) at 85.4% ECR@1 requires average 1.17 attempts per evaluation. At 1M evaluations/month, reliability issues cost $1,200 annually plus latency variance.

Cost spans 175×: From $0.45/1K (GPT-OSS 20B low) to $78.96/1K (GPT-5 high), enabling informed model selection based on deployment constraints.

Production Guidance

GPT-4o Mini emerges as production-optimal: best-in-class accuracy (6.07 MAAE), high reliability (96.6% ECR@1), and exceptional cost-effectiveness ($1.01/1K)—delivering 78× cost reduction versus GPT-5 (high reasoning) while maintaining superior accuracy.

4.6 Multi-Agent System for Acceptance Test Generation

Building upon the LLM-as-Judge framework presented in Section 4.5, this section presents CMAS4G2 (Cooperative Multi-Agent System for Gherkin Generation) [**?**], demonstrating that multi-agent architectures can automate behavior-driven devel-

opment (BDD) test generation using open-weight models on consumer hardware. The LAJ framework from Section 4.5 serves as the evaluation mechanism, enabling scalable quality assessment of generated tests.

4.6.1 Problem Context

Automated acceptance testing remains difficult: translating high-level requirements into executable tests demands domain expertise, sustained effort, and careful maintenance. Traditional tools struggle to bridge the semantic gap between natural-language requirements and testable specifications. While LLMs show promise in code generation, their application to Gherkin-style acceptance testing remains underexplored. The central challenge is producing tests that are both semantically faithful to requirements and executable in CI/CD pipelines.

4.6.2 Architecture

Figure 4.3 presents the overall architecture of the CMAS4G2 framework. The system adopts a circular workflow composed of three specialized agents, with Jira and Git integrated via APIs to support coordinated execution and automated state updates.

CMAS4G2 leverages Microsoft's AutoGen Swarm framework to orchestrate interactions among three specialized agents:

- **Supervisor Agent:** Acts as the central orchestration component, managing workflow execution and ensuring correct handoffs between specialized agents.
- **Validator Agent:** Focuses on Jira ticket analysis and content validation. This agent establishes direct API connectivity with Jira to retrieve ticket data and perform systematic validation checks.
- **Quality Assurance Agent:** Transforms validated requirements into executable Gherkin acceptance tests, combining domain knowledge in test design patterns with automated code generation capabilities.

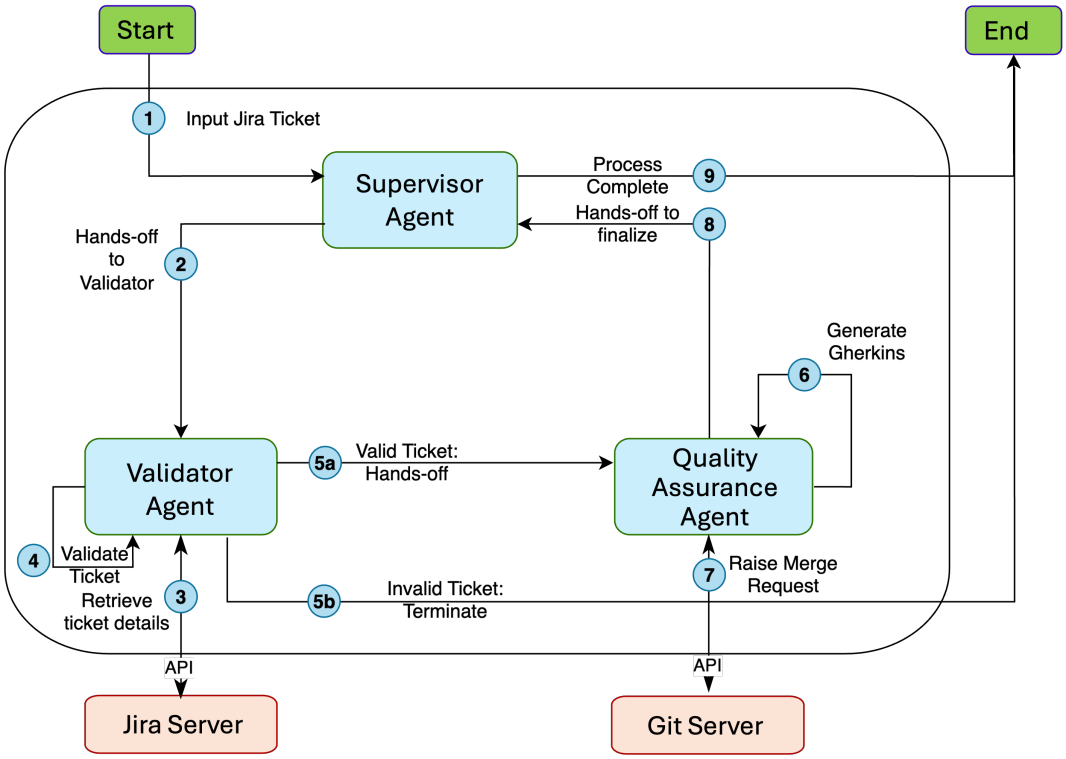


Figure 4.2: Architecture and operational workflow of CMAS4G2. The diagram illustrates the nine-step process involving the Supervisor, Validator, and Quality Assurance agents.

The end-to-end workflow proceeds through the following phases:

1. **Initialization Phase:** The Supervisor Agent receives the input Jira ticket and orchestrates its handoff to the Validator Agent.

2. **Validation Phase:** The Validator Agent retrieves ticket details via the Jira API and performs content validation. Tickets that fail validation are rejected with documented deficiencies.

3. **Generation Phase:** Upon successful validation, the Quality Assurance Agent generates Gherkin test scripts and creates automated merge requests in Git.

4. **Completion Phase:** The Supervisor Agent manages the final coordination steps and concludes the workflow.

4.6.3 Evaluation Framework

We employ the LLM-as-a-Judge (LAJ) framework from Section 4.5 for evaluating generated Gherkin scripts, using GPT-4o Mini as the judge model following calibration against expert human annotations. This integration demonstrates the practical utility of the LAJ framework for downstream agentic applications.

Dataset

The benchmark comprises 200 synthetic Jira tickets based on the open-source Kill Bill subscription billing platform:

- **Valid Tickets (IDs 1–100):** Complete tickets for successful Gherkin generation
- **Invalid Tickets (IDs 101–200):** Deliberately incomplete tickets to test rejection logic

Performance Metrics

- **Valid Ticket Success Rate:** Percentage of well-formed tickets for which the system successfully generates Gherkin scripts
- **Invalid Ticket Success Rate:** Percentage of incomplete tickets correctly rejected before test generation
- **Overall Success Rate:** Combined success: (Valid SR + Invalid SR)/2
- **Average Test Coverage:** Mean coverage score assigned by the LAJ framework

4.6.4 Results

Table 4.7 presents performance across model configurations.

Table 4.7: CMAS4G2 Performance on 200-Ticket Kill Bill Benchmark. Values are mean±std across 5 runs.

Model	Config	Valid (%)	Invalid (%)	Overall (%)	Coverage (%)
Claude Sonnet 4	–	**100.0±0**	**80.8±0**	**90.4±0**	**91.6±0**
GPT-OSS 20B	medium	99.2±2	76.0±0	87.6±1	85.4±0
	high	1.6±3	73.0±0	37.3±2	16.8±34
Qwen3-32B	no think	99.2±1	35.2±0	67.2±1	86.9±0
	think	87.2±5	69.0±0	78.1±2	83.7±0
Qwen3-8B	no think	58.8±24	41.0±0	49.9±12	79.5±3
	think	98.0±2	57.0±0	77.5±1	79.2±0

Key Findings

Open-Weight Models Approach Proprietary Performance. GPT-OSS 20B (medium) achieves 87.6% overall success and 85.4% coverage, approaching within 2.8 percentage points of Claude Sonnet 4's 90.4% and 91.6% while enabling fully local deployment.

Reasoning Configuration Critically Affects Quality. The impact of reasoning mode varies dramatically by model size:

- **Small models benefit dramatically:** Qwen3-8B improves from 49.9% to 77.5% overall success with thinking mode—a 27.6 percentage point gain
- **Large models show mixed effects:** Qwen3-32B degrades from 99.2% to 87.2% valid ticket success with thinking mode, while improving invalid ticket rejection from 35.2% to 69.0%
- **Excessive reasoning causes catastrophic failure:** GPT-OSS 20B (high) collapses to 1.6% valid success due to context exhaustion before completing Gherkin generation

These reasoning configuration findings directly inform Chapter 5, where we systematically characterize the relationship between reasoning intensity and task complexity across sentiment analysis tasks.

Invalid Ticket Rejection Remains Challenging. All models struggle with invalid ticket rejection compared to valid ticket processing. Claude Sonnet 4 achieves only 80.8% rejection rate, with open-weight models ranging from 35.2% to 76.0%. This asymmetry suggests that correctly identifying *missing* information is harder than processing *present* information.

4.6.5 Integration Summary

The successful integration of CMAS4G2 with the LAJ framework demonstrates the practical value of our multi-dimensional evaluation approach. By establishing ECR@1 and adjusted cost metrics in Section 4.5, we enabled reliable, cost-effective evaluation of generated tests at scale—a critical capability for operationalizing automated test generation in enterprise CI/CD pipelines.

4.7 Hierarchical Multi-Agent System for Payments

While the preceding sections focus on back-office automation and software quality assurance, payment processing presents distinct challenges: traditional payment infrastructure employs anti-bot mechanisms, multi-factor authentication, and fraud detection systems that constrain agentic technology adoption. This section presents HMASP (Hierarchical Multi-Agent System for Payments) [80], the first LLM-based multi-agent system to implement end-to-end agentic payment workflows.

4.7.1 Architecture Overview

HMASP addresses end-to-end agentic payment processing through a modular, four-level hierarchical design implemented using LangGraph [105]. Figure 4.3 illustrates the architecture.

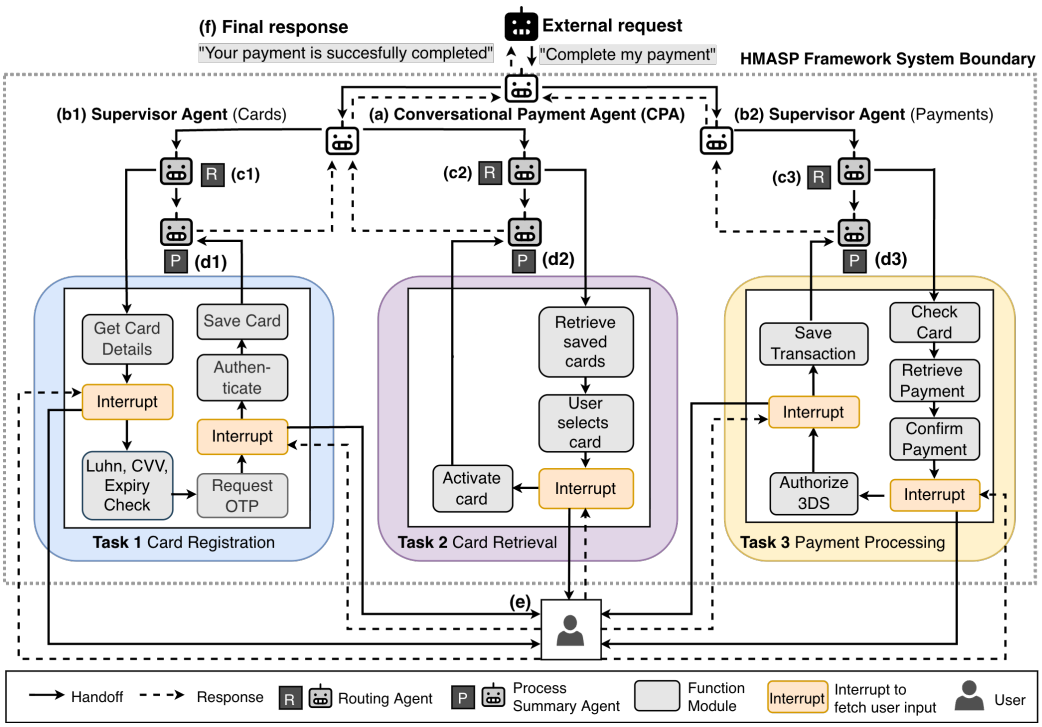


Figure 4.3: Overview of the proposed HMASP. An external request (e.g., "Complete my payment") is sent to the Conversational Payment Agent (CPA - first agent level); (a) The CPA processes the request and delegates it to one of the two Supervisors (b1, b2 - second agent level) for further handling; (b) The selected Supervisor routes the request to a suitable Routing agent (c1, c2, or c3 - third agent level); (c) Routing agents initiate the corresponding task-specific workflow (Task 1, Task 2, Task 3); Upon initiation, a series of task-relevant function modules are triggered to complete the request; (d) Upon task completion, the Process summary agent (d1, d2, d3 - fourth agent level) compiles the outcome and generates a response. This summary response is propagated upward the system hierarchy through Supervisor agents and CPA; (e) When user input is required to complete the task, an interrupt mechanism is triggered to obtain the necessary input before execution resumes; (f) Finally, the CPA responds to the request (e.g., "Your payment has been successfully completed").

Agent Hierarchy

The hierarchy comprises four distinct agent levels:

1. **Conversational Payment Agent (CPA):** Primary orchestrator receiving external requests. The CPA handles all incoming requests and coordinates task delegation to supervisors. It can also reject irrelevant requests.

2. **Supervisor Agents:** Decision-makers that delegate requests based on context and workload to appropriate sub-workflows. Each supervisor is responsible for

reporting outcomes back to the CPA.

3. **Routing Agents:** Identify and initiate task-specific workflows. They serve as the final decision layer on whether to trigger payment-related function modules or reject irrelevant requests.

4. **Process Summary Agents:** Interact with underlying systems to execute operations and generate structured responses on workflow outcomes.

Architectural Patterns

HMASP incorporates several architectural patterns ensuring reliable payment processing:

Shared State Variables: Graph States consist of user-defined variables that persist throughout MAS runtime. Agents can only access their assigned states, preventing unauthorized read/write across state variables.

Information Determinism: Critical payment information is stored in state variables rather than generated by LLMs. This decouples payment-related processing from generative processes—payment amounts, card numbers, and transaction IDs are retrieved from state variables, never hallucinated.

Structured Handoffs: Agents use LangGraph's handoff mechanism to seamlessly pass processes to one another, enabling coordination where requests flow from CPA through the hierarchy.

Interrupt Mechanism: When user input is required during execution (e.g., OTP verification, card selection), an interrupt mechanism pauses the workflow to collect necessary input before proceeding.

4.7.2 Evaluation

We constructed a dataset of 1,000 payment-related inputs across four task categories: Card Registration (250), Card Retrieval (250), Payment Checkout (250), and Irrelevant Inputs (250).

4.7.3 Results

Table 4.8 presents task success rates across models.

Table 4.8: HMASP Task Success Rates (%) Across Models

Model	Task 1	Task 2	Task 3	Task 4
GPT-4.1 (Baseline)	**99.6**	**99.6**	**99.6**	**100.0**
Qwen2.5:32b	97.2	98.4	98.8	100.0
Qwen2.5:14b	94.0	95.2	98.8	100.0
Qwen3:32b	99.6	35.2	39.6	100.0
Mistral-small3.2:24b	94.8	94.0	82.8	100.0
Llama3.1:70b	87.2	88.4	47.2	33.2
Llama3.1:8b	38.0	38.4	39.2	3.6

Key Findings

Open-Source Viability: Qwen2.5:32b achieves an average 97.0% task success rate across payment-relevant tasks (Tasks 1–3), demonstrating viability for end-to-end agentic payments with open-weight models.

Inconsistent Performance Across Tasks: Some models show inconsistent behavior—Qwen3:32b achieves 99.6% on Task 1 but only 35.2% and 39.6% on Tasks 2 and 3, indicating that strong performance on one task does not guarantee reliability across the workflow.

Irrelevant Input Handling: Most models correctly reject irrelevant inputs (Task 4), with Qwen2.5 variants achieving 100%. However, Llama3.1 models showed weakness, with Llama3.1:8b achieving only 3.6%.

4.7.4 Novelty and Contributions

To our knowledge, HMASP is the first LLM-based multi-agent system supporting fully end-to-end agentic payment workflows. Table 4.9 provides comparative analysis with existing approaches.

Method	Open-Weight LLM	Full Payment Processing	TR Success Rate (%)	TR Precision-Recall (%)	TR F1-Score (%)
LLMPA [53]	Yes	No[1]	76.5	–	–
Method 1 [28]	No	No	93.3	85.7–30.2	44.77
Method 2 [135]	Yes	No	–	N/A[2]–73.7	–
HMASP (Ours)	**Yes**	**Yes**	**97.7**	**99.4–98.8**	**99.2**

[1] Processing stops at network boundary; defaults to in-app checkout.
[2] Not reported.

Table 4.9: A comparative overview of existing autonomous agents in payment-relevant processes with their Task-Relevant (TR) metrics. For HMASP, averages from the best-performing open-weight LLM (Qwen2.5:32b) were reported.

4.8 Agentic Success Rate: Measuring Workflow Fidelity in Multi-Agent Payment Systems

While Section 4.7 demonstrated the feasibility of hierarchical multi-agent payment processing, the evaluation relied exclusively on Task Success Rate (TSR) and Agent Handoff F1-Score (HF1)—metrics that capture *what* the system achieves but not *how* it gets there. This section introduces the **Agentic Success Rate (ASR)** [78], a trajectory-level metric that exposes systematic procedural deviations invisible to both TSR and HF1. Applied to the HMASP architecture redesigned for inter-agent standards compliance, ASR reveals that models achieving perfect scores on conventional metrics nonetheless skip required confirmation checkpoints during payment checkout—a finding with direct regulatory significance for PCI-DSS compliance [131]. This section extends the 6-page conference version [78] with deeper deviation analysis, per-sample trajectory decomposition, and input-phrasing sensitivity analysis from the 18-model, 90,000-instance evaluation.

4.8.1 Motivation: Why Task Success Is Insufficient

The limitations of TSR and HF1 become apparent when applied to multi-step payment workflows. TSR measures only whether the final outcome is correct—a payment completes, a card is registered—without examining intermediate steps. HF1 evaluates routing decisions in a bag-of-edges fashion, ignoring their temporal order and the overall path structure. Neither captures *workflow fidelity*: whether the agent followed the expected sequence of steps efficiently and completely.

In agentic payment systems, where auditability and compliance are paramount, understanding *how* an agent reached an outcome is as important as the outcome itself. This concern is echoed by recent work arguing that standard benchmarks fail to capture the risks LLM agents pose in financial applications [21], and by failure analyses showing that multi-agent system breakdowns often stem from procedural deviations rather than outright task failures [19]. PCI-DSS requires auditable transaction workflows where each processing step is logged; bypassing expected control returns creates gaps in the audit trail even when transaction outcomes are correct.

4.8.2 Architecture Adaptations for A2A and UCP Compliance

To apply ASR in a standards-compliant setting, we redesign the HMASP architecture (Section 4.7) for inter-agent operability, spanning three subsystems and four layers (Figure 4.4):

1. A **UCP-compliant gateway** that exposes discovery profiles and checkout lifecycle endpoints, built on the Unified Commerce Protocol (UCP) [155] achieving approximately 85% compliance including full discovery profiles, checkout session lifecycle management, and JWS detached signatures (ES256) for request integrity.

2. An **A2A protocol layer** that translates JSON-RPC 2.0 calls into LangGraph invocations via the Agent-to-Agent (A2A) protocol [49], an open standard for

inter-agent communication.

3. The **LangGraph agent hierarchy** implementing the HMASP agents in a consolidated three-level structure: the CPA (Level 1) routes to domain-specific Supervisors—Cards Supervisor (**CS**) and Payment Supervisor (**PS**)—at Level 2, which dispatch to task-specific Workflows—Card Registration Workflow (**CRW**), Card Selection Workflow (**CSW**), and Payment Processing Workflow (**PPW**)—at Level 3. An Off-Topic Handler (**OTH**) at Level 2 rejects non-payment inputs.

4. An **MCP payment gateway** providing card validation, OTP, 3D-Secure, and tokenization services as tool calls.

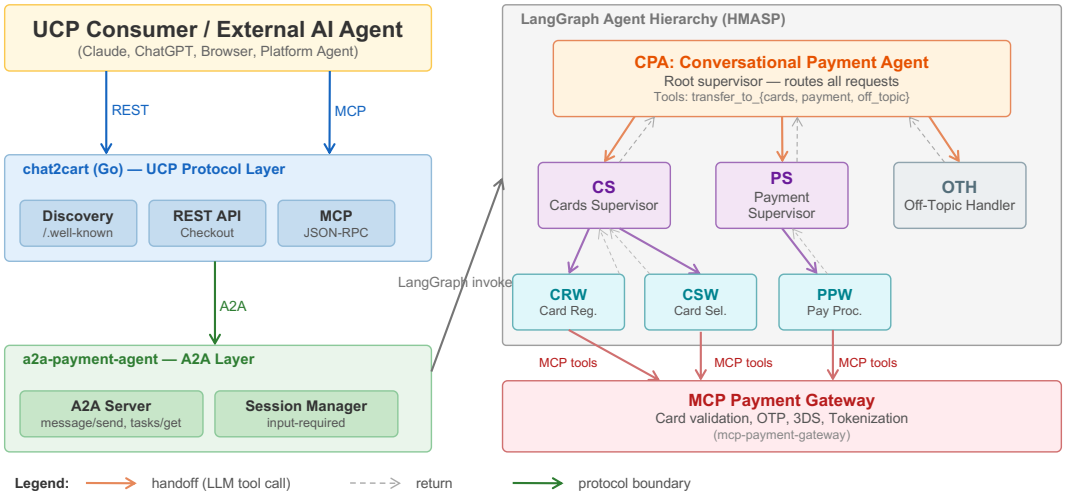


Figure 4.4: Standards-compliant HMASP architecture. External agents interact via UCP-compliant REST/MCP endpoints (top), which delegate to the A2A protocol layer (middle). The LangGraph agent hierarchy (centre) implements the HMASP agents: the CPA routes to domain supervisors (CS, PS) and the Off-Topic Handler (OTH); supervisors dispatch to task-specific workflows (CRW, CSW, PPW). Workflows invoke the MCP Payment Gateway (bottom) for card validation, OTP, 3DS, and tokenization. Solid arrows denote handoffs (LLM tool calls); dashed arrows denote returns.

A key architectural change is the replacement of LangGraph's interrupt mechanism with a multi-turn state-machine protocol. While the original HMASP relies on interrupts for human-in-the-loop interactions (e.g., collecting card details, OTP verification), this mechanism assumes direct interaction with the LangGraph runtime.

In the A2A design, the agent graph returns an input-required status to the calling agent along with a session identifier; the external agent collects the needed information and re-invokes the graph with the session context. Each dialogue turn—initial request, user confirmation, and verification code entry—maps to a separate A2A message/send call. This design preserves multi-turn interaction semantics while making the CPA interoperable with any A2A-compliant client agent.

4.8.3 Agentic Success Rate (ASR)

ASR compares the full ordered trajectory of agent execution against the expected one at the *transition* (bigram) level, drawing on the conformance checking paradigm from process mining [156, 18]. Each consecutive pair of agents in a trajectory is a transition; for example, the trajectory $[A, B, C]$ yields the transition multiset $\{(A,B),\ (B,C)\}$. Given expected trajectory $E = [e_1, \ldots, e_n]$ and observed trajectory $O = [o_1, \ldots, o_m]$, let B_E and B_O denote their respective transition multisets:

$$\text{TR} = \frac{|B_E \cap B_O|}{|B_E|}, \quad \text{TP} = \frac{|B_E \cap B_O|}{|B_O|}, \quad \text{ASR} = F_1(\text{TR}, \text{TP}) \tag{4.3}$$

An ASR of 100% indicates perfect transition alignment. Low Transition Recall (TR) signals skipped handoffs ("model moves" in process mining terminology); low Transition Precision (TP) signals redundant ones ("log moves"). Because ASR operates on transitions rather than individual agent visits or unordered edge sets, it directly measures whether each agent handed off control to the correct next agent—precisely the property needed for compliance auditing.

Expected Trajectories

Table 4.10 specifies the expected trajectories for each task scenario. The multi-turn structure reflects the A2A protocol design: each dialogue turn requires a full round trip through the agent hierarchy, with the CPA returning control to the external agent between turns to collect user input or deliver intermediate results.

Table 4.10: Expected trajectories per scenario. → denotes agent handoff; $[\cdots]^{\times 2}$ denotes two identical consecutive repetitions (one per remaining dialogue turn). CPA: Conversational Payment Agent; PS: Payment Supervisor; PPW: Payment Processing Workflow; CS: Cards Supervisor; CRW: Card Registration Workflow; CSW: Card Selection Workflow; OTH: Off-Topic Handler.

Task	Scenario	Steps	Expected Trajectory
T1	Card registration	13	CPA→CS→CRW→CS→CPA→[CS→CRW→CS→CPA]$^{\times 2}$
T2	Card retrieval	5	CPA→CS→CSW→CS→CPA
T3	Payment processing	11	CPA→PS→CPA→[PS→PPW→PS→CPA]$^{\times 2}$
T4	Irrelevant input	1	CPA *(alt.: CPA→OTH→CPA, 3 steps)*

The T3 trajectory is central to our analysis. Its 11 steps encode three dialogue turns: the CPA hands off to the Payment Supervisor, which returns to the CPA to prompt for explicit user *confirmation* before payment processing; after confirmation, PS invokes the Payment Processing Workflow for 3DS initiation; and a second PPW call completes the transaction. Table 4.11 documents the purpose of each step.

Table 4.11: T3 (Payment Processing) trajectory detail: 11 steps across 3 dialogue turns. Each step documents the agent activated and its role in the workflow.

Step	Agent	Turn	Purpose
1	CPA	1	Receives "Complete my purchase" request; routes to PS
2	PS	1	Retrieves cart/amount; presents payment review to user
3	CPA	1	Delivers review summary; returns `input-required` for confirmation
4	PS	2	Receives user confirmation "yes"; dispatches to PPW
5	PPW	2	Initiates 3DS authentication; requests verification code
6	PS	2	Propagates 3DS prompt upward
7	CPA	2	Delivers 3DS prompt; returns `input-required`
8	PS	3	Receives 3DS code; re-dispatches to PPW
9	PPW	3	Processes payment transaction; confirms success
10	PS	3	Propagates completion result upward
11	CPA	3	Delivers payment success message to external agent

4.8.4 Experimental Setup

Models. We evaluate 18 models spanning open-weight and proprietary LLMs: 11 from the original HMASP study [80] (Qwen3 8B/14B/32B, Qwen2.5 7B/14B/32B, Mistral-Small-3.2 24B, Magistral 24B, Llama 3.1 8B/70B, GPT-4.1) plus 7 additional models (GPT-5.2, GPT-OSS 20B/120B, Llama 3.3 70B, Qwen3 30B, GLM-4.7-Flash, and Mistral 7B). Proprietary models (GPT-4.1, GPT-5.2) are accessed via the OpenAI API; all open-weight models run locally using Ollama on an Apple M3 Max with 96 GB unified memory and 4-bit quantization (MXFP4 for GPT-OSS models; Q4_K_M for all others).

Dataset and Repeat Protocol. We use the HMASP evaluation dataset of 1,000 data points across four scenarios (250 each): T1 (card registration), T2 (card retrieval), T3 (payment processing), and T4 (irrelevant input rejection). Each model–scenario pair is evaluated 5 times independently, yielding 90,000 total instances (18 models $\times$ 1,000 data points $\times$ 5 repeats). We report mean and standard deviation across repeats.

System Engineering. We improve the HMASP system through prompt engineering (iterative refinement of agent roles, expected handoffs, and task boundaries) and two deterministic routing guards: a *cart-items guard* in the payment supervisor that suppresses the payment-review path when `cart_items` is empty, and *card-view pattern matching* in the CPA for keyword-based fallback routing on ambiguous card intents.

4.8.5 Results

Effectiveness of System Engineering

Table 4.12 compares TSR between the original HMASP [80] and our system for representative models: the two strongest HMASP baselines and the three most-improved

open-weight models (ranked by average Δ across all tasks). The top-baseline models, GPT-4.1 and Qwen2.5:32b, were already near-saturated and remain so under our system ($\Delta \leq +4.4$ pp), validating that strong baselines are not regressed. Notably, Qwen2.5:32b achieves perfect TSR (100%) with zero variance across all four task types, making it the only model to do so; GPT-4.1 shows marginal regressions on T1 (-1.4 pp) and T3 (-0.2 pp), likely attributable to stochastic decoding rather than systemic failure. The gains for previously struggling models are striking: Llama3.1:8b improves by an average of +67.9 pp across tasks, Magistral:24b by +54.2 pp, and Llama3.1:70b by +33.5 pp—confirming that the original failures were addressable through system engineering rather than model replacement. The most dramatic single-task improvements are Llama3.1:8b's T1 TSR leap from 6.0% to 99.8% (+93.8 pp) and Magistral:24b's T3 TSR surge from 3.2% to 94.7% (+91.5 pp). Results are stable across 5 repeats ($\sigma \leq 1.9$ for all reported cases).

Table 4.12: HMASP vs. Ours: TSR (%) for top HMASP-baseline models and most-improved models. H = HMASP baseline, O = Ours (mean $\pm$ std over 5 repeats), Δ = improvement. **Bold**: $\geq$20 pp gains. HMASP-top models selected by baseline TSR; top-3 improved open-weight models ranked by average Δ across all tasks.

	T1			T2			T3			T4		
Model	**H**	**O**	**Δ**	**H**	**O**	**Δ**	**H**	**O**	**Δ**	**H**	**O**	**Δ**
HMASP Top Baseline												
GPT-4.1	99.6	98.2±0.4	−1.4	99.6	99.9±0.2	+0.3	100	99.8±0.4	−0.2	100	100	+0.0
Qwen2.5:32b	96.4	100	+3.6	98.8	100	+1.2	95.6	100	+4.4	100	100	+0.0
Top-3 Improved												
Llama3.1:8b	6.0	99.8±0.2	**+93.8**	10.4	98.8±0.5	**+88.4**	4.0	93.4±0.9	**+89.4**	100	100	+0.0
Magistral:24b	44.0	90.0±1.9	**+46.0**	18.4	97.7±1.0	**+79.3**	3.2	94.7±1.2	**+91.5**	100	100	+0.0
Llama3.1:70b	66.0	100	**+34.0**	74.0	100	**+26.0**	26.0	98.9±1.2	**+72.9**	98.8	100	+1.2

ASR Reveals Hidden Workflow Deviations

Table 4.13 presents TSR, HF1, and ASR for the top 8 models on the T3 (Payment) scenario, alongside averages across T1–T3. All models achieve 100% on T4; T1 and T2 are near-perfect ($\geq$97.7% TSR) and omitted for brevity. A striking result is that **all six open-weight models outperform both proprietary GPT models** on average ASR, with the top four achieving $\geq$99.9% compared to 98.4% for GPT-4.1

and GPT-5.2. Every model *except* `mistral-small3.2:24b` exhibits a T3 ASR gap—from 0.01% (`qwen2.5:32b`) to 2.9% (`GPT-4.1`)—despite high TSR and HF1. Notably, `mistral-small3.2:24b` achieves a perfect 100% across all three metrics, demonstrating that enforcing the confirmation checkpoint is achievable without sacrificing task performance. The highlighted cells are the sharpest examples: `qwen2.5:32b` and `qwen3:8b` achieve T3 TSR = HF1 = 100% yet ASR $<$ 100%, exposing skipped confirmation checkpoints invisible to conventional metrics.

Table 4.13: T3 (Payment) results (%, mean $\pm$ std over 5 repeats) and average across T1–T3 for the top 8 models. Sorted by Avg ASR desc. Yellow: TSR = HF1 = 100% but ASR $<$ 100%. All six open-weight models outperform both proprietary GPT models.

	T3 (Payment)			Avg (T1–T3)		
Model	**TSR**	**HF1**	**ASR**	**TSR**	**HF1**	**ASR**
MSmall3.2:24b	100	100	100	100.0	100.0	100.0
Qwen2.5:32b	100	100	99.99±0.02	100.0	100.0	100.00±0.01
Qwen3:8b	100	100	99.97±0.02	99.9±0.1	99.93±0.05	99.9±0.1
Qwen3:14b	99.7±0.3	99.8±0.2	99.6±0.3	99.9±0.1	99.9±0.1	99.9±0.1
Llama3.1:70b	98.9±1.2	98.8±1.1	98.8±1.2	99.6±0.4	99.6±0.4	99.6±0.4
Qwen2.5:14b	99.8±0.2	99.8±0.2	99.6±0.2	99.1±0.5	99.5±0.3	99.4±0.3
GPT-4.1	99.8±0.4	99.9±0.2	97.1±0.3	99.3±0.3	99.7±0.2	98.4±0.3
GPT-5.2	99.9±0.2	100.0±0.1	97.3±0.1	99.3±0.2	99.6±0.1	98.4±0.2

Root Cause: The T3 Confirmation Shortcut

The root cause is consistent across 7 of the 8 top models. The expected T3 trajectory has 11 agent hops (10 transitions), including an intermediate return to the CPA for explicit user *confirmation* before payment processing. For direct payment phrases (e.g., "Process my payment"), models infer unambiguous intent and skip this checkpoint, executing a 9-step trajectory (8 transitions) instead. Table 4.14 details the step-by-step comparison. Steps 3–4—the CPA confirmation checkpoint and the subsequent PS re-entry—are the two steps skipped. Their omission is invisible to TSR (the payment still completes) and to HF1 (every expected handoff edge type appears in the 9-step trajectory, just not in the expected order with CPA intermediation). Only ASR, which compares the full ordered sequence, detects their absence.

Concretely, the expected trajectory has 10 transitions; the observed 9-step trajec-

Table 4.14: Expected (11-step) vs. observed (9-step) T3 trajectory for unambiguous payment inputs. Shaded rows mark the two steps skipped by models.

Step	Expected	Observed	Description
1	CPA	CPA	User submits payment request
2	PS	PS	PS receives request
3	CPA	*skipped*	**CPA confirmation checkpoint**
4	PS	*skipped*	**PS re-entry after user confirms**
5	PPW	PPW	3DS authentication initiated
6	PS	PS	PS receives 3DS result
7	CPA	CPA	CPA prompts user for OTP
8	PS	PS	PS receives OTP
9	PPW	PPW	Payment transaction completed
10	PS	PS	PS receives completion result
11	CPA	CPA	CPA delivers success message

tory has 8 transitions, of which all 8 appear in the expected set:

- **Transition Recall** $= 8/10 = 80\%$ — two expected transitions are absent
- **Transition Precision** $= 8/8 = 100\%$ — every transition taken is correct
- **ASR** $= F_1(80\%, 100\%) = 88.9\%$ per affected sample

The task succeeds (TSR = 100%), and all pairwise handoff edge types appear in both trajectories (HF1 = 100%), but ASR exposes the procedural shortcut. This failure mode is consistently characterized by *missing transitions* (low TR) rather than redundant ones (low TP). Payments succeed because the skipped steps serve orchestration and auditability purposes rather than functional transaction requirements. TSR and HF1 cannot detect these deviations because the final outcome is correct and all pairwise handoff edge types appear in the trajectory—just not in the expected sequence with CPA intermediation. This is precisely the kind of deviation that matters for payment compliance: the payment completes, but the audit trail is incomplete.

4.8.6 Deep Dive: Per-Sample Deviation Analysis

The conference version [78] reported aggregate ASR scores; here we decompose performance at the individual sample level, revealing the full extent of hidden

deviations and their triggers.

Hidden Deviation Rates

Table 4.15 presents per-sample T3 analysis for representative models. A *hidden deviation* is a sample where the task succeeds on TSR but the agent follows a deviant trajectory (ASR $< 100\%$). The aggregate gap depends on shortcut frequency: GPT models skip the checkpoint on $\sim$61 of 250 samples per repeat (T3 ASR $\approx$ 97.1–97.3%), `qwen2.5:32b` and `qwen3:8b` deviate on $<$1% of samples, and `mistral-small3.2:24b` enforces the checkpoint on every sample (T3 ASR = 100%).

Table 4.15: Per-sample T3 analysis for representative models: task failures vs. hidden deviations (samples that succeed on TSR but deviate on ASR). Hidden rate = hidden deviations / successful samples. `MSmall3.2:24b` is the only model with zero hidden deviations.

Model	T3 TSR	Failures	Hidden	Perfect	Hidden Rate
MSmall3.2:24b	100.0%	0	0	250	0.0%
Qwen3:32b	100.0%	0	59	191	23.6%
Qwen2.5:32b	100.0%	0	60	190	24.0% (mean)
Qwen3:8b	100.0%	0	59	191	23.6% (mean)
GPT-5.2	99.9%	$<$1	$\sim$62	$\sim$187	24.9%
GPT-4.1	99.8%	$<$1	$\sim$61	$\sim$188	24.5%
Qwen2.5:14b	99.8%	$<$1	$\sim$57	$\sim$192	22.9%
Llama3.1:70b	98.9%	$\sim$3	$\sim$60	$\sim$187	24.3%
Llama3.1:8b	93.4%	$\sim$17	$\sim$59	$\sim$174	25.3%

The hidden deviation rate is remarkably stable across models that exhibit the shortcut: rates cluster in the 22–26% range regardless of model size, model family, or overall TSR. Models with higher TSR simply have fewer outright failures but the *same* proportion of hidden shortcut deviations among their successes. The sole exception—`mistral-small3.2:24b`—demonstrates that the confirmation checkpoint *can* be consistently enforced, making it a useful reference point for architectural remediation.

Most critically, per-sample analysis confirms that every single hidden deviation follows the *identical* 9-step shortcut—there is exactly one deviant trajectory pattern

across all affected samples. This uniformity is itself a significant finding: the deviation is not stochastic or model-specific but reflects a systematic interaction between input phrasing and the multi-turn protocol structure.

Input Phrasing as the Primary Deviation Trigger

Per-sample cross-model analysis reveals that the shortcut is primarily triggered by *specific input phrasings* rather than by model-specific reasoning failures. Of the 250 T3 test inputs, a subset is shortcutted by nearly all models unanimously (e.g., "Can you Process my payment for my shopping cart", "I want to pay"), while the majority are never shortcutted by any model. A gradient of intermediate-ambiguity inputs accounts for the per-model variation.

Inputs that are universally shortcutted share a common linguistic signature: short, imperative phrasing with unambiguous payment intent (e.g., "Pay for my order", "Checkout now"). Inputs that are never shortcutted tend to use more deliberative or indirect formulations (e.g., "Complete my purchase please"). This input-level regularity explains the tight ASR band across models: all models (except `mistral-small3.2:24b`) respond to the same phrasing cues, and the variation in per-model shortcut counts reflects only minor differences in threshold sensitivity to intermediate-ambiguity inputs.

This finding has important implications for system design. The deviation is not a failure of model capability—it reflects a rational inference that direct imperatives constitute implicit confirmation. The architectural remedy is therefore not model replacement but enforcement of the CPA confirmation checkpoint through deterministic routing logic, regardless of the model's assessment of user intent.

4.8.7 Discussion: Regulatory and Architectural Implications

The T3 shortcut has direct regulatory significance. PCI-DSS [131] requires auditable workflows where each processing step is logged; bypassing confirmation checkpoints

creates gaps in the audit trail even when outcomes are correct. The T3 ASR gap of 0.01–2.9% across 7 of 8 top models—spanning both proprietary (GPT-4.1, GPT-5.2) and open-weight families of varying sizes (8B–70B parameters)—indicates that this is a structural property of the agentic architecture rather than a deficiency correctable through model scaling. The fact that `mistral-small3.2:24b` achieves perfect scores demonstrates that explicit enforcement of the CPA confirmation checkpoint is both necessary and achievable.

ASR thus serves dual roles. As an **evaluation metric**, it provides trajectory-level visibility that TSR and HF1 cannot, detecting procedural deviations that would otherwise remain invisible. As a **diagnostic signal**, it precisely localizes the source of deviation (T3 steps 3–4, triggered by imperative input phrasings), enabling targeted architectural remediation: enforcing mandatory CPA checkpoint returns before payment commitment through deterministic routing logic.

Combined with the structured error taxonomy from Section 4.4 and the ECR@1 metric from Section 4.5, ASR completes a comprehensive diagnostic toolkit for multi-agent LLM systems. Each metric addresses a different evaluation dimension: the error taxonomy classifies *failure types*, ECR@1 captures *operational reliability*, and ASR measures *workflow fidelity*—together enabling systematic diagnosis of failure modes impossible with traditional binary evaluation.

4.8.8 Ongoing Work: ConvPayMAS and Enterprise Evaluation Framework

The HMASP prototype (Section 4.7) and its ASR-evaluated A2A redesign (Sections 4.8.2–4.8.3) demonstrated feasibility and identified critical workflow deviations. Building on these findings, we are evolving the system into **ConvPayMAS** [24] (Conversational Payment Multi-Agent System)—an agentic payment framework implementing Google's Agent Payment Protocol (AP2) [50] with patent-pending security features for Mastercard's agentic commerce offerings. A demonstration of

the current system was accepted at AAMAS 2026 (25th International Conference on Autonomous Agents and Multiagent Systems); however, significant engineering work remains before production readiness, including comprehensive evaluation, scalability testing, and regulatory certification. This subsection describes the architecture and design direction of this ongoing effort.

Architecture

Figure 4.5 presents the ConvPayMAS ecosystem architecture. The system comprises three top-level agents communicating via standardized protocols:

1. **ConvPayMAS** (server-side): A secure multi-agent system providing a conversational interface for payment processing. Its Conversational Supervisor Agent (CSA) serves as the sole entry point for external agents, delegating to five specialized sub-agents: *List Card*, *Create Card*, *Validate Card*, *Charge Card*, and *Recommender*. Payment capabilities are exposed through an MCP tools server supporting sessions, wallet operations, and real-time authorization via Mastercard infrastructure.

2. **Shopper Agent** (user-side): Interprets user intent, navigates marketplaces, and orchestrates the shopping journey through sub-agents for discovery and address management.

3. **E-commerce Agent** (merchant-side): Handles catalog queries, cart operations, and initiates payment via ConvPayMAS.

Security and Protocol Design

The ConvPayMAS design introduces three advances beyond the A2A-compliant HMASP redesign:

Full AP2 Implementation with Three-Mandate Verification. We implement Google's Agent Payment Protocol (AP2) [50] using a cryptographic chain of three

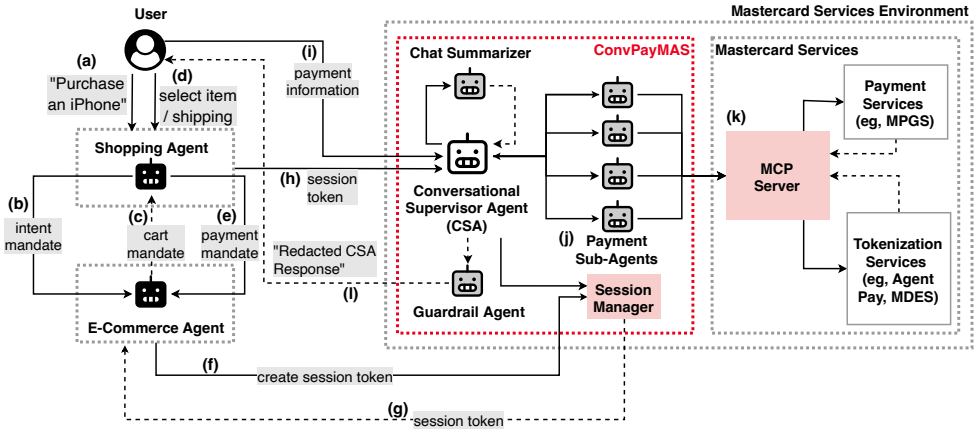


Figure 4.5: Overview of the agentic payment ecosystem with ConvPayMAS. The Shopper Agent (a) creates an Intent Mandate and communicates with the E-commerce Agent (b–d) via A2A streaming; upon item selection, a Payment Mandate is created (e) and validated against the cryptographic chain; a transaction session is established (f–g) and the user is handed off to ConvPayMAS (h–i), where the CSA routes to specialized sub-agents (j) that invoke MCP payment tools (k) for card operations and authorization; confirmation flows back through the conversational loop (l).

mandates that enables verifiable authorization and dispute resolution:

- *Intent Mandate*: Captures the user's purchase intent, created by the Shopper Agent at the start of the transaction.
- *Cart Mandate*: A merchant-signed mandate (HMAC-SHA256) binding specific items, prices, and shipping options to the transaction. Any price modification requires a new signed Cart Mandate.
- *Payment Mandate*: Created by the Shopper Agent, containing a JWT that references the Cart Mandate's authorization and a hashed representation of the payment contents, binding user consent to the exact cart state.

The three mandates form a hash chain that the E-commerce Agent validates end-to-end—verifying JWT signatures, expiry times, audience claims, and chain integrity—before creating a transaction session. This cryptographic chain directly addresses the T3 confirmation shortcut identified by ASR: rather than relying on LLM-mediated confirmation (which models can bypass), mandate verification is

enforced deterministically through cryptographic validation, making the confirmation checkpoint architecturally unbypassable.

Card Data Security. Card data (PAN, CVV, expiry) is encrypted client-side with AES-256-CBC using per-session keys issued by the MCP server. PANs are validated with Luhn checks and expiry dates verified before wallet storage. A Guardrail Agent redacts PCI/PII data from conversation logs before any external transmission, ensuring that sensitive cardholder data never leaves the ConvPayMAS perimeter—a critical requirement for PCI-DSS scope isolation.

Patent-Pending Security Features. Several security innovations in ConvPayMAS are the subject of pending patent applications. While details cannot be disclosed here, these features address key challenges in agentic commerce including secure delegation of payment authority across agent boundaries, tamper-resistant transaction state management, and cryptographic binding of conversational context to payment authorization. These patent-pending capabilities create competitive advantages for Mastercard's agentic commerce offerings and distinguish ConvPayMAS from academic prototypes.

Payment Flow

The end-to-end transaction proceeds through four phases: (1) *Intent & Discovery*—the user expresses purchase intent, the Shopper Agent creates an Intent Mandate and communicates with the E-commerce Agent, which returns a signed Cart Mandate; (2) *Selection & Payment Mandate*—the user selects items and shipping, and a Payment Mandate is created binding authorization to the cart via hash chain; (3) *Session Creation*—the E-commerce Agent validates all three mandates' cryptographic integrity and creates a transaction session; (4) *Payment Execution*—the user is handed off to ConvPayMAS with a session token, where the CSA routes to appropriate sub-agents for card operations, authorization, and confirmation.

Enterprise Agentic Evaluation and Optimization Framework

A key direction of ongoing work is an **Agentic Evaluation and Optimization Framework** being designed on top of the ConvPayMAS platform, leveraging the metrics and diagnostic toolkit developed in this dissertation. The goal is to operationalize the research contributions of this chapter into a production-grade engineering practice. The planned framework is structured around three pillars:

1. **Tool Call Reliability:** Measures whether individual agent tool invocations execute correctly, grounded in the structured error taxonomy (Section 4.4). This pillar captures initialization failures, parameter errors, execution exceptions, and result interpretation errors at the atomic operation level, providing the foundation for diagnosing why agents fail.

2. **Workflow Fidelity:** Evaluates whether agents follow prescribed multi-step procedures, operationalized through ASR (Section 4.8) and complemented by TSR and HF1. This pillar detects the trajectory-level deviations that tool-call metrics alone cannot capture—such as the T3 confirmation shortcut where individual tool calls succeed but the overall workflow violates compliance requirements.

3. **Cost Tracking:** Monitors token consumption, API costs, and latency across the agent ecosystem, enabling Pareto-optimal model selection. This pillar extends the cost-performance analysis methodology from the LLM-as-Judge framework (Section 4.5), where 175× cost variation was observed across evaluation models, to the full agentic pipeline.

The envisioned framework implements an **Evaluate** → **Diagnose** → **Optimize** continuous improvement loop: evaluation metrics from all three pillars would feed into automated diagnostic analysis that identifies specific failure patterns, which in turn drive targeted optimization of agent prompts, routing logic, and model selection. This closed-loop approach would systematically convert ASR-identified deviations (such as the input-phrasing-triggered T3 shortcut) into architectural fixes, and extend

naturally to new agentic use cases beyond payments. The combination of ASR-based trajectory evaluation, the structured error taxonomy, and ECR@1 operational reliability metrics provides the diagnostic foundation upon which this enterprise framework is being constructed. Realizing this vision requires substantial additional work, including automated deviation detection at scale, integration with production monitoring infrastructure, and validation across diverse agentic use cases beyond the payment domain.

4.9 Practical Deployment Guidelines

Based on our research across seven systems, we provide consolidated guidelines for enterprise agentic AI deployment:

4.9.1 Architecture Design

1. **Modular agent specialization:** Design agents with clearly defined, focused responsibilities
2. **Hierarchical coordination:** Employ multi-level hierarchies for complex workflows, enabling modular task execution
3. **Information determinism:** Store critical information in state variables, not LLM outputs, preventing hallucination of sensitive data
4. **Structured handoffs:** Implement clear protocols for inter-agent communication
5. **Human oversight:** Incorporate validation checkpoints for high-impact decisions

4.9.2 Model Selection

1. **Large models ($\geq$32B):** Provide production-ready reliability (ASR $>$93%) for autonomous workflows

2. **Mid-tier models (7B–14B):** Suitable for interactive workflows with human review, balancing accuracy and latency

3. **Small models (<4B):** Unsuitable for multi-step agentic pipelines despite acceptable performance on isolated tasks

4. **Consider edge deployment:** Reduces latency, improves privacy, and eliminates API costs—qwen2.5:32b matches proprietary performance while enabling local deployment

4.9.3 Reliability Engineering

1. **Measure ECR@1:** Always capture first-attempt success rates alongside accuracy

2. **Implement retry logic:** Budget 5–15% overhead for reliability failures

3. **Monitor operational metrics:** Track ECR@1, latency, and cost in production

4. **Use adjusted cost metrics:** Account for retry overhead in total cost of ownership

4.9.4 Evaluation Strategy

1. **Use trajectory-level metrics:** Adopt ASR or similar workflow-fidelity metrics to diagnose procedural deviations beyond binary success/failure—particularly critical in regulated domains where compliance requires auditable execution paths

2. **Distinguish orchestration from processing:** Tool-calling failures require different interventions than classification errors; trajectory shortcuts may require architectural enforcement rather than model replacement

3. **Hardware-aware benchmarking:** Test on target deployment hardware to establish realistic performance expectations; model-size invariance in structured architectures enables sub-billion-parameter edge deployment

4. **Multi-dimensional evaluation:** Assess accuracy, reliability, workflow fidelity, and cost together for production readiness

4.10 Summary

This chapter demonstrated the viability and effectiveness of agentic AI for automating complex enterprise workflows, addressing the second research question (RQ2) on designing and evaluating multi-agent LLM systems for reliable, scalable enterprise automation. Key contributions include:

1. **Multi-agent invoice reconciliation (Section 4.3):** Validated architecture achieving 99.9% success rate with edge-deployed open-source models, demonstrating that qwen2.5:32b outperforms proprietary alternatives while consuming only 523J per task on consumer hardware

2. **Diagnostic framework for tool invocation reliability (Section 4.4):** Novel structured error taxonomy providing systematic failure characterization for multi-agent systems, distinguishing tool initialization, parameter handling, execution, and result interpretation errors

3. **LLM-as-Judge framework with ECR@1 (Section 4.5):** Production-ready evaluation system introducing Evaluation Completion Rate to capture operational reliability, revealing 175× cost variation and demonstrating that GPT-4o Mini achieves best-in-class accuracy at 78× cost reduction versus premium models

4. **CMAS4G2 for acceptance test generation (Section 4.6):** Cooperative multi-agent system leveraging the LLM-as-Judge framework, demonstrating that open-weight models (GPT-OSS 20B: 87.6% success) approach proprietary performance (Claude Sonnet 4: 90.4%) while revealing critical reasoning configuration trade-offs

5. **Hierarchical Multi-Agent System for Payments (Section 4.7):** First LLM-based multi-agent system implementing end-to-end agentic payment workflows, achieving 97.0% task success rate with architectural patterns enabling modular payment processing

6. **Agentic Success Rate for Payment Systems (Section 4.8):** Trajectory-level workflow fidelity metric that compares observed and expected agent execution sequences at the transition (bigram) level, grounded in process-mining conformance checking. Across 18 LLMs and 90,000 task instances, ASR exposes systematic procedural deviations—7 of 8 top models skip CPA-mediated confirmation checkpoints during payment checkout (T3 ASR 97.1–99.99% despite near-perfect TSR and HF1). Notably, all six open-weight models in the top 8 outperform both proprietary GPT models on average ASR, and the shortcut is model-agnostic, input-driven, and requires architectural rather than model-level remediation.

7. **ConvPayMAS: Ongoing Work (Section 4.8.8):** Ongoing evolution of the research prototype toward a production agentic payment system with AP2 three-mandate verification (Intent, Cart, Payment mandates with cryptographic hash chain), AES-256-CBC card data encryption, PCI/PII guardrail agent, and patent-pending security features. Demonstrated at AAMAS 2026 as a work in progress, with significant remaining work in comprehensive evaluation, scalability testing, and an enterprise agentic evaluation and optimization framework being designed on top.

The ASR and ECR@1 metrics represent methodological contributions with broad applicability, extending the multi-dimensional evaluation philosophy established with RAP in Chapter 3. ASR, introduced in Section 4.8, compares observed and expected agent execution trajectories at the transition (bigram) level, enabling detection of procedural deviations invisible to outcome-based metrics—a capability shown to be essential for payment compliance auditing. Combined with the structured error

taxonomy from Section 4.4, these tools provide comprehensive diagnostics for multi-agent systems. ECR@1 captures operational reliability invisible to accuracy-only evaluation, revealing cost implications crucial for production deployment. Together, these metrics complement traditional accuracy measures and enable meaningful comparison across diverse agentic architectures.

A consistent theme across all seven systems is the viability of open-source models for enterprise deployment. In the ASR evaluation, all six open-weight models in the top 8 outperform both proprietary GPT models on average ASR, with qwen2.5:32b and mistral-small3.2:24b achieving perfect 100.0% while GPT-4.1 and GPT-5.2 reach only 98.4%. This extends to invoice reconciliation (qwen2.5:32b: 100% matching GPT-4.1) and payment processing (97.0% average TSR). The ASR analysis further demonstrates that architectural design choices—deterministic routing guards, structured task decomposition, and prompt engineering—are the dominant performance drivers: the T3 confirmation shortcut is model-agnostic and input-driven, affecting both open-weight and proprietary models alike. This finding parallels the Orca-2 results from Chapter 3 and reinforces the dissertation's broader thesis that properly configured open-source models can match or exceed proprietary alternatives for specific enterprise applications.

These findings provide a foundation for enterprises seeking to automate knowledge-intensive workflows using agentic AI while maintaining accuracy, privacy, and cost-effectiveness. The ASR analysis reveals that in structured multi-agent architectures, architectural design choices (deterministic routing guards, multi-turn protocol structure) dominate over model capacity—a finding that complements the reasoning configuration insights from CMAS4G2, where small models benefit dramatically from thinking mode while large models can degrade with excessive reasoning. Both observations directly inform Chapter 5, where we systematically characterize these trade-offs across 504 configurations spanning diverse sentiment analysis tasks.

Chapter 5

Enterprise Deployment and Application Strategies

5.1 Introduction and Background

Beyond Retrieval-Augmented Generation and agentic systems, enterprises require guidance on broader deployment decisions and specialized applications. This chapter addresses RQ3: *What deployment configurations, reasoning enhancement techniques, and application strategies maximize LLM effectiveness across diverse enterprise use cases?*

Building on the RAG optimization techniques established in Chapter 3 and the multi-agent reliability frameworks developed in Chapter 4, this chapter extends the investigation to edge deployment, reasoning enhancement, and cross-domain application patterns. The RAP metric's success in balancing quality with output characteristics motivates our development of additional domain-specific metrics, while the ASR framework's emphasis on operational reliability informs our evaluation of reasoning systems.

The research synthesizes findings from seven publications spanning edge deployment, reasoning enhancement, and cross-domain applications:

1. **LLMs at the Edge** [68]: Performance and efficiency evaluation with Ollama

on diverse hardware, evaluating models from 0.5B to 90B parameters across six hardware platforms including consumer-grade devices (IJCNN 2025)

2. **Deploying Reasoning LLMs** [75]: Cross-architecture evaluation of 17 open-weight reasoning LLMs (DeepSeek-R1, Qwen3, GPT-OSS) for sentiment analysis on consumer hardware (IEEE ICDMW 2025)

3. **Logical Reasoning with LLMs** [69]: Few-shot prompting and fine-tuning evaluation on Turtle Soup lateral thinking puzzles, introducing the Valid Classification Ratio metric (IEEE SSCI 2025)

4. **Explainable Sentiment Analysis with DeepSeek-R1** [67]: First comprehensive evaluation of DeepSeek-R1's explainable sentiment classification with few-shot learning (IEEE Intelligent Systems)

5. **Task Complexity Matters** [71]: First large-scale study of reasoning effectiveness across 504 configurations, revealing task-complexity-dependent performance patterns and introducing Pareto frontier analysis for model selection (PAKDD 2026, Submitted)

6. **Chinese-English Translation Optimization** [70]: Comprehensive LLM evaluation across zero-shot, few-shot, and fine-tuned paradigms using the MAC parallel corpus (IEEE SSCI 2025)

7. **LLM Sentiment for Soccer Betting** [146]: Novel hybrid framework integrating LLM-derived sentiment features into machine learning models for sports prediction (IEEE ICDMW 2025)

This chapter is organized around three themes: (1) edge deployment strategies for privacy-preserving and cost-effective applications, (2) reasoning enhancement techniques including few-shot prompting and fine-tuning, and (3) cross-domain applications demonstrating the generalizability of LLM optimization techniques.

5.2 Edge Deployment Strategies

Enterprise adoption of LLMs faces critical constraints around privacy, latency, and infrastructure costs. Edge deployment—executing models locally on consumer-grade hardware—addresses these challenges while enabling new application paradigms. The democratization of LLMs through edge deployment holds significant potential for social good: by enabling local execution, edge LLMs enhance privacy in sensitive domains like healthcare and education, reduce the environmental footprint of AI deployment, and expand access to AI in regions with limited cloud infrastructure. These considerations connect directly to the multi-agent systems explored in Chapter 4, where edge deployment enables privacy-preserving automation of sensitive business processes like invoice reconciliation.

5.2.1 LLMs at the Edge with Ollama

Motivation and Approach

Energy efficiency is a critical consideration in deploying large language models due to their significant environmental footprint. We leverage Ollama, an open-source framework that facilitates efficient local deployment of LLMs, to evaluate performance and efficiency across diverse hardware platforms. Ollama emerges as a lightweight framework designed to simplify local LLM deployment and execution, offering an intuitive API for model management while enhancing transparency, privacy, and performance. Its cross-platform compatibility across macOS, Linux, and Windows allows organizations to leverage existing hardware for AI applications efficiently.

Experimental Setup

We evaluate open-source LLMs from the Qwen2.5 and Llama3 families (0.5B–90B parameters) across six hardware platforms spanning high-end systems, consumer

laptops, and edge devices:

- **High-end Desktop GPU:** NVIDIA RTX A6000 Ada Generation (48GB GDDR6 ECC, 18,176 CUDA cores, 568 Tensor cores)
- **Consumer Desktop GPU:** NVIDIA GeForce RTX 4090 (24GB GDDR6X, 16,384 CUDA cores, 512 Tensor cores)
- **Laptop GPU:** NVIDIA GeForce RTX 4090 Laptop (16GB GDDR6, 9,728 CUDA cores, 304 Tensor cores)
- **Apple Silicon (High-end):** M3 Max MacBook Pro (96GB unified memory)
- **Apple Silicon (Mid-range):** M4 Max MacBook Pro (48GB unified memory)
- **Edge Device:** NVIDIA Jetson AGX Orin Developer Kit (64GB memory, 2,048 CUDA cores, 64 Tensor cores, up to 275 TOPS)

The evaluation utilizes the Global Maritime Risk Incident Dataset (GMRID) v3, chosen for its domain specificity to the maritime industry where LLM applications are relatively unexplored. This dataset builds on the maritime risk identification work described in Chapter 4, providing a consistent evaluation domain across multiple studies. Table 5.1 details the evaluated models with their specifications under Ollama's Q4_K_M quantization.

Evaluation Metrics

We employ comprehensive metrics capturing both performance and efficiency. These metrics complement the RAP metric introduced in Chapter 3 by focusing on deployment-specific considerations rather than output quality alone:

1. **Energy per Inference:** Energy required to process a single input sequence, measured in Joules:

$$\text{Energy per Inference (J)} = \text{Time (s)} \times P_{\text{CPU+GPU}}(\text{W}) \tag{5.1}$$

Table 5.1: Ollama Models Evaluated for Edge Deployment

Model	**Size**	**Precision**	**Category**
qwen2.5:0.5b	397 MB	Q4_K_M	Small
llama3.2:1b	1.3 GB	Q4_K_M	Small
qwen2.5:1.5b	986 MB	Q4_K_M	Small
llama3.2:3b	2.0 GB	Q4_K_M	Small
qwen2.5:3b	1.9 GB	Q4_K_M	Small
qwen2.5:7b	4.7 GB	Q4_K_M	Medium
llama3.1:8b	4.9 GB	Q4_K_M	Medium
llama3.2-vision:11b	7.9 GB	Q4_K_M	Medium
qwen2.5:14b	9.0 GB	Q4_K_M	Medium
qwen2.5:32b	19 GB	Q4_K_M	Medium
llama3.1:70b	42 GB	Q4_K_M	Large
llama3.3:70b	42 GB	Q4_K_M	Large
qwen2.5:72b	47 GB	Q4_K_M	Large
llama3.2-vision:90b	54 GB	Q4_K_M	Large

where $P_{\text{CPU+GPU}}$ is the average power consumption during inference, excluding idle power.

2. **Performance-to-Power Ratio:** Performance achieved per unit of energy consumed:

$$\text{Performance-to-Power Ratio} = \frac{\text{F1 Score (\%)}}{\text{Energy Consumed (J)}} \tag{5.2}$$

3. **Throughput:** Tokens processed per second (t/s), enabling comparison of inference speed across platforms.

Results

Figure 5.1 presents performance and energy efficiency metrics on RTX A6000, revealing distinct trade-offs between model size, accuracy, and energy consumption.

Key findings from our comprehensive edge deployment evaluation:

1. **Quantization effectiveness:** Ollama's Q4_K_M quantization outperforms bitsandbytes 4-bit implementation, with llama3.1:8b achieving a mean F1 score of 92.87% ($\pm$0.46%) compared to its baseline of 89.50% (3.37 percentage point

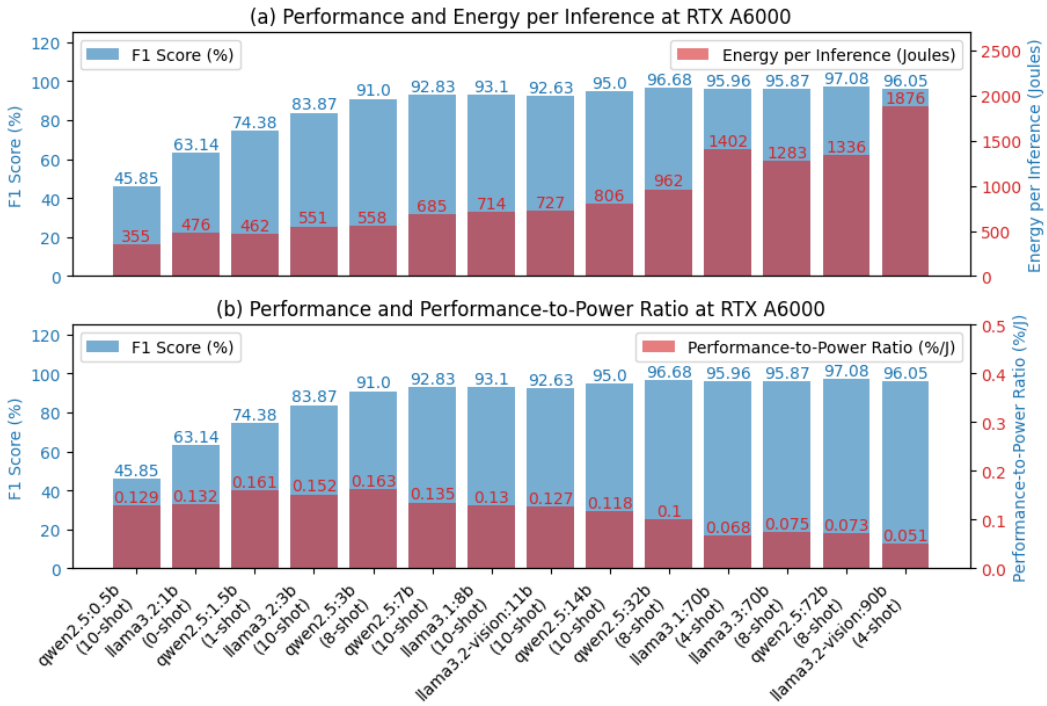


Figure 5.1: Performance and Energy Efficiency Metrics on RTX A6000. (a) F1 Score (%) vs. Energy per Inference (Joules). (b) F1 Score (%) vs. Performance-to-Power Ratio (%/J). Large models achieve high F1 scores but consume significantly more energy, while small models offer superior energy efficiency with acceptable performance trade-offs.

improvement), and llama3.1:70b improving to 95.73% (±0.23%) from 95.51% (0.22 percentage point improvement). This finding reinforces the open-source viability theme established in Chapter 3, demonstrating that optimization techniques can unlock competitive performance from smaller models.

2. **WSL performance gains:** Windows Subsystem for Linux (WSL) delivers substantial throughput improvements over native Windows. On RTX A6000, qwen2.5:0.5b processes 16,933 tokens/s on WSL compared to 816 tokens/s on Windows—a 21× speedup. This gap narrows for larger models, with differences as low as 15% at 90B parameters.

3. **Optimal model sizing:** Medium-sized models (7B–32B) achieve the best balance between performance and efficiency:

   - qwen2.5:32b attains 96.75% (±0.11%) F1 score with practical inference

speeds (mean 710 t/s across platforms)

- qwen2.5:7b achieves 92.64% (±0.14%) F1 with mean throughput of 2,873 t/s
- llama3.1:8b reaches 92.87% (±0.46%) F1 with mean throughput of 3,017 t/s

4. **Cross-platform stability:** Larger models exhibit increased cross-platform consistency, with F1 score standard deviations narrowing from 0.26–0.90% (small models) to 0.07–0.23% (large models), indicating more robust quantization at scale.

5. **Energy trade-offs:** Large models such as qwen2.5:72b and llama3.2-vision:90b achieve high F1 scores (97.08% and 96.05%) but consume 1,283–1,876 Joules per inference. Small models like llama3.2-3b and qwen2.5:3b consume only 551–558 Joules while achieving 83.87% and 91.00% F1 respectively. Small models like qwen2.5:1.5b and qwen2.5:3b achieve the highest performance-to-power ratios (0.161 and 0.163 %/J), ideal for energy-efficient deployments.

Tables 5.2, 5.3, and 5.4 summarize performance across hardware platforms by model category.

Table 5.2: Edge Deployment Performance: Small Models (0.5B–3B)

Model	**Mean F1 (%)**	**Std (%)**	**Mean T-put (t/s)**	**Mean Time (s)**
qwen2.5:0.5b	46.66	0.77	7,141	0.942
llama3.2:1b	63.17	0.33	2,033	1.357
qwen2.5:1.5b	74.21	0.90	3,479	1.038
llama3.2:3b	83.97	0.32	4,400	1.328
qwen2.5:3b	90.89	0.26	3,833	1.346

Table 5.3: Edge Deployment Performance: Medium Models (7B–32B)

Model	Mean F1 (%)	Std (%)	Mean T-put (t/s)	Mean Time (s)
qwen2.5:7b	92.64	0.14	2,873	1.840
llama3.1:8b	92.87	0.46	3,017	1.785
llama3.2-vision:11b	92.89	0.33	2,950	1.746
qwen2.5:14b	95.00	0.14	1,629	3.004
qwen2.5:32b	96.75	0.11	710	7.781

Table 5.4: Edge Deployment Performance: Large Models (70B–90B)

Model	Mean F1 (%)	Std (%)	Mean T-put (t/s)	Mean Time (s)
llama3.1:70b	95.73	0.23	287	8.762
llama3.3:70b	95.84	0.07	368	16.513
qwen2.5:72b	96.90	0.20	263	10.660
llama3.2-vision:90b	96.04	0.07	90	17.739

5.2.2 Deploying Reasoning LLMs on Consumer Hardware

Motivation

Deploying reasoning-enabled LLMs for sentiment analysis requires balancing accuracy, explainability, and efficiency under consumer hardware constraints. We present a cross-architecture evaluation of three reasoning model families—DeepSeek-R1, Qwen3, and GPT-OSS—spanning 17 open-weight variants. This addresses a critical gap: most existing evaluations target datacenter or cloud settings, leaving practitioners without evidence-based guidelines for consumer hardware deployment.

Reasoning Intensity Metric

We introduce Reasoning Intensity (RI) to capture trade-offs between explanation depth and computational cost:

$$RI = \frac{\text{Mean Reasoning Tokens (MRT)}}{\text{Mean Reasoning Tokens (MRT)} + \text{Mean Output Tokens (MOT)}} \quad (5.3)$$

For DeepSeek-R1 models, both the *reasoning content* (step-by-step thought process) and the final JSON output are included in the token count. The reasoning content refers to the complete trace of the model's thought process generated during

inference, including intermediate evaluations and decision-making steps. This metric enables systematic comparison of reasoning strategies across architectures and complements the evaluation framework established in earlier chapters by capturing a dimension—reasoning overhead—not addressed by traditional accuracy metrics.

Experimental Setup

We evaluate models on fine-grained Amazon Reviews sentiment classification (5 classes: Strongly Negative, Negative, Neutral, Positive, Strongly Positive) using few-shot prompting. The primary benchmark comprises 1,838 reviews with human annotation, split 70%/30% for training/testing. To assess temporal robustness, we introduce Testset2: 574 manually annotated contemporary Amazon reviews collected in mid-2025.

Model Families:

- **DeepSeek-R1:** 8B, 14B, 32B, and 70B variants
- **Qwen3:** 0.6B, 1.7B, 4B, 8B, 14B, 30B, and 32B variants
- **GPT-OSS:** 20B and 120B, each with low/medium/high reasoning configurations

All models were executed via Ollama v0.11.4 using 4-bit quantization (MXFP4 for GPT-OSS; Q4_K_M for others), with a context length of 8,192 tokens.

Results

Table 5.5 presents deployment performance across model families on NVIDIA H100 GPU (96GB VRAM).

Key findings reveal three distinct reasoning regimes:

1. **Best accuracy:** GPT-OSS achieves the highest F1 (86.2% with gpt-oss:120b medium configuration) but requires 81 GB VRAM, far exceeding consumer GPU limits.

Table 5.5: Cross-Architecture Reasoning Performance (5-Level Sentiment) on H100 GPU

Model	**F1 (%)**	**Acc (%)**	**MRT**	**MOT**	**RI**	**VRAM**	**Time (s)**
DeepSeek-R1 Family							
deepseek-r1:8b	71.0	68.8	338	49	0.873	13 GB	3.71
deepseek-r1:14b	77.4	75.5	248	54	0.819	18 GB	4.88
deepseek-r1:32b	**81.7**	**79.9**	241	34	0.873	31 GB	7.05
deepseek-r1:70b	74.2	71.3	237	40	0.855	58 GB	12.08
Qwen3 Family							
qwen3:0.6b	65.2	64.8	236	72	0.765	6.1 GB	1.34
qwen3:1.7b	75.7	76.8	217	57	0.790	6.8 GB	1.56
qwen3:4b	84.3	83.7	284	60	0.825	11 GB	3.93
qwen3:8b	**84.8**	**84.9**	249	59	0.807	13 GB	8.35
qwen3:14b	84.6	83.7	245	61	0.799	19 GB	10.20
qwen3:30b	84.3	84.8	1234	20	0.984	26 GB	12.35
qwen3:32b	82.3	81.3	279	60	0.821	40 GB	15.71
GPT-OSS Family							
gpt-oss:20b (low)	66.9	64.2	13	18	0.417	23 GB	0.64
gpt-oss:20b (medium)	81.7	81.3	165	28	0.851	23 GB	2.30
gpt-oss:20b (high)	84.6	83.8	744	30	0.961	23 GB	8.60
gpt-oss:120b (low)	85.5	85.3	22	40	0.361	81 GB	1.58
gpt-oss:120b (medium)	**86.2**	**86.4**	115	42	0.728	81 GB	3.15
gpt-oss:120b (high)	83.4	82.8	679	43	0.940	81 GB	11.84

2. **Practical balance:** Qwen3 delivers the most practical balance for consumer deployment. Within the 16GB VRAM constraint of an RTX 4090 Laptop, three models are viable:

   - qwen3:8b: Optimal accuracy (84.8% F1), 13GB VRAM, 8.35s inference
   - qwen3:4b: Balanced performance (84.3% F1), 11GB VRAM, 3.93s inference
   - qwen3:1.7b: Resource-constrained option (75.7% F1), 6.8GB VRAM, 1.56s inference

3. **Reasoning intensity patterns:** The results reveal three operational regimes:

   - *Minimal Reasoning* (RI < 0.5): GPT-OSS low configurations prioritize throughput over explainability
   - *Balanced Reasoning* ($0.5 \leq$ RI ≤ 0.85): Most top-performing models operate here, including gpt-oss:120b (medium) achieving 86.2% F1 with RI = 0.728
   - *Intensive Reasoning* (RI > 0.85): DeepSeek-R1 models allocate 85–98% of tokens to reasoning, enhancing explainability with modest accuracy gains

4. **Architecture over scale:** Larger models do not always outperform smaller counterparts. deepseek-r1:32b (81.7% F1) substantially outperforms deepseek-r1:70b (74.2% F1), demonstrating that architectural efficiency matters more than parameter count. This finding echoes the Orca-2-13b results from Chapter 3, reinforcing the dissertation's broader finding that model optimization often matters more than scale.

Temporal Robustness Evaluation

To assess model robustness under temporal drift, we evaluate on Testset2 comprising 574 manually annotated contemporary reviews from mid-2025.

Table 5.6: Cross-Platform Temporal Robustness: Original vs. Contemporary Reviews

Model	Hardware	Original F1 (%)	Testset2 F1 (%)	Δ F1	RI Stability
qwen3:1.7b	H100	75.7	57.9	-17.8	$\pm$0.012
qwen3:1.7b	RTX 4090	72.6	58.1	-14.5	$\pm$0.010
qwen3:4b	H100	84.3	67.8	-16.5	$\pm$0.011
qwen3:4b	RTX 4090	81.7	65.7	-16.0	$\pm$0.001
qwen3:8b	H100	84.8	72.3	-12.5	$\pm$0.021
qwen3:8b	RTX 4090	82.1	73.3	-8.8	$\pm$0.017

Results reveal consistent 10–17 percentage point F1 degradation across models on contemporary data, though reasoning intensity remains remarkably stable ($\pm$0.02) across temporal domains. Notably, qwen3:8b exhibits superior temporal resilience, maintaining the smallest relative degradation among evaluated models. This suggests that while models' reasoning processes are robust, their learned sentiment patterns require periodic updating to account for evolving language use and product categories.

5.3 Reasoning Enhancement Techniques

5.3.1 Logical Reasoning via Lateral Thinking Puzzles

The Turtle Soup Benchmark

To assess logical reasoning capabilities, we utilize the Turtle Soup Question Answering (TSQA) dataset from the "Logical Reasoning with LLM" competition. The Turtle Soup game, popularized by the book *Challenging Lateral Thinking Puzzles*, presents mystery puzzles requiring lateral thinking—players must deduce hidden rationales through yes/no questions. The game's name originates from a classic problem: a man orders turtle soup at a restaurant and subsequently commits suicide; players must deduce the rationale through constrained questioning.

The dataset comprises:

- 25,000 labeled training questions

- 3,000 labeled validation questions
- Chinese language content (adding complexity for primarily English-trained models)

Model responses are constrained to five categories: (Yes), (No), (Unimportant), (Wrong Question), and (Correct Answer).

Methodology

Model Selection: We evaluate diverse state-of-the-art LLMs including GPT-4o, o1-preview, o1-mini, Qwen2.5 (3B–72B), Llama3.1 (8B–70B), InternLM2.5 (7B, 7B-1M, 20B), and Mistral-7B. For open-source models lacking native Chinese support, we utilized their fine-tuned Chinese versions.

Few-shot Prompting: We employ a concise system prompt ("You are an expert in logical reasoning") with representative examples covering all answer categories. Few-shot configurations range from 0 to 50 shots.

Parameter-Efficient Fine-Tuning: We apply LoRA [65] (rank 64, alpha 128) and QLoRA [33] (4-bit quantization) for efficient adaptation on consumer-grade hardware. This approach aligns with the parameter-efficient techniques that enabled the multi-agent systems in Chapter 4 to run on edge hardware.

Valid Classification Ratio (VCR)

We introduce the Valid Classification Ratio (VCR) as a novel metric for assessing output reliability:

$$VCR = \frac{\text{Number of valid classifications}}{\text{Total number of outputs}} \tag{5.4}$$

VCR measures the proportion of responses that map to valid answer categories, capturing model reliability beyond accuracy. During evaluation, we observed that some models produce outputs containing extraneous text or failing to match predefined labels, necessitating post-processing before F1 calculation. This metric

complements the Agentic Success Rate (ASR) introduced in Chapter 4, but focuses specifically on output validity rather than tool-calling reliability. While ASR measures whether agents successfully execute multi-step workflows, VCR measures whether models produce outputs conforming to expected formats—a prerequisite for downstream processing in production systems.

Results

Table 5.7 presents logical reasoning performance across few-shot and fine-tuned configurations.

Table 5.7: Logical Reasoning Performance on TSQA Benchmark

Model	**Method**	**Shots**	**Acc (%)**	**F1 (%)**	**VCR (%)**
GPT-4o	Few-shot	10	79.12	80.36	99.2
o1-preview	Few-shot	50	75.89	77.08	98.7
o1-mini	Few-shot	50	74.56	75.89	98.4
Qwen2.5-72B	Few-shot	10	79.85	**80.88**	98.5
Qwen2.5-7B	Few-shot	50	74.23	75.45	97.8
Qwen2.5-3B	Few-shot	50	71.34	72.67	96.9
Llama3.1-70B	Few-shot	20	76.89	78.12	98.1
Llama3.1-8B	Few-shot	50	73.45	74.78	97.5
InternLM2.5-7B-1M	Few-shot	50	75.67	77.01	97.9
InternLM2.5-7B	Few-shot	50	74.12	75.34	97.6
Mistral-7B	Few-shot	50	72.89	74.12	97.2
Llama3.1-70B	Fine-tuned	–	79.65	**80.77**	99.1
Qwen2-72B	Fine-tuned	–	79.21	80.33	99.0
InternLM2.5-7B-1M	Fine-tuned	–	78.92	80.28	98.8
InternLM2.5-20B	Fine-tuned	–	78.89	80.28	98.9
InternLM2.5-7B	Fine-tuned	–	78.45	79.89	98.7
Qwen2.5-7B	Fine-tuned	–	77.89	79.23	98.6

Key findings:

1. **Open-source competitiveness:** Qwen2.5-72B achieved the highest few-shot F1 score (80.88%), surpassing GPT-4o's 80.36% with only 10-shot prompting, demonstrating that open-source models can match or exceed proprietary alternatives on complex reasoning tasks. This reinforces the open-source viability

findings from Chapter 3 where Orca-2-13b achieved comparable performance to GPT-4 in RAG applications.

2. **Unexpected o1-preview performance:** The o1-preview model, despite its purported superior reasoning capabilities, exhibited lower F1 scores (77.08%) compared to GPT-4o even with 50-shot prompting. This warrants further investigation into whether o1-preview's reasoning strengths are better suited to other task types.

3. **Fine-tuning superiority:** Fine-tuning generally yields superior results, with Llama3.1-70B achieving 80.77% F1 after fine-tuning—surpassing all few-shot configurations except Qwen2.5-72B.

4. **Extended context benefit:** InternLM2.5-7B-1M, with its 1-million-token context window, demonstrated significant improvement from few-shot (77.01%) to fine-tuned (80.28%), ranking third overall among fine-tuned models despite its smaller parameter count.

5. **VCR improvements:** Fine-tuning substantially improves VCR, with all fine-tuned models exceeding 98.6% valid classification rates compared to 96.9–99.2% for few-shot configurations. Notably, Mistral-7B showed extremely low VCR values (ranging from 1.17% to 14.20%) in few-shot settings, but fine-tuning improved VCR to 100% within just 0.2 epochs, demonstrating the dramatic reliability improvements achievable through targeted optimization.

Fine-Tuning Analysis

We observed that training beyond 1 epoch often leads to overfitting. To address this, we implemented early stopping with 0.2 epoch granularity and selected optimal checkpoints based on validation performance. For instance, Qwen2-72B achieved peak F1 (80.33%) at 0.8 epochs, while smaller models typically required 0.8–1.0 epochs. Larger models consistently peaked earlier (2–3 epochs) than smaller ones (4–5 epochs), suggesting that larger models learn task-specific patterns more efficiently

but are also more prone to overfitting.

5.3.2 Explainable Sentiment Analysis with DeepSeek-R1

Motivation

Large language models have transformed sentiment analysis, yet balancing accuracy, efficiency, and explainability remains a critical challenge. DeepSeek-R1, an open-source reasoning model trained via reinforcement learning, introduces a novel approach by generating step-by-step reasoning processes during inference. This distinguishes it from conventional LLMs that prioritize direct responses, offering native interpretability without external explanation mechanisms.

Experimental Setup

We evaluate DeepSeek-R1 (full 671B model and distilled variants: 8B, 14B, 32B, 70B) against GPT-4o and GPT-4o-mini on three affective computing tasks:

- **Amazon Reviews:** 5-class sentiment classification (Strongly Negative to Strongly Positive), 1,838 human-annotated reviews
- **IMDB Movie Reviews:** Binary classification (Negative/Positive)
- **GoEmotions:** 27-class emotion recognition for nuanced sentiment evaluation

We employ few-shot prompting (0-shot to 50-shot configurations) to assess in-context learning effectiveness.

Results

Table 5.8 presents sentiment analysis performance across tasks.

Key findings:

1. **Superior few-shot efficiency:** DeepSeek-R1 achieves 91.39% F1 on 5-class sentiment with just 30 shots, representing an approximately eightfold improvement

Table 5.8: DeepSeek-R1 Performance on Sentiment Classification Tasks

Model	Task	Acc (%)	F1 (%)	Shots	T-put (t/s)
Amazon Reviews (5-class)					
DeepSeek-R1 (671B)	5-class	91.29	**91.39**	30	334
GPT-4o	5-class	86.57	87.99	40	1,220
GPT-4o-mini	5-class	81.31	83.02	50	2,124
deepseek-r1:32b	5-class	79.85	80.89	50	254
deepseek-r1:14b	5-class	78.23	79.06	30	414
deepseek-r1:8b	5-class	76.77	78.31	40	520
IMDB Binary Sentiment					
DeepSeek-R1 (671B)	Binary	99.12	**99.31**	5	334
GPT-4o	Binary	97.23	97.47	20	1,220
deepseek-r1:70b	Binary	97.89	98.04	20	52
deepseek-r1:32b	Binary	95.73	95.75	10	254
GoEmotions (27-class)					
DeepSeek-R1 (671B)	27-class	45.33	46.02	10	48
GPT-4o	27-class	45.17	45.99	10	424
deepseek-r1:70b	27-class	44.00	44.57	10	52

in few-shot efficiency over GPT-4o (which requires 40 shots for 87.99% F1). This demonstrates that reasoning-native architectures can achieve strong performance with fewer examples.

2. **Binary task excellence:** On IMDB binary classification, DeepSeek-R1 achieves 99.31% accuracy with only 5 shots, demonstrating exceptional performance on simpler tasks with minimal prompting.

3. **Architecture-specific distillation effects:** The 32B Qwen2.5-based distilled model outperforms the 70B Llama-based variant by 6.69 percentage points on Amazon Reviews (80.89% vs. 74.20%), suggesting that distillation quality depends more on base architecture compatibility than parameter count.

4. **Throughput trade-off:** DeepSeek-R1's reasoning process reduces throughput (334 t/s at peak) compared to GPT-4o (1,220 t/s), but offers superior explainability via transparent reasoning traces. This trade-off is acceptable for applications requiring auditability.

Explainability Analysis

DeepSeek-R1's reasoning traces demonstrate sophisticated analysis patterns that enhance transparency:

- **Resolution of mixed experiences:** Both DeepSeek-R1 and GPT-4o correctly identify neutral sentiment in mixed reviews, but follow distinct reasoning paths. GPT-4o balances product failure against positive customer service in a single-layer explanation, while DeepSeek-R1 traces temporal progression from negative experience to resolution, providing dual-layer explanation with both a brief prompt-compliant summary and extensive step-by-step reasoning.
- **Emotional vs. functional analysis:** DeepSeek-R1 assigns greater weight to emotional impact (e.g., feeling "duped"), leading to different classifications than GPT-4o's focus on functional benefits. This heightened sensitivity to emotional language enables more nuanced sentiment detection.
- **Treatment of indirect experiences:** GPT-4o interprets widespread user complaints as indicative of negative sentiment even without direct personal experience, while DeepSeek-R1 classifies such reviews as "Neutral" since they focus on reporting issues rather than expressing direct dissatisfaction.

The key stylistic difference is that DeepSeek-R1 generates an *intrinsic reasoning trace*—not prompted, but inherent to its architecture—representing a unique form of built-in explainability that emerges naturally from reinforcement learning training.

5.3.3 Task Complexity and Reasoning Effectiveness

Motivation

Building on the initial investigation of DeepSeek-R1 in Section 5.3.2, we significantly extend the scope to address a central question: *Do reasoning capabilities justify their computational overhead for sentiment tasks of varying complexity?* Where

Section 5.3.2 established DeepSeek-R1's superior few-shot efficiency (approximately 8× over GPT-4o), this section investigates whether such advantages generalize across task complexities and model architectures.

While reasoning-enhanced LLMs have demonstrated strong performance on mathematical and logical tasks, the effectiveness of reasoning mechanisms for foundational NLP applications such as sentiment analysis remains largely untested. Recent work in financial sentiment analysis provides early evidence of a potential mismatch, demonstrating that chain-of-thought reasoning leads to "overthinking" and degraded performance compared to intuitive classification [108]. This suggests a task-complexity hypothesis: simple classification tasks with clear categorical boundaries may require different cognitive strategies than complex tasks with nuanced distinctions.

Experimental Setup

We conduct the first large-scale, cross-architectural comparison of reasoning versus non-reasoning modes across sentiment analysis tasks of varying complexity. The evaluation spans 504 unique configurations (7 model families × 3 datasets × 7 shot levels × thinking/non-thinking variants):

Model Families:

- **DeepSeek-R1 Series:** Full 671B model and distilled variants (8B, 14B, 32B, 70B), compared against base models DeepSeek-V3, LLaMA (3.1-8B, 3.3-70B), and Qwen2.5 (14B, 32B)
- **Qwen3:** Dense models (4B, 8B, 14B, 32B) supporting both thinking (T) and non-thinking (N) modes with adaptive "thinking budgets"
- **Granite3.3:** Lightweight models (2B, 8B) with conditional reasoning in T and N modes
- **Magistral:** 24B model with RL-based reasoning in T and N modes

Datasets Spanning Complexity Levels:

- **IMDB Movie Reviews:** Binary sentiment (positive/negative), representing simple classification
- **Amazon Reviews:** Five-class sentiment (strongly negative to strongly positive), representing moderate complexity
- **GoEmotions:** 27 emotion categories, representing high complexity

Models were evaluated under 0–5–10–20–30–40–50-shot settings, with stratified sampling for few-shot exemplars.

Results: Distilled vs. Base Model Performance

Table 5.9 presents performance comparisons between reasoning-enhanced distilled models and their base counterparts.

Table 5.9: Reasoning-Focused Distillation vs. Base Model Performance. Diff. indicates performance difference (Distilled − Base). Failure Rate (FR) shows percentage of comparisons where distilled models underperform or match base models.

Dataset	Model Pair	Base 0S	Dist. 0S	Diff. 0S	Diff. BS	FR (%)
IMDB	DSV3 vs DSR1	97.9	78.0	-19.9	-0.0	100
	Llama3.3-70B vs DSR1-70B	98.0	97.1	-0.9	-1.3	
	Qwen2.5-32B vs DSR1-32B	97.8	96.0	-1.8	-1.3	
Amazon	Llama3.3-70B vs DSR1-70B	82.5	64.1	-18.4	-12.7	80
	Qwen2.5-14B vs DSR1-14B	74.7	61.9	-12.8	-8.0	
	Qwen2.5-32B vs DSR1-32B	63.8	69.0	+5.2	-3.9	
GoEmotions	DSV3 vs DSR1	39.1	42.8	+3.7	+3.8	50
	Llama3.3-70B vs DSR1-70B	38.6	34.0	-4.6	+4.8	
	Qwen2.5-32B vs DSR1-32B	34.0	34.7	+0.7	+3.2	

Key findings reveal systematic task-complexity patterns:

1. **Zero-shot degradation:** Distilled models underperform base models in zero-shot settings, with the largest drops on simple tasks: -19.9 pp (DSR1 vs. DSV3 on IMDB), -18.4 pp (DSR1-70B vs. Llama3.3-70B on Amazon). Mean degradation by dataset: -5.9 pp (IMDB), -7.6 pp (Amazon), -2.0 pp (GoEmotions).

2. **Few-shot recovery:** Gaps narrow with examples—IMDB mean gap improves from -5.9 to -1.3 pp; Amazon from -7.6 to -5.2 pp. GoEmotions shows notable reversals where distilled models outperform bases (DSR1-70B: +4.8 pp, DSR1-32B: +3.2 pp).

3. **Task-dependent failure rates:** IMDB shows 100% failure rate (all 10 comparisons favor base models), Amazon shows 80%, while GoEmotions shows 50%—decreasing failure rates with increasing task complexity validate the hypothesis that reasoning becomes beneficial as classification granularity increases.

Results: Thinking vs. Non-Thinking Mode Analysis

Table 5.10 compares thinking and non-thinking modes across architectures with runtime-activated reasoning.

Table 5.10: Thinking vs. Non-Thinking Mode Performance. Diff. = F1_Thinking − F1_Non-thinking. Speed Ratio (SR) = Thinking latency / Non-thinking latency.

Dataset	Model	Non-think.	Thinking	Diff.	SR	FR (%)
IMDB	Qwen3-4B	98.2	98.4	+0.2	5.5×	36
	Qwen3-14B	98.4	98.9	+0.5	4.3×	
	Magistral-24B	97.4	96.9	-0.5	9.7×	
	Granite3.3-8B	98.3	96.0	-2.2	1.0×	
Amazon	Qwen3-8B	84.5	84.8	+0.3	5.4×	57
	Magistral-24B	81.6	80.7	-0.9	14.4×	
	Qwen3-32B	89.0	84.8	-6.8	11.5×	
	Granite3.3-2B	74.1	63.9	-10.2	2.1×	
GoEmotions	Magistral-24B	35.0	43.7	+8.8	54.4×	14
	Qwen3-14B	40.3	47.0	+6.6	8.4×	
	Qwen3-32B	39.8	45.9	+6.2	7.4×	
	Qwen3-8B	41.0	44.0	+3.1	8.3×	

Key findings across 42 zero-shot comparisons:

1. **Task-complexity dependence:** Thinking mode helps GoEmotions (86% positive), shows mixed results on Amazon (43% positive), and balanced effects on IMDB (64% positive). Failure rates by dataset: Amazon 57%, IMDB 36%, GoEmotions 14%—substantially lower failure rates for complex emotion recognition

confirm reasoning mechanisms are most effective when task complexity justifies computational cost.

2. **Few-shot convergence patterns:** Thinking advantages often diminish with examples. Magistral exhibits dramatic shifts: Amazon from +12.2 pp (zero-shot) to -0.9 pp (best-shot); IMDB from +2.1 pp to -0.5 pp. This suggests that on simpler tasks, examples provide sufficient guidance without reasoning overhead.

3. **Efficiency trade-offs:** Speed ratios range from 1.0× to 54.4×, with highest overhead on GoEmotions (Magistral: 54.4×). Mean overhead across model families: Granite 1.0×–5.5×, Qwen3 2.5×–11.5×, Magistral 9.7×–54.4×.

Aggregate Performance Across Task Complexity

Table 5.11 presents aggregated statistics across all 504 configurations, stratified by dataset and model type.

Table 5.11: Aggregated Performance Statistics Across 504 Configurations. Statistics reported as mean ± std across 12 model pairs and 7 shot levels (0–50 shots).

Dataset	**Type**	**N**	**F1 (%)**	**Diff. (%)**
IMDB	Base/Non-think.	84	91.5 ± 20.8	-4.8 ± 6.3
	Dist./Think.	84	86.7 ± 25.7	
Amazon	Base/Non-think.	84	74.9 ± 18.9	-3.6 ± 2.2
	Dist./Think.	84	71.3 ± 17.7	
GoEmotions	Base/Non-think.	84	36.6 ± 4.0	$+2.0 \pm 1.0$
	Dist./Think.	84	38.6 ± 7.3	

The decreasing performance gap with increasing task complexity—from -4.8 pp (binary) through -3.6 pp (5-class) to +2.0 pp (27-class)—validates the task-complexity hypothesis: reasoning mechanisms become more beneficial as classification granularity increases.

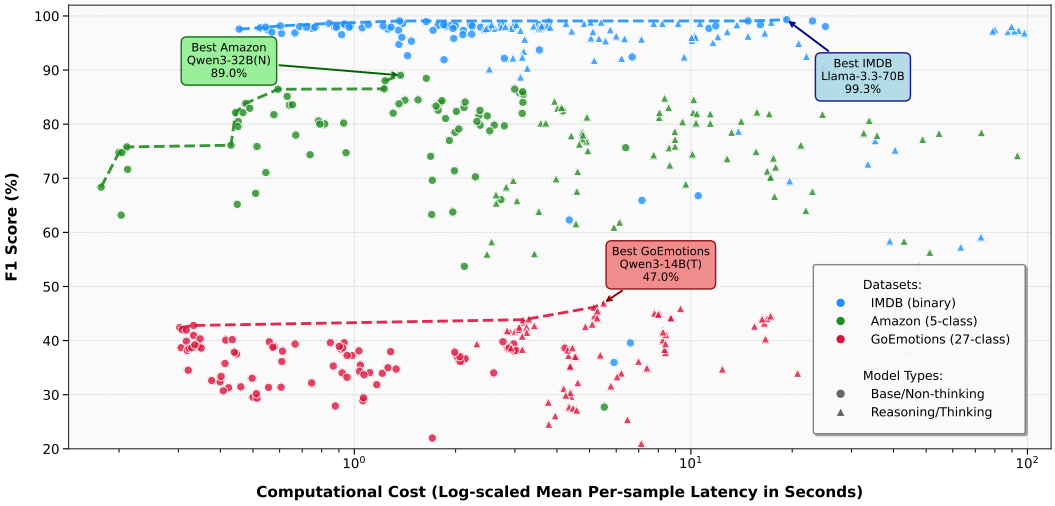


Figure 5.2: Performance vs. computational cost. Circles: base/non-thinking; triangles: reasoning/thinking. Colors: IMDB (blue), Amazon (green), GoEmotions (red). Dashed lines: Pareto frontiers where no configuration dominates on both metrics. Evaluated on H100 via Ollama; DSR1/V3 excluded for consistency. Labels show top performers per dataset.

Pareto Frontier Analysis

Figure 5.2 visualizes efficiency-performance trade-offs across all configurations. The Pareto frontier represents configurations where no alternative improves F1 without increasing latency, or reduces latency without sacrificing F1. This analysis extends the cost-effectiveness evaluation approach used for the LAJ framework in Chapter 4, applying similar principles to reasoning model selection.

Key observations from Pareto analysis:

1. **Task-dependent model representation:** IMDB frontier is dominated by base models achieving 97–99% F1 with minimal latency. GoEmotions frontier includes reasoning models more frequently, justifying their overhead through accuracy gains.

2. **Cost-performance curves:** IMDB shows flat frontier at high F1, indicating diminishing returns from additional compute. GoEmotions exhibits steeper slope, where latency increases yield greater accuracy improvements.

3. **Practical guidance:** Base models dominate for most sentiment classification

tasks. Reasoning should be reserved for complex emotion recognition where 2–4 pp improvements justify 2.1×–54× computational overhead.

Summary of Task Complexity Findings

This comprehensive evaluation reveals that reasoning effectiveness in sentiment analysis is systematically task-dependent:

1. **Simple tasks (binary classification):** Base/non-thinking models provide superior performance (4.8 pp advantage) with lower computational overhead. Reasoning introduces noise rather than clarification.

2. **Moderate tasks (5-class sentiment):** Mixed results with 3.6 pp average base model advantage. Reasoning can help disambiguate adjacent categories but may overcomplicate straightforward cases.

3. **Complex tasks (27-class emotion):** Reasoning models achieve 2.0 pp advantage, with the lowest failure rate (14%) among thinking vs. non-thinking comparisons. Extended reasoning helps navigate fine-grained distinctions between similar emotions.

4. **Few-shot learning superiority:** Regardless of reasoning mode, few-shot learning yields improvements over zero-shot in the vast majority of cases, positioning it as a more robust strategy than reasoning activation for enhancing sentiment analysis performance.

These findings provide principled deployment guidance: reserve reasoning for complex multi-class problems where accuracy gains justify computational overhead, and leverage few-shot learning for consistent improvements across all task complexities.

5.4 Cross-Domain Applications

5.4.1 Chinese-English Translation Optimization

Motivation

Machine translation between Chinese and English poses particular challenges due to significant differences in syntax, semantics, and morphology. Traditional approaches have struggled with complexities inherent in Chinese-English translation, particularly in handling idiomatic expressions, word order differences, and character-based text segmentation. We comprehensively evaluate LLMs across zero-shot, few-shot, and fine-tuned paradigms to establish best practices for practical deployment.

Dataset

We use the MAC (Manually Aligned Chinese-English) dataset[1], comprising parallel Mandarin-English texts from six Chinese novels representing diverse genres: humor, martial arts, classics, war, romance, and science fiction. After preprocessing to remove entries with missing content and splitting 80%/20%, the dataset contains 4,528 training and 1,133 testing sentence pairs.

Evaluation Metrics

1. **COMET-22:** Neural-based metric using contextual embeddings from pretrained language models. COMET-22 shows strong correlation with human judgments for Chinese-English translation, outperforming traditional metrics like BLEU and TER in alignment with human evaluations.

2. **Translation Completeness Ratio (TCR):** During evaluation, we observed that some LLM outputs contained untranslated segments with Chinese characters.

[1] `https://github.com/bfsujason/mac`

TCR measures the proportion of translations fully rendered into English:

$$TCR = \frac{\text{Number of complete translations}}{\text{Total number of entries}} \tag{5.5}$$

where a "complete translation" is defined as one where no Chinese characters remain in the output text. This metric addresses a reliability dimension similar to VCR for classification tasks, capturing whether models produce deployable outputs.

3. **Characters per Second (CPS):** Translation efficiency metric measuring processing speed:

$$CPS = \frac{\text{Number of Chinese characters in dataset}}{\text{Total translation time (seconds)}} \tag{5.6}$$

Results

Table 5.12 presents translation performance across paradigms.

Table 5.12: Chinese-English Translation Performance

Model	**Paradigm**	**COMET**	**TCR (%)**	**CPS**
Qwen2-72B	Zero-shot	**0.7324**	98.2	12.4
GPT-4o	Zero-shot	0.7258	99.1	18.2
GPT-4o-mini	Zero-shot	0.7259	98.8	24.5
InternLM2.5-7B	Zero-shot	0.7174	97.5	38.2
Qwen2-7B	Zero-shot	0.7170	97.8	35.6
Llama-3.1-8B	Zero-shot	0.7089	96.8	32.1
Mistral-7B	Zero-shot	0.6945	95.2	41.8
Phi-3.5-mini	Zero-shot	0.6823	94.5	45.2
Qwen2-72B	Fine-tuned	**0.7345**	99.2	12.1
Llama-3.1-70B	Fine-tuned	0.7312	99.0	14.9
InternLM2.5-7B	Fine-tuned	0.7289	98.8	36.8
Qwen2-7B	Fine-tuned	0.7256	98.5	34.2
Llama-3.1-8B	Fine-tuned	0.7234	98.2	30.5
Phi-3.5-mini	Fine-tuned	0.7198	98.5	41.8
Mistral-7B	Fine-tuned	0.7145	97.8	39.5

Key findings:

1. **Zero-shot performance:** Qwen2-72B demonstrated the highest zero-shot performance (COMET 0.7324), followed closely by GPT-4o (0.7258) and GPT-4o-mini (0.7259). This suggests that models with strong multilingual pretraining can achieve competitive translation quality without task-specific examples.

2. **Fine-tuning benefits:** Fine-tuning improves both translation quality and efficiency for open-source models. Fine-tuned Qwen2-72B achieves COMET 0.7345 with 99.2% TCR, representing modest but consistent improvements. Notably, for all open-source LLMs, fine-tuning not only improves COMET scores but also increases CPS rates compared to few-shot learning.

3. **Efficiency-quality trade-off:** Smaller fine-tuned models (Phi-3.5-mini, InternLM2.5-7B) offer 2–3× speed improvements with acceptable quality loss, suitable for high-throughput applications where perfect translation is not critical.

4. **Optimal fine-tuning epochs:** Larger models peak earlier (2–3 epochs) than smaller ones (4–5 epochs), with training beyond optimal epochs leading to overfitting. This pattern is consistent with our findings in logical reasoning fine-tuning (Section 5.3.1).

5. **LoRA/QLoRA effectiveness:** Parameter-efficient fine-tuning with LoRA/QLoRA enables the same LLM to be adapted for multiple tasks. Different adapters can be swapped for inference, creating a highly versatile solution without retraining entire models.

5.4.2 LLM Sentiment for Sports Betting Prediction

Motivation

Traditional sports betting models have primarily relied on quantitative statistics, often overlooking the predictive power of qualitative insights such as sentiment from match commentaries. We present a novel hybrid framework integrating LLM-derived sentiment features into machine learning models for soccer betting prediction,

demonstrating that pre-trained LLMs can effectively capture latent factors often missed by traditional statistical methods.

Methodology

Data Sources:

- Match statistics from API-Football (2015-16 to 2023-24 seasons, 9 seasons)
- Betting odds from Football-Data.co.uk (1X2 pre-match odds from BetFair)
- Post-match commentaries from BBC Sport, scraped using Python Selenium

Data Partitioning: 2015–19 for feature selection/hyperparameter tuning; 2015–22 for training; 2022–23 for testing; 2023–24 for betting simulation.

LLM Sentiment Analysis: We employ zero-shot prompting with DeepSeek-R1, GPT-4o-mini, GPT-4.1, and Qwen2.5-Plus to extract sentiment ratings (1–5 scale: Strongly Negative to Strongly Positive) from match commentaries. The final sentiment feature is computed as the difference between home and away team ratings.

Machine Learning Models: Logistic Regression, SVC, Random Forest, and XGBoost, evaluated under both accuracy-tuned and F1-tuned configurations with grid search hyperparameter optimization and 5-fold cross-validation.

Betting Strategy: Simulated bankroll of $10,000 using one-eighth Kelly betting criterion, placing bets only when positive expected value conditions are satisfied.

Results

Table 5.13 presents betting simulation returns for the 2023-24 EPL season.

Key findings:

1. **F1 tuning superiority:** F1-tuned models consistently outperform accuracy-tuned configurations, with returns often 2–3$\times$ higher. This demonstrates that optimizing for balanced precision and recall is better aligned with profitable sports betting than accuracy optimization, as F1 tuning produces more calibrated probability

Table 5.13: Soccer Betting Returns: Baseline vs. LLM-Enhanced Models (%)

Model	LLM Enhancement	Accuracy-Tuned	F1-Tuned
Logistic Regression	Baseline	75.12	105.22
Logistic Regression	DeepSeek-R1	89.35	**174.01**
Logistic Regression	GPT-4.1	79.52	60.02
Logistic Regression	GPT-4o-mini	27.10	110.64
Logistic Regression	Qwen2.5-Plus	41.98	75.04
SVC	Baseline	72.18	33.07
SVC	DeepSeek-R1	25.02	25.02
SVC	GPT-4o-mini	19.12	89.81
Random Forest	Baseline	-21.01	113.96
Random Forest	GPT-4.1	7.32	**279.48**
Random Forest	Qwen2.5-Plus	65.54	74.04
Random Forest	DeepSeek-R1	22.03	164.76
XGBoost	Baseline	8.30	**247.97**
XGBoost	DeepSeek-R1	92.07	33.15
XGBoost	GPT-4o-mini	-5.55	-16.09
XGBoost	GPT-4.1	5.72	30.06

estimates. This finding reinforces the broader dissertation theme that metric selection must align with downstream objectives.

2. **Best performing configurations:**

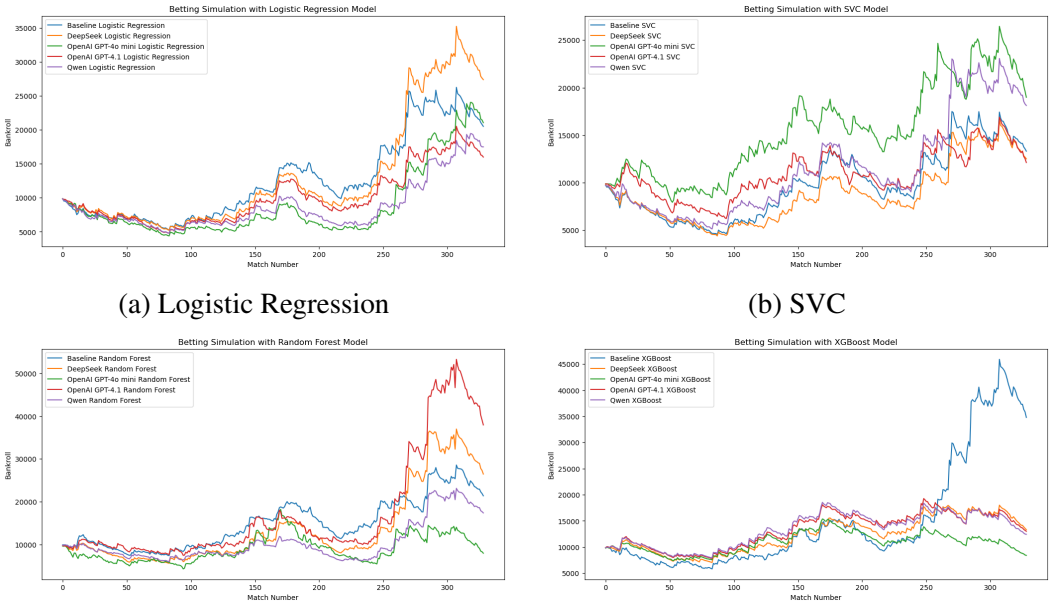
   - Random Forest + GPT-4.1 (F1-tuned): 279.48% return—highest overall
   - XGBoost Baseline (F1-tuned): 247.97% return—strong baseline performance
   - Logistic Regression + DeepSeek-R1 (F1-tuned): 174.01% return—demonstrating that even simple models benefit from sentiment enhancement
   - Random Forest + DeepSeek-R1 (F1-tuned): 164.76% return

3. **LLM-model compatibility:** The impact of LLM-derived sentiment features varies across architectures. While certain combinations produce exceptional results (Random Forest + GPT-4.1), the same LLM can show different effects with different model architectures. Notably, XGBoost baseline outperforms

some LLM-enhanced configurations, underscoring the importance of systematic empirical validation.

4. **Zero-shot effectiveness:** Modern LLMs serve as powerful zero-shot sentiment analysis engines requiring no domain-specific training, enabling rapid deployment of sentiment-enhanced prediction systems. This demonstrates that thoughtful feature engineering with LLM-derived features can surpass architectural complexity.

Figure 5.3 illustrates bankroll evolution across the 2023-24 season, demonstrating the superior stability and growth of F1-tuned models compared to accuracy-tuned alternatives.



(a) Logistic Regression (b) SVC

(c) Random Forest (d) XGBoost

Figure 5.3: Betting simulation bankroll evolution across matches for the 2023-24 EPL season comparing F1-tuned baseline models with their LLM-infused variations across four different machine learning architectures. The results demonstrate that F1-tuned configurations consistently produce more stable growth trajectories and higher final returns.

5.4.3 Entity Matching with LLMs at Scale

Motivation

Entity matching (EM)—the task of identifying records that refer to the same real-world entity across different data sources—is fundamental to data integration in enterprises. Financial institutions, for example, must match raw transaction descriptors to canonical merchant records. Traditional approaches rely on rule-based, distance-based, or supervised-learning pipelines [40, 47, 110, 121], but LLMs now enable zero-shot entity matching that requires no task-specific training data. However, three questions remain open for practitioners: (1) can open-weight models match proprietary APIs on EM tasks? (2) how does configurable reasoning effort affect accuracy and cost? and (3) what budget tier achieves acceptable performance?

Background: ComEM Framework

We adopt the Compound Entity Matching (ComEM) framework of Wang et al. [159], which defines three LLM-based matching strategies:

- **Matching:** The LLM acts as a binary classifier, independently determining whether each pair (r, r_i) is a match.
- **Comparing:** The LLM compares two candidate records r_i and r_j against the anchor r to decide which is more consistent, requiring $O(kn)$ invocations.
- **Selecting:** The LLM directly selects the matching record from the full candidate list, requiring only $O(1)$ invocations per anchor but sensitive to position bias.

ComEM composes these into a two-stage pipeline. In the *ranking stage*, a cost-efficient model (Flan-T5-XL) scores all n blocking candidates using the matching strategy and retains the top-k. In the *selection stage*, a more capable LLM applies the selecting strategy on the filtered k candidates, with a "[0] if there is none" abstention option. Wang et al. show that $k = 4$ balances F1 and cost while mitigating position bias and reducing context length.

Experimental Setup

Datasets. We use 8 pyJedAI clean-clean entity resolution benchmark datasets spanning three domains: product matching (Abt-Buy, Amazon-Google, Walmart-Amazon), citation deduplication (DBLP-ACM, DBLP-Scholar), and entertainment record linkage (IMDB-TMDB, IMDB-TVDB, TMDB-TVDB). For each dataset, 400 anchor records are sampled (300 with matches, 100 without; seed 42), yielding 3,200 selecting-stage evaluations across all datasets. Sparkly blocking at $n = 10$ candidates per anchor is applied, consistent with the original ComEM protocol.

Models. We evaluate 46 model configurations spanning four categories:

- **OpenAI proprietary** (15 variants): GPT-4o-mini, GPT-4o, GPT-4.1 family (nano/mini/standard), GPT-5 family (nano/mini/standard $\times$ low/medium/high reasoning), GPT-5.2 (low/medium/high)
- **Anthropic proprietary** (10 variants): Claude Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6—each with and without thinking/adaptive-thinking modes
- **Open-weight local** (7 models): GPT-OSS (20B, 120B) at three reasoning levels, GLM-4.7-Flash, Gemma3:27b, Llama4:Scout, Granite4:32b—deployed on NVIDIA DGX Spark (128 GB unified memory) using Ollama v0.15.6 with sequential inference (batch size $= 1$, single concurrent request), and `num_ctx=8192`.
- **Open-weight via OpenRouter API** (8 models): DeepSeek-Chat-v3.1, Kimi-K2, GLM-5, Qwen3-235b, MiniMax-M2.5—with and without reasoning/thinking variants

All models use temperature 0 and the identical ComEM selecting-stage prompt. Only the selecting-stage LLM and reasoning configuration vary; all other pipeline components remain fixed.

Evaluation metrics. We report F1 score on the positive class (matching pairs), together with Precision and Recall, computed as the mean across all 8 datasets.

Cost metric. We compute API-equivalent token cost as Cost $= N_{\text{in}} \times P_{\text{in}} + (N_{\text{out}} + N_{\text{reason}}) \times P_{\text{out}}$, summed across all 8 datasets for numerical precision.

Results

Table 5.14 presents the top-performing models.

Table 5.14: Entity Matching: Top 15 of 46 Model Configurations (Mean F1 across 8 datasets)

Rank	Model	F1	Prec.	Rec.	Cost ($)
1	gpt-5 (medium)	93.17	92.04	94.42	12.29
2	gpt-5 (low)	93.14	92.26	94.12	6.25
3	gpt-5.2 (low)	92.75	90.58	95.12	3.43
4	gpt-5.2 (high)	92.54	90.22	95.08	5.74
5	gpt-5.2 (medium)	92.53	90.33	94.92	4.52
6	gpt-5 (high)	92.14	92.66	91.71	24.50
7	gpt-oss:120b (high)	92.10	89.91	94.46	1.20
8	kimi-k2-think	91.84	90.00	93.83	8.29
9	gpt-5-mini (low)	91.72	88.64	95.08	0.71
10	gpt-5-mini (high)	91.62	88.47	95.08	2.30
11	gpt-5-mini (medium)	91.46	88.21	95.04	1.28
12	gpt-oss:20b (high)	91.36	89.86	93.00	0.46
13	minimax-m2.5	91.34	90.64	92.12	3.74
14	gpt-oss:120b (medium)	91.26	90.29	92.33	0.38
15	claude-haiku-4.5 (think)	91.19	88.63	94.00	8.64

Open-weight models rival proprietary APIs. GPT-OSS:120b (high) achieves 92.10 F1, outperforming prior-generation proprietary baselines GPT-4.1 (90.36) and GPT-4o (89.58) while costing only $1.20—approximately 20× less than GPT-5 (high) at $24.50. GPT-OSS:20b (high) achieves 91.36 F1 at just $0.46. This competitiveness reflects the architectural maturity of recent open-weight models and the structured nature of the EM task, which favours strong instruction-following over broad world knowledge.

Reasoning effort has opposite effects across families. Table 5.15 summarises the impact of reasoning effort across model families. The optimal reasoning level is family-dependent rather than universal.

For GPT-5, low reasoning achieves 93.14 F1 at $6.25, virtually identical to

Table 5.15: Impact of reasoning effort on entity matching across model families. F1 is the mean across 8 datasets. "Best" = recommended setting considering both F1 and cost; *medium is marginally higher (93.17 vs. 93.14) but costs 2× more. ΔF1 = F1 range (max−min); ΔCost = cost multiplier from lowest to highest setting. For models with a thinking toggle, "On" and "Off" denote thinking enabled/disabled; "Adapt" denotes adaptive thinking.

Model Family	Low	Medium	High	Best	ΔF1	ΔCost	Trend
gpt-5	**93.14**	93.17	92.14	Low*	1.03	3.9×	↓
gpt-5.2	**92.75**	92.53	92.54	Low	0.22	1.7×	≈
gpt-5-mini	**91.72**	91.46	91.62	Low	0.26	3.2×	≈
gpt-5-nano	88.38	**89.34**	85.71	Medium	3.63	6.3×	∩
gpt-oss:120b	90.11	91.26	**92.10**	High	1.99	7.1×	↑
gpt-oss:20b	88.74	90.87	**91.36**	High	2.62	5.8×	↑
	Off	**On / Adapt**					
claude-sonnet-4.5	79.57	**89.70**		On	10.13	1.9×	↑↑
claude-haiku-4.5	82.64	**91.19**		On	8.55	2.0×	↑↑
claude-opus-4.6	85.18	**90.13**		Adapt	4.95	1.4×	↑
claude-sonnet-4.6	82.48	82.55		Adapt	0.07	1.0×	≈
claude-opus-4.5	**80.53**	80.28		Off	0.25	2.6×	↓
glm-5	79.77	**91.02**		On	11.25	23×	↑↑
qwen3-235b	**86.85**	81.76		Off	5.09	34×	↓
deepseek-v3.1	79.16	**86.52**		On	7.36	7.0×	↑

medium (93.17 at $12.29) while costing 49% less; high reasoning actually *degrades* performance to 92.14 F1 at 4× the cost. GPT-5.2 and GPT-5-mini show minimal sensitivity, making low the value choice in both cases. In contrast, open-weight models benefit substantially from extended reasoning: GPT-OSS:120b improves from 90.11 to 92.10 F1 (+1.99 pp) and GPT-OSS:20b from 88.74 to 91.36 F1 (+2.62 pp) with increasing effort.

The Claude family exhibits the most extreme and asymmetric thinking effects. Thinking primarily boosts precision while maintaining recall: Sonnet 4.5 gains +10.13 F1 (precision +15.22 pp), Haiku 4.5 gains +8.55 F1, and Opus 4.6 gains +4.95 F1 with adaptive thinking. Yet Opus 4.5 *loses* 0.25 F1 with thinking at 2.6× cost, and Sonnet 4.6 gains only +0.07. Without thinking, all Claude models share high recall (92–93%) but poor precision (70–79%), suggesting they default to predicting "match" without deeper analysis. Thinking can also hurt elsewhere: Qwen3-235b drops 5.09 F1 at 34× cost.

This family-dependent pattern extends the task-complexity findings from Section 5.3.3: just as reasoning effectiveness depends on task complexity, it also depends on model family, with open-weight models benefiting from extended deliberation likely because they have less internalised entity matching knowledge than frontier proprietary models.

Cost-performance analysis. Pareto frontier analysis across all 46 configurations reveals three actionable tiers:

- **Ultra-budget** (<$0.10): Qwen3-235b (86.85 F1) and GPT-OSS:20b-low (88.74 F1) provide viable performance at minimal cost.
- **Value** ($0.46–$1.20): GPT-OSS:20b-high (91.36 F1) and GPT-OSS:120b-high (92.10 F1) offer the steepest quality-per-dollar gains, approaching frontier-model accuracy.
- **Premium** ($3+): Diminishing returns—each additional dollar beyond $3 buys <0.3 F1 points. GPT-5 (low) at $6.25 achieves 93.14 F1; paying 4× more for GPT-5 (high) *reduces* F1 to 92.14.

Only 13 of 46 configurations form the Pareto frontier, dominated by the GPT ecosystem (GPT-OSS variants, GPT-5-mini, GPT-5.2, GPT-5) plus Gemma3:27b and Qwen3-235b. No Claude or GLM configuration reaches the frontier. Figure 5.4 visualises this cost-performance landscape.

These findings directly inform model selection for production entity matching systems, where the value tier (GPT-OSS at $0.46–$1.20) offers near-frontier accuracy with infrastructure costs compatible with high-volume transaction processing [82].

5.5 Summary and Implications

This chapter addressed RQ3 by examining deployment configurations, reasoning enhancement techniques, and application strategies that maximize LLM effectiveness

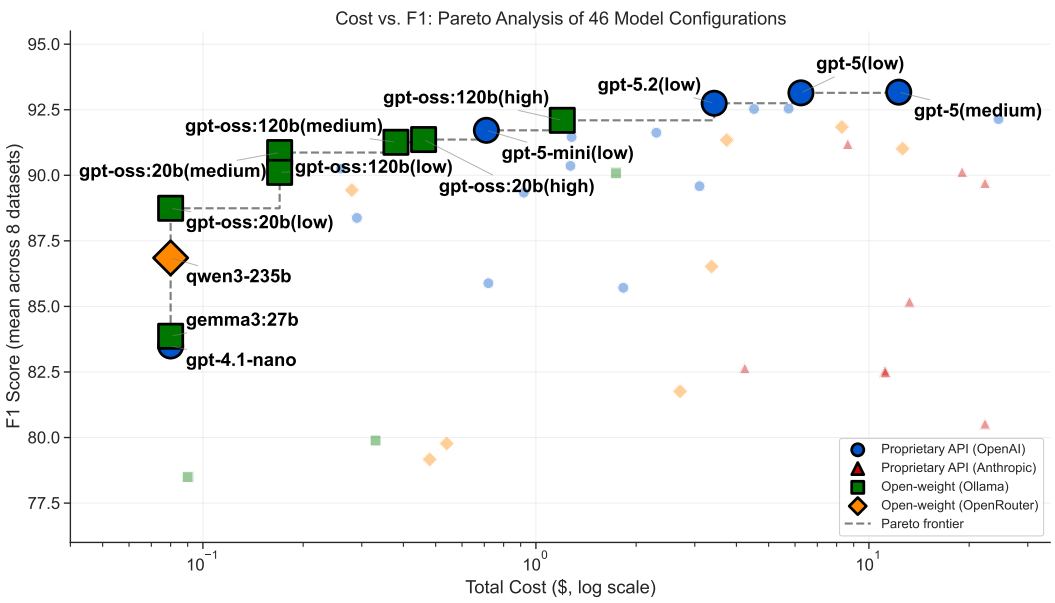


Figure 5.4: Cost vs. F1 Pareto analysis of 46 model configurations for entity matching using the compound strategy. Each point represents a model's mean F1 (across 8 datasets) and total API-equivalent token cost. The dashed red step line traces the Pareto frontier. Marker shapes distinguish proprietary API models (circles), open-weight models deployed locally via Ollama (squares), and open-weight models accessed via the OpenRouter API (diamonds).

across diverse enterprise use cases. The findings connect to and extend the optimization principles established in Chapters 3 and 4, demonstrating their applicability across edge deployment, reasoning enhancement, and cross-domain applications.

5.5.1 Edge Deployment Guidelines

1. **Optimal model sizing:** Medium-sized models (7B–32B) achieve the best balance between performance and efficiency for edge deployment. qwen2.5:32b attains 96.75% (±0.11%) F1 with practical inference speeds, while smaller models like qwen2.5:3b achieve 90.89% F1 with significantly lower resource requirements.

2. **Quantization effectiveness:** Ollama's Q4_K_M quantization enables efficient deployment on consumer hardware while preserving or improving performance compared to bitsandbytes implementations (up to 3.37 percentage point F1 improvement observed).

3. **Platform optimization:** WSL delivers up to 21× throughput improvements over native Windows for smaller models, enabling efficient deployment on consumer-grade systems without Linux installation.

4. **Consumer hardware viability:** For reasoning tasks, models up to 14B parameters are feasible on 16GB consumer GPUs. qwen3:8b offers optimal accuracy-efficiency trade-offs (84.8% F1, 13GB VRAM, 8.35s inference) for privacy-preserving local deployment.

5. **Energy considerations:** Small models (1.5B–3B) achieve the highest performance-to-power ratios (0.161–0.163 %/J), ideal for energy-constrained deployments, while large models (70B+) consume 1,283–1,876 Joules per inference.

5.5.2 Reasoning Enhancement Insights

1. **Open-source competitiveness:** Fine-tuned Qwen2.5 and Llama3.1 models match or exceed GPT-4o on complex reasoning tasks. Qwen2.5-72B achieves 80.88% F1 on TSQA with only 10-shot prompting, surpassing GPT-4o's 80.36%. This extends the open-source viability findings from Chapter 3 to reasoning-intensive tasks.

2. **Task-complexity-dependent reasoning effectiveness:** Comprehensive evaluation across 504 configurations reveals that reasoning effectiveness is systematically task-dependent. Base/non-thinking models outperform reasoning variants by 4.8 ± 6.3 pp on binary classification (IMDB) and 3.6 ± 2.2 pp on 5-class sentiment (Amazon), while reasoning models achieve 2.0 ± 1.0 pp advantage on 27-class emotion recognition (GoEmotions). This task-complexity hypothesis provides principled guidance: reserve reasoning for complex multi-class problems where accuracy gains justify 2.1×–54× computational overhead.

3. **Distillation transfer limitations:** Reasoning patterns optimized for mathematical

and logical tasks transfer poorly to simpler sentiment classification. Failure rates decrease with task complexity: 100% (IMDB), 80% (Amazon), 50% (GoEmotions), indicating reasoning distillation becomes more beneficial as classification granularity increases.

4. **Valid Classification Ratio (VCR):** Novel metric for assessing output reliability beyond accuracy, particularly important for structured classification tasks where invalid outputs cause downstream failures. Fine-tuning improves VCR to >98.6% across evaluated models, with some models improving from as low as 1.17% to 100% within 0.2 epochs.

5. **Reasoning Intensity (RI):** Novel metric capturing the trade-off between explanation depth and computational cost. Three operational regimes identified: Minimal (RI < 0.5), Balanced (0.5–0.85), and Intensive (> 0.85), with balanced regime yielding optimal accuracy-efficiency trade-offs.

6. **Explainability benefits:** DeepSeek-R1's intrinsic reasoning traces provide transparent decision-making processes without external explanation mechanisms, enhancing auditability for enterprise applications.

7. **Few-shot learning superiority:** Few-shot learning yields improvements over zero-shot in the vast majority of cases, regardless of reasoning mode, positioning it as a more robust and consistent strategy than reasoning activation for enhancing sentiment analysis performance. DeepSeek-R1 demonstrates approximately 8× improvement in few-shot efficiency over GPT-4o (30 shots vs. 40 shots for comparable performance).

8. **Pareto-optimal model selection:** Pareto frontier analysis reveals that base models dominate efficiency-performance trade-offs for most sentiment tasks. IMDB frontier is dominated by base models achieving 97–99% F1 with minimal latency, while GoEmotions includes reasoning models that justify overhead through accuracy gains.

9. **Temporal robustness challenges:** All evaluated models exhibit 10–17 percentage point F1 degradation on contemporary data, though reasoning intensity remains stable (± 0.02). This suggests models require periodic updating to maintain accuracy on evolving content.

5.5.3 Cross-Domain Application Patterns

1. **Fine-tuning universality:** Parameter-efficient fine-tuning (LoRA/QLoRA) consistently improves performance across translation, reasoning, and sentiment tasks. The technique enables multi-task adaptation through swappable adapters without full model retraining.

2. **Zero-shot LLM capabilities:** Modern LLMs effectively extract nuanced signals from unstructured text without domain-specific training, enabling rapid deployment across diverse domains. This was demonstrated in both Chinese-English translation (COMET 0.7324 for Qwen2-72B) and sports sentiment extraction.

3. **Optimization strategy alignment:** Task-specific optimization dramatically improves practical outcomes. F1 tuning for sports betting produces 2–3$\times$ higher returns than accuracy tuning, demonstrating that metric selection must align with downstream objectives.

4. **Novel evaluation metrics:** This chapter introduces four domain-specific metrics, complementing the RAP metric from Chapter 3 and the ASR/ECR@1 metrics from Chapter 4:

    - **VCR** (Valid Classification Ratio) for output reliability in classification tasks
    - **RI** (Reasoning Intensity) for reasoning-explainability trade-offs
    - **TCR** (Translation Completeness Ratio) for translation reliability
    - **CPS** (Characters per Second) for translation efficiency

5. **Entity matching at scale:** Evaluation of 46 model configurations across 8 benchmark datasets reveals that open-weight models rival proprietary APIs (GPT-OSS:120b achieves 92.10 F1, outperforming GPT-4.1 at 20× lower cost). Reasoning effort has opposite effects across model families—low is optimal for GPT-5, high is critical for open-weight models, and Claude shows extreme asymmetric thinking effects. Cost-performance Pareto analysis identifies sharp diminishing returns beyond \$3, with ultra-budget configurations (<\$0.10) achieving 87–89 F1.

These findings provide practitioners with evidence-based guidelines for deploying LLMs across diverse enterprise contexts, balancing performance, efficiency, explainability, and cost considerations. The research demonstrates that open-source models can achieve enterprise-grade performance when properly optimized, enabling privacy-preserving edge deployment while maintaining competitive accuracy with proprietary alternatives.

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

Having previewed the key contributions organized by type in Chapter 1 (Section 1.4), we now provide a detailed synthesis organized by research pillar, reflecting on what was achieved across the research portfolio spanning seventeen peer-reviewed publications at premier AI/NLP venues.

This dissertation has systematically investigated optimization strategies for deploying Large Language Models in enterprise applications. The research contributions span three interconnected pillars—Retrieval-Augmented Generation optimization, agentic AI for business automation, and enterprise deployment strategies—supported by cross-domain validation across diverse enterprise applications.

6.1.1 RAG Optimization Contributions (Chapter 3)

The first pillar addressed RQ1: *How can RAG systems be optimized to improve response quality while addressing critical issues such as content repetition in open-source LLMs?*

1. **Repetition-Aware Performance (RAP) Metric:** We introduced a novel evaluation metric that addresses a critical gap in assessing open-source LLM out-

puts. RAP quantifies and penalizes repetition through the formula RAP $= P \times (1 - \text{RR})^3$, enabling effective tuning of the repetition penalty parameter (RPP). Comprehensive experiments across twelve models and three datasets demonstrated that RAP enables up to 93.1% reduction in repetition with minimal performance degradation (typically under 4%). The cubic penalty function was validated through ablation studies as optimal across diverse tasks.

2. **Repetition Detection Algorithm (ReDA):** Complementing the RAP metric, we developed ReDA for systematic identification and quantification of repetitive content in LLM outputs, enabling automated quality assessment in production systems.

3. **Comparative Evaluation Framework:** Systematic benchmarking on the PCI DSS compliance domain established that smaller, efficient models like Orca-2-13b can achieve competitive—and sometimes superior—performance to proprietary alternatives in RAG applications. Orca-2-13b achieved 99.3% overall RAGAS score with perfect faithfulness, challenging assumptions that larger models are necessary for high-quality enterprise deployments.

4. **Evaluation-Guided Hyperparameter Search:** We formalized an optimization procedure using RAP as the target metric, enabling systematic identification of configurations that balance factual accuracy, relevance, and output conciseness.

6.1.2 Agentic AI Contributions (Chapter 4)

The second pillar addressed RQ2: *How can multi-agent LLM architectures be designed to reliably automate complex enterprise workflows while providing measurable reliability guarantees?*

1. **Multi-Agent Invoice Reconciliation System:** We designed and validated a three-agent architecture (Data Engineering, Email, Reconciliation) achieving 99.90% success rate with the Qwen2.5-32B model—outperforming all proprietary cloud

APIs while consuming only 523 Joules per task on consumer hardware (RTX 4090).

2. **12-Category Error Taxonomy:** We developed a comprehensive diagnostic framework for tool invocation reliability, distinguishing tool initialization, parameter handling, execution, and result interpretation errors. This taxonomy enables systematic failure characterization and targeted improvement strategies for multi-agent systems.

3. **Evaluation Completion Rate (ECR@1):** We introduced an operational reliability metric measuring first-attempt evaluation success rates. ECR@1 reveals reliability variations from 85.4% to 100% across models, with direct cost implications via retry overhead invisible to accuracy-only evaluation.

4. **LLM-as-a-Judge (LAJ) Framework:** We developed a production-ready, rubric-driven framework for evaluating Gherkin acceptance tests with structured JSON outputs, revealing 175× cost variation ($0.45–$78.96 per 1K evaluations) and demonstrating that GPT-4o Mini achieves best-in-class accuracy at 78× cost reduction versus premium models.

5. **Hierarchical Multi-Agent System for Payments (HMASP):** We designed the first LLM-based multi-agent system for end-to-end agentic payment workflows, featuring a four-level hierarchy with architectural patterns including shared state variables, decoupled message states, and structured handoff protocols, achieving 97.0% task success rate.

6. **Agentic Success Rate (ASR):** We introduced a trajectory-level workflow fidelity metric grounded in process-mining conformance checking that compares observed and expected agent execution sequences at the transition level. Across 18 LLMs and 90,000 task instances, ASR exposes systematic procedural deviations invisible to TSR and HF1—7 of 8 top models skip confirmation checkpoints during payment checkout despite near-perfect task success—demonstrating that

trajectory-level evaluation is essential in regulated domains where compliance and auditability are paramount.

6.1.3 Enterprise Deployment and Application Contributions (Chapter 5)

The third pillar addressed RQ3: *What strategies enable effective LLM deployment across diverse enterprise applications, hardware configurations, and domain requirements?*

1. **Comprehensive Edge Deployment Benchmarking:** Systematic evaluation across six hardware platforms (from NVIDIA H100 to consumer laptops and edge devices) established that medium-sized models (7B–32B) achieve optimal performance-efficiency trade-offs. WSL provides up to 21× speedup over native Windows for smaller models, while Ollama's Q4_K_M quantization enables efficient deployment with up to 3.37 percentage point F1 improvement over bitsandbytes implementations.

2. **Reasoning Intensity (RI) Metric:** We introduced a metric quantifying the proportion of computational effort devoted to reasoning, computed as $\text{RI} = \frac{\text{MRT}}{\text{MRT}+\text{MOT}}$. Three operational regimes were identified: Minimal ($\text{RI} < 0.5$), Balanced (0.5–0.85), and Intensive (> 0.85), with balanced regime yielding optimal accuracy-efficiency trade-offs.

3. **Valid Classification Ratio (VCR):** We introduced a reliability metric measuring the proportion of valid classifications among total outputs. Fine-tuning dramatically improves VCR from as low as 1.17% to 100% within 0.2 epochs, providing practitioners with a concrete measure of deployment readiness for structured classification tasks.

4. **Task-Complexity-Dependent Reasoning Effectiveness:** Through comprehensive evaluation of 504 configurations across seven model families (DeepSeek-R1,

Qwen3, Granite, Magistral, and their base variants), we established that reasoning effectiveness is systematically task-dependent. Base/non-thinking models outperform reasoning variants by 4.8 ± 6.3 percentage points on binary classification (100% failure rate for reasoning) and 3.6 ± 2.2 pp on 5-class sentiment (80% failure rate), while reasoning models achieve 2.0 ± 1.0 pp advantage on 27-class emotion recognition (only 14% failure rate for thinking modes). This task-complexity hypothesis provides principled guidance for model selection.

5. **Pareto Frontier Analysis:** We conducted the first Pareto frontier analysis of performance versus computational cost for reasoning models in sentiment analysis, demonstrating that base models dominate efficiency-performance trade-offs for most sentiment tasks, with reasoning justified only for complex emotion recognition where 2–4 pp improvements justify 2.1×–54× computational overhead.

6. **Translation Completeness Ratio (TCR) and Characters per Second (CPS):** We introduced domain-specific metrics for machine translation reliability and efficiency, addressing the observation that LLMs sometimes produce outputs with untranslated segments.

7. **Cross-Domain Validation:** Applications across financial prediction, sentiment analysis, machine translation, logical reasoning, sports predictive analytics, and entity matching validated the generalizability of optimization techniques. F1-tuned models for sports betting achieved 279.48% returns (Random Forest + GPT-4.1), demonstrating that metric selection must align with downstream objectives.

8. **Entity Matching at Scale:** The largest empirical evaluation of LLM-based entity matching to date—46 model configurations across 8 benchmark datasets—revealed that open-weight models rival proprietary APIs (GPT-OSS:120b achieves 92.10 F1, outperforming GPT-4.1 at 20× lower cost) and that reasoning effort has family-dependent effects: low is optimal for GPT-5, high is critical for open-weight models, and Claude shows extreme asymmetric thinking impacts

(+10.13 F1 for Sonnet 4.5, −0.25 for Opus 4.5). Cost-performance Pareto analysis provides actionable budget-tier guidance for production deployment.

6.1.4 Unifying the Evaluation Metrics for Practice

This dissertation introduces multiple complementary metrics because enterprise readiness is multi-dimensional: a model can be accurate yet operationally unreliable, cost-prohibitive, or unsuitable under latency and privacy constraints. To reduce practitioner burden, we group the metrics into three layers and recommend a minimal decision workflow:

- **Task quality metrics** (e.g., F1, COMET) quantify whether the system produces correct outputs for the target task.
- **Reliability metrics** (e.g., RAP, VCR, ECR@1, ASR, TCR) quantify whether outputs are usable, complete, and robust under operational conditions (including tool use and constrained formats).
- **Efficiency metrics** (e.g., energy per task, throughput such as CPS) quantify whether the system meets latency, cost, and resource constraints.

In practice, we recommend first screening candidate systems by *reliability* (to eliminate configurations that frequently fail, repeat, or violate output constraints), then optimizing *quality* subject to the reliability floor, and finally selecting a configuration that satisfies *efficiency* requirements for the intended deployment tier (cloud, on-prem, or edge).

6.2 Key Findings

Several cross-cutting themes emerge from this research:

6.2.1 Finding 1: Open-Source Viability

Open-source LLMs, when properly optimized, achieve performance competitive with—or exceeding—proprietary alternatives across diverse enterprise tasks.

This finding has significant implications for enterprise AI adoption:

- Orca-2-13b achieves 99.3% RAGAS score in RAG applications (Chapter 3), matching GPT-4
- Qwen2.5-72B surpasses GPT-4o on logical reasoning (80.88% vs. 80.36% F1) with only 10-shot prompting (Chapter 5)
- qwen2.5:32b achieves 99.90% success rate on invoice reconciliation (Chapter 4), outperforming all proprietary cloud APIs
- qwen2.5:32b achieves 100.0% ASR matching GPT-4.1 on payment workflows, while enabling privacy-preserving edge deployment (Chapter 4)
- GPT-OSS:120b achieves 92.10 F1 on entity matching, outperforming GPT-4.1 (90.36) and GPT-4o (89.58) at 20× lower cost (Chapter 5)

Organizations can leverage open-source models to reduce costs, maintain data privacy through edge deployment, and avoid vendor lock-in while achieving comparable or superior performance.

6.2.2 Finding 2: Architecture Over Scale

Larger models do not always outperform smaller counterparts; architectural design and optimization strategies often matter more than parameter count.

Counter-intuitive findings across multiple studies demonstrate this principle:

- DeepSeek-R1:32b (81.7% F1) substantially outperforms DeepSeek-R1:70b (74.2% F1) on sentiment analysis (Chapter 5)
- 32B Qwen2.5-based distillation outperforms 70B Llama-based variant by 6.69 percentage points (Chapter 5)

- qwen3:8b achieves highest performance (84.8% F1) in its family, surpassing 14B, 30B, and 32B variants (Chapter 5)
- Medium-sized models (7B–32B) provide optimal accuracy-efficiency trade-offs for edge deployment (Chapter 5)
- Orca-2-13b matches or exceeds GPT-4 performance in RAG applications despite being significantly smaller (Chapter 3)

Strategic model selection based on task requirements and deployment constraints is essential, as the largest models may not justify additional resource costs.

6.2.3 Finding 3: Task Complexity Determines Reasoning Effectiveness

Reasoning capabilities improve performance only for sufficiently complex tasks; simpler tasks often suffer degradation when reasoning is applied.

Our comprehensive evaluation across 504 configurations reveals systematic task-complexity patterns:

- Binary classification (IMDB): 100% failure rate for reasoning models, -4.8 ± 6.3 pp average degradation
- 5-class sentiment (Amazon): 80% failure rate, -3.6 ± 2.2 pp average degradation
- 27-class emotion (GoEmotions): Only 14% failure rate for thinking modes, $+2.0 \pm 1.0$ pp average improvement

This task-complexity hypothesis provides principled deployment guidance: reserve reasoning for complex multi-class problems where accuracy gains justify 2.1×–54× computational overhead, and leverage base models for simpler classification tasks. Moreover, entity matching evaluation across 46 configurations reveals that reasoning effectiveness is also *family-dependent*: low reasoning is optimal for

GPT-5 (93.14 vs. 92.14 F1 for high), whereas high reasoning is critical for open-weight models (GPT-OSS:120b gains +1.99 F1), and Claude models exhibit extreme asymmetric sensitivity (+10.13 F1 for Sonnet 4.5, -0.25 for Opus 4.5).

6.2.4 Finding 4: Few-Shot Learning Superiority

Few-shot learning yields more robust and consistent improvements than reasoning activation across all task complexities.

Regardless of whether models are reasoning/thinking-based or base/non-thinking:

- Few-shot learning improves over zero-shot in the vast majority of configurations
- DeepSeek-R1 achieves 99.31% accuracy on binary sentiment with just 5 shots, an $8\times$ improvement in few-shot efficiency over GPT-4o
- Performance gains vary by model architecture and task complexity, but the direction is consistently positive
- Few-shot prompting enables rapid adaptation without fine-tuning overhead

Few-shot prompting provides a more reliable enhancement strategy than reasoning mode activation for sentiment analysis and related classification tasks.

6.2.5 Finding 5: Novel Metrics Address Evaluation Gaps

Traditional accuracy metrics are insufficient for enterprise deployment; operational reliability, output quality, and efficiency require dedicated measurement frameworks.

This dissertation introduces seven complementary metrics addressing distinct evaluation dimensions:

These metrics capture operational dimensions essential for enterprise deployment: output quality (RAP), classification reliability (VCR), agentic workflow success (ASR, ECR@1), reasoning efficiency (RI), and translation completeness (TCR, CPS).

Table 6.1: Summary of Novel Evaluation Metrics Introduced

Metric	Purpose	Chapter
RAP	Repetition-aware model evaluation	3
VCR	Valid structured output proportion	5
ECR@1	First-attempt operational reliability	4
ASR	Trajectory-level workflow fidelity	4
RI	Reasoning-explainability trade-offs	5
TCR	Translation reliability	5
CPS	Translation efficiency	5

6.2.6 Finding 6: Modular Multi-Agent Architectures Enable Reliable Automation

Well-designed agent systems with clear specialization, sequential coordination, and appropriate human oversight achieve high reliability even with complex multi-step processes.

Key architectural principles validated across seven multi-agent systems:

- Modular agent specialization with clearly defined responsibilities
- Hierarchical coordination enabling modular task execution
- Information determinism: storing critical data in state variables, not LLM outputs
- Structured handoff protocols for inter-agent communication
- Human oversight mechanisms for high-impact decisions

The structured error taxonomy and ASR metric enable systematic diagnosis and improvement of multi-agent system reliability.

6.3 Practical Implications

For enterprise practitioners, this research provides actionable guidance across the LLM deployment lifecycle:

6.3.1 Model Selection

1. Consider mid-tier open-source models (7B–32B) for cost-effective deployments with production-ready reliability
2. Evaluate task-specific performance rather than relying on general benchmarks
3. Match reasoning capabilities to task complexity: use base models for simple classification, reserve reasoning for complex multi-class problems
4. Use proprietary APIs as comparison baselines rather than default choices
5. For agentic workflows: large models ($\geq$32B) provide production-ready reliability (ASR $>$93%); small models ($<$4B) are unsuitable for multi-step pipelines

6.3.2 Optimization Strategy

1. Invest in systematic hyperparameter tuning using metrics like RAP for RAG systems
2. Apply targeted fine-tuning for complex reasoning tasks—it consistently outperforms few-shot prompting for deployment reliability (VCR improvements from 1.17% to 100%)
3. Employ chat templates rather than generic prompts for RAG applications to reduce repetition
4. Align optimization metrics with downstream objectives (e.g., F1 tuning for betting produces 2–3$\times$ higher returns than accuracy tuning)
5. Leverage few-shot learning as a robust baseline enhancement across all task complexities

6.3.3 Evaluation Framework

1. Incorporate RAP alongside traditional accuracy measures for RAG systems

2. Use VCR to assess output reliability for structured classification tasks

3. Measure ECR@1 for agentic systems to capture operational reliability and retry costs

4. Adopt ASR or similar stepwise metrics to diagnose failure modes beyond binary success/failure

5. Consider Pareto frontier analysis for cost-performance trade-off decisions

6. Use LLM-as-Judge approaches for scalable evaluation with appropriate cost-reliability trade-offs (175× cost variation observed)

6.3.4 Architecture Design

1. Adopt modular multi-agent designs with clearly defined agent responsibilities for complex workflows

2. Implement hierarchical coordination for payment and financial processing systems

3. Store critical information in deterministic state variables, not LLM outputs, to prevent hallucination of sensitive data

4. Define structured handoff protocols for inter-agent communication

5. Incorporate human oversight mechanisms for high-impact decisions

6. Budget 5–15% overhead for reliability failures and implement appropriate retry logic

6.3.5 Infrastructure Planning

1. Consumer-grade GPUs (RTX 4090, 24GB) support models up to 14B parameters for reasoning tasks

2. Professional workstations (48GB+ VRAM) enable 32B–70B model deployment

3. Consider Ollama's Q4_K_M quantization to extend hardware capabilities with minimal performance loss

4. WSL provides up to 21× throughput improvements over native Windows for smaller models

5. Apple Silicon (M3/M4 Max) provides competitive performance for edge scenarios

6. Small models (1.5B–3B) achieve highest performance-to-power ratios (0.161–0.163 %/J) for energy-constrained deployments

6.3.6 Limitations and Threats to Validity

While the proposed metrics, architectures, and deployment guidelines are empirically validated across multiple tasks, models, and hardware configurations, several limitations should be noted.

Scope and Generalizability

The experiments cover a broad range of enterprise-relevant tasks (e.g., retrieval-augmented generation, agentic workflows, sentiment classification, machine translation, and domain-specific analytics). However, the conclusions may not fully generalize to domains with substantially different constraints or data characteristics (e.g., highly specialized biomedical text, low-resource languages, or long-context legal discovery pipelines). In particular, domain shift can affect both retrieval quality (for RAG) and downstream decision reliability (for agentic systems), and thus deployment recommendations should be revalidated when the target domain differs materially from the evaluation settings.

Dataset and Benchmark Bias

Several evaluations rely on established public benchmarks that are primarily English-centric and may not represent the distribution of enterprise documents encountered in

production (e.g., internal memos, proprietary templates, multilingual mixed-format documents). This creates potential bias in reported performance and reliability. Future work should extend evaluation to multilingual, format-diverse, and organization-specific corpora to better quantify robustness under real enterprise data distributions.

Model and Version Volatility

The LLM landscape evolves rapidly, with frequent model releases, parameter updates, and changes in inference stacks. As a result, absolute performance numbers may drift over time, and direct replication can be challenging when (i) model checkpoints are updated, (ii) system prompts or safety layers change, or (iii) inference frameworks alter kernel-level optimizations. To mitigate this, we report configurations and settings where possible and emphasize *comparative* and *principled* findings (e.g., task-dependent reasoning effectiveness, reliability failure modes, and trade-offs between accuracy, cost, and energy) that are expected to remain more stable than any single point estimate.

Reproducibility Constraints for Proprietary Models

Some baselines involve proprietary models accessed via hosted APIs. These systems may change without notice, and full transparency into training data, decoding strategies, and infrastructure-level optimizations is unavailable. Consequently, comparisons between open-weight and proprietary models may be affected by unobserved factors beyond the researcher's control. Where feasible, we triangulate findings using multiple model families and focus on reproducible open-weight configurations to strengthen external validity.

Operational Assumptions in Agentic Systems

Agentic evaluations necessarily assume specific tool interfaces, orchestration frameworks, and workflow definitions. Real deployments can introduce additional failure

modes—for example, authentication issues, rate limits, upstream service outages, tool schema changes, or human-in-the-loop delays. Although the proposed error taxonomy and reliability metrics capture core failure patterns, deployment teams should treat them as a foundation and extend them to include infrastructure- and organization-specific operational risks.

6.4 Future Research Directions

Based on the findings and limitations of this research, several promising directions for future work emerge:

6.4.1 Advanced RAG Architectures

Future research should explore RAG systems that dynamically adjust retrieval strategies:

- Query-dependent chunk selection and filtering
- Iterative retrieval with progressive refinement
- Multi-stage retrieval combining sparse and dense methods
- Self-evaluating RAG with automatic quality assessment using RAP-like metrics
- Integration of repetition penalty optimization into the retrieval pipeline

6.4.2 Adaptive Reasoning Architectures

Our task-complexity findings motivate research into systems that dynamically adjust reasoning intensity:

- Hybrid architectures that trigger costly reasoning only when ambiguity warrants it
- Query-complexity detection to route requests to appropriate reasoning modes

- Adaptive thinking budgets that balance accuracy gains against computational overhead
- Task-specific distillation objectives tailored to affective reasoning rather than generic logical skills
- Empirical investigation of the task-complexity threshold where reasoning transitions from harmful to beneficial, potentially enabling automated routing decisions

6.4.3 Enhanced Multi-Agent Coordination

Building on our multi-agent foundations, future work should investigate:

- Dynamic agent allocation based on task complexity and ASR predictions
- Learning-based coordination protocols optimized via the structured error taxonomy
- Fault-tolerant architectures with graceful degradation informed by ECR@1 patterns
- Hybrid human-agent workflows with adaptive automation levels
- Cross-domain transfer of agentic patterns from payments to other financial processes

6.4.4 Multimodal Enterprise Applications

The intersection of agentic AI with emerging modalities presents opportunities:

- Vision-language agents for document understanding extending invoice reconciliation
- Speech-enabled enterprise assistants with reasoning capabilities
- Video analysis for quality control and monitoring
- Multimodal sentiment analysis combining text and visual signals

6.4.5 Continual Learning and Adaptation

Our temporal robustness findings (10–17 pp degradation on contemporary data) motivate research into models that adapt over time:

- Techniques for updating models with new domain knowledge
- Methods for detecting and handling distribution shift
- Efficient fine-tuning for rapid customization
- Knowledge retention strategies to prevent catastrophic forgetting
- Periodic RAP recalibration as language patterns evolve

6.4.6 Sustainable AI Deployment

Our energy efficiency findings (performance-to-power ratios varying from 0.05 to 0.163 %/J) motivate research into:

- Energy-efficient inference techniques optimized for edge deployment
- Carbon-aware model selection and scheduling
- Lifecycle assessment of AI system deployment
- Trade-offs between accuracy, reasoning intensity, and environmental impact

6.5 Concluding Remarks

The rapid advancement of Large Language Models presents both opportunities and challenges for enterprise adoption. This dissertation has demonstrated that systematic optimization—spanning model selection, hyperparameter tuning, prompting strategies, fine-tuning, architectural design, and evaluation methodology—enables effective deployment of LLMs for diverse business applications.

The research establishes that open-source models, when properly optimized, can achieve performance competitive with proprietary alternatives while offering advantages in customization, privacy, and cost. Across seventeen peer-reviewed publications at premier venues, we demonstrated this principle in RAG systems (Orca-2-13b matching GPT-4), logical reasoning (Qwen2.5-72B surpassing GPT-4o), multi-agent automation (qwen2.5:32b outperforming cloud APIs), entity matching (GPT-OSS:120b outperforming GPT-4.1 at 20× lower cost), and payment workflow fidelity (qwen2.5:32b achieving 100.0% ASR matching GPT-4.1).

A central contribution is the recognition that reasoning effectiveness is task-dependent. The comprehensive evaluation across 504 configurations challenges the prevailing narrative that enhanced reasoning universally improves LLM performance. For simple binary classification, reasoning introduces noise rather than clarity; for complex 27-class emotion recognition, reasoning provides measurable benefits. This task-complexity hypothesis offers principled guidance for practitioners navigating the expanding landscape of reasoning-capable models.

A distinguishing methodological contribution is the development of seven novel evaluation metrics that capture operational dimensions invisible to traditional accuracy-focused assessment. These metrics—RAP, VCR, ECR@1, ASR, RI, TCR, and CPS—provide practitioners with concrete, measurable targets for deployment readiness across the three research pillars. Table 6.1 summarizes these contributions, demonstrating how the dissertation addresses evaluation gaps across RAG systems, agentic workflows, and enterprise deployment scenarios.

Multi-agent architectures represent a promising paradigm for automating complex workflows that traditionally require human coordination. Our invoice reconciliation system demonstrates that edge-deployed open-source models can outperform cloud APIs while maintaining privacy and reducing operational costs. The structured error taxonomy and ASR metric provide systematic tools for diagnosing and improving multi-agent reliability.

Perhaps most importantly, this research emphasizes that thoughtful engineering

of the entire deployment pipeline—not merely model selection—determines success in enterprise AI applications. The contributions, guidelines, and insights presented here provide a foundation for responsible and effective integration of generative AI into enterprise operations.

As LLM capabilities continue to advance, the optimization principles established in this dissertation will require ongoing refinement. However, the fundamental insight remains: bridging the gap between AI potential and practical deployment requires systematic attention to evaluation, optimization, and architecture across the entire system lifecycle. The demonstrated success of open-source models on consumer hardware significantly reduces barriers to enterprise AI adoption, enabling organizations of all sizes to deploy sophisticated AI systems while maintaining control over their data, costs, and operational requirements.

Bibliography

[1] D. B. Acharya, K. Kuppan, and B. Divya. Agentic ai: Autonomous intelligence for complex goals–a comprehensive survey. *IEEE Access*, 2025.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[3] M. AI. Magistral-small-2506: 24b parameter reasoning model, 2025. URL `https://huggingface.co/mistralai/Magistral-Small-2506`. Official model card for Magistral Small 24B parameter reasoning model.

[4] T. Al-Hindi, P. Nakov, and T. Baldwin. Multitask learning for logical reasoning with pre-trained models. *arXiv preprint arXiv:2303.06279*, 2023.

[5] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang. Automated unit test improvement using large language models at meta. *arXiv preprint arXiv:2402.09171*, February 2024.

[6] Anthropic. Claude 3 model card. `https://www.anthropic.com/claude`, 2024.

[7] Apple. Mlx: An array framework for apple silicon. `https://github.com/ml-explore/mlx`, 2024.

[8] C. Artho, K. Suzaki, E. Di Nitto, Y. Tanabe, and M. Hagiya. Crap4j: Change risk anti-patterns detection for java. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 2*, ICSE '10, pages 499–500, New York, NY, USA, 2010. ACM.

[9] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.

[10] N. Barlaug and J. A. Gulla. Neural networks for entity matching: A survey. *ACM Transactions on Knowledge Discovery from Data*, 15(3):1–37, 2021.

[11] N. Batchelder. Coverage.py: Code coverage measurement for python. `https://coverage.readthedocs.io`, 2024. Accessed: 2024.

[12] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

[13] O. Benjelloun, H. Garcia-Molina, D. Menestrina, Q. Su, S. Euijong Whang, and J. Widom. Swoosh: A generic approach to entity resolution. volume 18, pages 255–276, 2009.

[14] M. Bilenko, R. J. Mooney, W. W. Cohen, P. Ravikumar, and S. E. Fienberg. Adaptive name matching in information integration. *IEEE Intelligent Systems*, 18(5):16–23, 2003.

[15] O. Binette and R. C. Steorts. (almost) all of entity resolution. *Science Advances*, 8 (12), 2022.

[16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[17] R. Bunker and T. Susnjak. Machine learning for sports prediction: A comprehensive review and comparison of methods. *IEEE Access*, 10:1–24, 2022.

[18] J. Carmona, B. van Dongen, A. Solti, and M. Weidlich. *Conformance Checking: Relating Processes and Models*. Springer, 2018. ISBN 978-3-319-99413-0.

[19] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, et al. Why do multi-agent llm systems fail?, 2025.

[20] K. Chawla, T. Khot, A. Sabharwal, and W.-t. Yih. Ai agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

[21] Z. Chen, A. Bhatia, S. L. Zhang, S. Choi, M. Saar-Tsechansky, and M. Ghassemi. Standard benchmarks fail – auditing LLM agents in finance must prioritize risk. *arXiv preprint arXiv:2502.15865*, 2025.

[22] K. T. Chitty-Venkata, S. Raskar, B. Kale, F. Ferdaus, A. Tanikanti, K. Raffenetti, V. Taylor, M. Emani, and V. Vishwanath. Llm-inference-bench: Inference benchmarking of large language models on ai accelerators. *arXiv preprint arXiv:2411.00136*, 2024.

[23] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 30, 2017.

[24] J. K. Chua, C. H. Tse, D. Huang, and Z. Wang. ConvPayMAS: Conversational payment multi-agent system with agent-to-agent protocol and three-mandate verification. In *Demonstration Track, Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026)*, Paphos, Cyprus, May 2026. Accepted for publication.

[25] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[26] M. . Company. The state of ai in 2024: Generative ai adoption spikes and starts to generate value. `https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai`, 2024.

[27] CrewAI. Crewai: Framework for orchestrating role-playing autonomous ai agents. `https://www.crewai.com/`, 2024.

[28] D. Dahiphale, N. Madiraju, J. Lin, R. Karve, M. Agrawal, A. Modwal, R. Balakrishnan, S. Shah, G. Kaushal, P. Mandawat, et al. Enhancing trust and safety in digital payments: An llm-powered approach. In *2024 IEEE International Conference on Big Data (BigData)*, pages 4854–4863. IEEE, 2024.

[29] X. Dai. Multiple nvidia gpus or apple silicon for large language model inference? GitHub Repository, 2024. URL `https://github.com/XiongjieDai/GPU-Benchmarks-on-LLM-Inference`. Comprehensive benchmarking using llama.cpp comparing NVIDIA GPUs vs Apple Silicon M1-M3.

[30] M. Daly, M. Hind, W. Geyer, A. Rawat, K. R. Varshney, and P. Sattigeri. Granite guardian 3.0: Efficient and safe ai models for enterprise applications. *arXiv preprint arXiv:2412.07724*, 2024. URL `https://arxiv.org/abs/2412.07724`.

[31] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

[32] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.

[33] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2023.

[34] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2019.

[35] N. Dhar, B. Deng, D. Lo, X. Wu, L. Zhao, and K. Suo. An empirical analysis and resource footprint study of deploying large language models on edge devices. In *ACMSE*, 2024.

[36] V. Dibia, A. Fourney, G. Bansal, A. Awadallah, D. Burger, C. Wang, E. Zhu, and R. W. White. Autogen studio: A no-code developer tool for building and debugging multi-agent systems. *arXiv preprint arXiv:2408.15247*, 2024. URL `https://arxiv.org/abs/2408.15247`.

[37] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

[38] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.

[39] M. Fan, X. Han, J. F. Chai, Chengliang, N. Tang, G. Li, and X. Du. Cost-effective in-context learning for entity resolution: A design space exploration. In *Proceedings of the 40th IEEE International Conference on Data Engineering*, pages 3696–3709, 2024.

[40] I. P. Fellegi and A. B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

[41] D. Feng et al. Profiling apple silicon performance for ml training. *arXiv preprint arXiv:2501.14925*, 2025. URL https://arxiv.org/abs/2501.14925.

[42] E. Frantar and D. Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. *arXiv preprint arXiv:2301.00774*, 2023.

[43] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[44] Z. Fu, W. Lam, A. M.-C. So, and B. Shi. A theoretical analysis of the repetition problem in text generation. *arXiv preprint arXiv:2012.14660*, 2020.

[45] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[46] G. Gerganov. llama.cpp: Port of facebook's llama model in c/c++. https://github.com/ggerganov/llama.cpp, 2024.

[47] L. Getoor and A. Machanavajjhala. Entity resolution: Theory, practice & open challenges. *Proceedings of the VLDB Endowment*, 5(12):2018–2019, 2012.

[48] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2014.

[49] Google. Agent2agent protocol (A2A): An open protocol for agent interoperability. https://google.github.io/A2A/, 2025. Accessed: 2026-02-15.

[50] Google. Powering AI commerce with the new agent payments protocol (AP2). https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol, 2025. Accessed: 2026-03-15.

[51] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[52] Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. *ICLR*, 2023.

[53] Y. Guan, D. Wang, Z. Chu, S. Wang, F. Ni, R. Song, and C. Zhuang. Intelligent agents with llm-based process automation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, page 5018–5027, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704901. doi: 10.1145/3637528.3671646.

[54] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[55] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang. Large language model based multi-agents: a survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, IJCAI '24, 2024. ISBN 978-1-956792-04-1. doi: 10.24963/ijcai.2024/890.

[56] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

[57] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[58] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[59] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[60] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[61] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, et al. Data interpreter: An llm agent for data science. *arXiv preprint arXiv:2402.18679*, 2024.

[62] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.

[63] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799, 2019.

[64] C.-Y. Hsieh, C.-L. Li, C.-k. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *ACL*, 2023.

[65] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022.

[66] D. Huang and Z. Wang. Evaluation of orca 2 against other llms for retrieval augmented generation. In *Trends and Applications in Knowledge Discovery and Data Mining*, pages 3–19, Singapore, 2024. Springer Nature Singapore. ISBN 978-981-97-2650-9.

[67] D. Huang and Z. Wang. Explainable sentiment analysis with DeepSeek-R1: Performance, efficiency, and few-shot learning. *IEEE Intelligent Systems*, 40(6):52–63, 2025. doi: 10.1109/MIS.2025.3614967.

[68] D. Huang and Z. Wang. LLMs at the edge: Performance and efficiency evaluation with ollama on diverse hardware. In *Proceedings of the 2025 International Joint Conference on Neural Networks (IJCNN), Rome, Italy*, pages 1–8, 2025.

[69] D. Huang and Z. Wang. Logical reasoning with LLMs via few-shot prompting and fine-tuning: A case study on turtle soup puzzles. In *2025 IEEE Symposium on Computational Intelligence in Natural Language Processing and Social Media (CI-NLPSoMe Companion)*, pages 1–5, 2025. doi: 10.1109/CI-NLPSoMeCompanion65206.2025.10977921.

[70] D. Huang and Z. Wang. Optimizing chinese-to-english translation using large language models. In *2025 IEEE Symposium on Computational Intelligence in Natural Language Processing and Social Media (CI-NLPSoMe)*, pages 1–7, 2025. doi: 10.1109/CI-NLP SoMe64976.2025.10970768.

[71] D. Huang and Z. Wang. Task complexity matters: An empirical study of reasoning in LLMs for sentiment analysis. In *Proceedings of the 30th Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD 2026)*, 2026. Submitted, November 2025.

[72] D. Huang, Z. Hu, and Z. Wang. Performance analysis of llama 2 among other llms. In *2024 IEEE Conference on Artificial Intelligence (CAI)*, pages 1081–1085. IEEE, 2024.

[73] D. Huang, G. Malwe, R. Varshney, A. Tse, and Z. Wang. Scalable invoice reconciliation for smes: Edge-deployed llms in multi-agent systems. In *2025 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, pages 1–6. IEEE, 2025.

[74] D. Huang, T.-S. Nguyen, F. Liausvia, and Z. Wang. Rap: A metric for balancing repetition and performance in open-source large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1479–1496, 2025.

[75] D. HUANG, N. TEO, A. CHEKURU, E. DEUTSCHMAN, and Z. WANG. Deploying reasoning LLMs for sentiment analysis: Architecture trade-offs on consumer hardware. In *IEEE ICDMW 2025*, 2025.

[76] D. Huang, S. Chew, A. Dutkiewicz, and Z. Wang. LLM as a judge for scalable test coverage evaluation: Accuracy, operational reliability, and cost. In *AAAI 2026 Workshop on Next Gen Code Development with Collaborative AI Agents*, 2026.

[77] D. Huang, S. Chew, and Z. Wang. CMAS4G2: A locally deployed cooperative multi-agent system for gherkin acceptance test generation. In *Trends and Applications in Knowledge Discovery and Data Mining (PAKDD 2026 Workshops)*, 2026. Accepted for publication.

[78] D. Huang, J. K. Chua, and Z. Wang. Beyond task success: Measuring workflow fidelity in LLM-based agentic payment systems. In *Proceedings of the 2nd International Workshop on AI for Digital Finance (AI4DF), PAKDD 2026 Workshops*, 2026. Accepted for publication.

[79] D. Huang, J. K. Chua, and Z. Wang. Agentic success rate: Measuring workflow fidelity in hierarchical multi-agent payment systems. *Journal of Autonomous Agents and Multi-Agent Systems (JAAMAS)*, 2026. Under review.

[80] D. Huang, J. K. Chua, and Z. Wang. A novel hierarchical multi-agent system for payments using LLMs. In *PAKDD 2026*, 2026. Submitted, November 2025.

[81] D. Huang, G. Malwe, and Z. Wang. When agents fail to act: A diagnostic framework for tool invocation reliability in multi-agent llm systems. In *ICAIBD 2026*, 2026. Submitted, November 2025.

[82] D. Huang, P. Pai, and Z. Wang. Entity matching with LLMs at scale: Accuracy, cost, and reasoning effects. In *Trends and Applications in Knowledge Discovery and Data Mining (PAKDD 2026 Workshops)*, 2026. Accepted for publication.

[83] J. Huang and K. C.-C. Chang. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403*, 2023.

[84] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.

[85] P. Hübner and A. Hu. Apple vs. oranges: Evaluating the apple silicon m-series socs for hpc performance and efficiency. *arXiv preprint arXiv:2502.05317*, 2025. URL `https://arxiv.org/abs/2502.05317`.

[86] IBM. Ibm granite 3.3 documentation, 2025. URL `https://www.ibm.com/granite/docs/models/granite/`. Official IBM Granite 3.3 model documentation and technical specifications.

[87] L. Inc. Langgraph documentation, 2024. URL `https://langchain-ai.github.io/langgraph/`. Framework for building stateful, multi-actor applications with LLMs.

[88] L. Inc. Langgraph: Build resilient language agents as graphs, 2024. URL `https://github.com/langchain-ai/langgraph`. Low-level orchestration framework for building controllable agents.

[89] M. A. Islam, M. E. Ali, and M. R. Parvez. MapCoder: Multi-agent code generation for competitive problem solving. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4912–4944, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.269. URL `https://aclanthology.org/2024.acl-long.269/`.

[90] Istanbul Team. Istanbul: Javascript test coverage tool. `https://istanbul.js.org`, 2024. Accessed: 2024.

[91] S. K. Jeyakumar, A. A. Ahmad, and A. G. Gabriel. Advancing agentic systems: Dynamic task decomposition, tool integration and evaluation using novel metrics and dataset. In *NeurIPS 2024 Workshop on Open-World Agents*.

[92] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2024.

[93] Z. Jin, A. Lalwani, T. Vaidhya, X. Shen, Y. Ding, Z. Lyu, M. Sachan, R. Mihalcea, and B. Schölkopf. Logical fallacy detection. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 7180–7198, 2022.

[94] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[95] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[96] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan. Ai agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

[97] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher. Ctrl: A conditional transformer language model for controllable generation. *arXiv preprint arXiv:1909.05858*, 2019.

[98] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, and H.-K. Song. Shortened llama: A simple depth pruning for large language models. *ICLR*, 2024.

[99] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2014.

[100] P. Koehn. *Statistical Machine Translation*. Cambridge University Press, 2009.

[101] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35: 22199–22213, 2022.

[102] S. Kokane, M. Zhu, T. Awalgaonkar, J. Zhang, T. Hoang, A. Prabhakar, Z. Liu, T. Lan, L. Yang, J. Tan, et al. Spectool: A benchmark for characterizing errors in tool-use llms. *arXiv preprint arXiv:2411.13547*, 2024.

[103] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

[104] LangChain. Langchain: Building applications with llms through composability. `https://www.langchain.com/`, 2024.

[105] LangChain. Langgraph: Multi-actor applications with llms. `https://github.com/langchain-ai/langgraph`, 2024.

[106] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[107] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36, 2023.

[108] J. Li, M. Chen, Q. Wang, and Y. Zhang. Reasoning or overthinking: Evaluating large language models on financial sentiment analysis. *arXiv preprint arXiv:2506.04574*, 2025.

[109] J. Li et al. Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466*, 2024. URL `https://arxiv.org/abs/2410.04466`. Comprehensive survey including Apple Silicon and NVIDIA GPU comparisons.

[110] L. Li, J. Li, and H. Gao. Rule-based method for entity resolution. *IEEE Transactions on Knowledge and Data Engineering*, 27(1):250–263, 2015.

[111] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan. Deep entity matching with pre-trained language models. volume 14, pages 50–60, 2020.

[112] Z. Li, X. Xu, T. Shen, C. Xu, J.-C. Gu, Y. Lai, C. Tao, and S. Ma. Leveraging large language models for NLG evaluation: Advances and challenges. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16028–16045, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.896. URL `https://aclanthology.org/2024.emnlp-main.896/`.

[113] C.-Y. Lin. Rouge: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.

[114] C. D. Manning and H. Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[115] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[116] T. Meuser, L. Lovén, M. Bhuyan, S. G. Patil, S. Dustdar, A. Aral, S. Bayhan, C. Becker, E. De Lara, A. Y. Ding, et al. Revisiting edge ai: Opportunities and challenges. *IEEE Internet Computing*, 28(4):49–59, 2024.

[117] Microsoft Open Source. Handoffs — autogen, 2025. URL `https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/design-patterns/handoffs.html`. Accessed Sep. 18, 2025.

[118] Microsoft Open Source. Swarm — autogen, 2025. URL `https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/swarm.html`. Accessed Sep. 18, 2025.

[119] A. Mitra, L. Del Corro, S. Mahajan, A. Codas, C. Simoes, S. Agarwal, X. Chen, A. Razdaibiedina, E. Jones, K. Aggarwal, et al. Orca 2: Teaching small language models how to reason. *arXiv preprint arXiv:2311.11045*, 2023.

[120] S. M. Mohammad and P. D. Turney. Crowdsourcing a word–emotion association lexicon. *Computational Intelligence*, 29(3):436–465, 2013.

[121] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, K. Karahalios, and A. Kumar. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data*, pages 19–34, 2018.

[122] A. Narayan, I. Chami, L. Orr, and C. Ré. Can foundation models wrangle your data? volume 16, pages 738–746, 2022.

[123] NVIDIA Corporation. NVIDIA RTX A6000 Ada Generation GPU Datasheet. `https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/rtx-6000/proviz-print-rtx6000-datasheet-web-2504660.pdf`, 2024. Accessed: 2025-07-09.

[124] NVIDIA Corporation. GeForce RTX 4090 Graphics Card. `https://www.nvidia.com/en-sg/geforce/graphics-cards/40-series/rtx-4090/`, 2024. Accessed: 2025-07-09.

[125] Ollama. Ollama: Get up and running with large language models locally. https://ollama.ai, 2024.

[126] OpenAI. Learning to reason with llms. https://openai.com/index/learning-to-reason-with-llms/, 2024.

[127] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.

[128] B. Pang, L. Lee, and S. Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the ACL-02 Conference on Empirical Methods in Natural Language Processing*, pages 79–86, 2002.

[129] G. Papadakis, E. Ioannou, C. Niedermeier, P. Fankhauser, and W. Nejdl. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys*, 53 (2):1–42, 2021.

[130] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.

[131] PCI Security Standards Council. Ai principles: Securing the use of ai in payment environments. https://blog.pcisecuritystandards.org/ai-principles-securing-the-use-of-ai-in-payment-environments, 2025. Accessed: 2025-11-15.

[132] R. Peeters and C. Bizer. Using ChatGPT for entity matching. In *International Conference on AI in Finance*, 2023.

[133] R. Peeters and C. Bizer. Entity matching using large language models. In *Proceedings of the International Conference on Extending Database Technology*, 2025.

[134] P. Putta, E. Mills, N. Garg, S. Motwani, C. Finn, D. Garg, and R. Rafailov. Agent q: Advanced reasoning and learning for autonomous ai agents. *arXiv preprint arXiv:2408.07199*, 2024.

[135] B. Qu, Z. Wang, D. Yagi, Z. Xu, Y. Zhao, Y. Shan, and F. Zahradnik. LLM-enhanced self-evolving reinforcement learning for multi-step E-commerce payment fraud risk detection. In G. Rehm and Y. Li, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 92–103, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-288-6. URL https://aclanthology.org/2025.acl-industry.9/.

[136] Qwen Team. Qwen3: Think deeper, act faster. https://qwenlm.github.io/blog/qwen3/, 2025. Accessed: 2025-05-01.

[137] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving language understanding by generative pre-training. OpenAI Technical Report, 2018.

[138] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[139] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[140] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[141] A. Rastogi, A. Q. Jiang, A. Lo, G. Berrada, G. Lample, J. Rute, J. Barmentlo, K. Yadav, K. Khandelwal, K. R. Chandu, L. Blier, L. Saulnier, M. Dinot, M. Darrin, N. Gupta, R. Soletskyi, S. Vaze, T. L. Scao, Y. Wang, et al. Magistral. *arXiv preprint arXiv:2506.10910*, 2025. URL `https://arxiv.org/abs/2506.10910`. Technical report on Magistral, Mistral's first reasoning model series.

[142] R. Rei, J. G. De Souza, D. Alves, C. Zerva, A. C. Farinha, T. Glushkova, A. Lavie, L. Coheur, and A. F. Martins. Comet-22: Unbabel-ist 2022 submission for the metrics shared task. In *Proceedings of the Seventh Conference on Machine Translation (WMT)*, pages 578–585, 2022.

[143] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

[144] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.

[145] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia. Ares: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[146] H. D. Samuel, D. Huang, J. Tan, and Z. Wang. Integrating LLM sentiment analysis into machine learning for soccer betting. In *IEEE ICDMW 2025*, 2025.

[147] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36: 68539–68551, 2023.

[148] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.

[149] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.

[150] H. Su, W. Shi, J. Kasai, Y. Wang, Y. Hu, M. Ostendorf, W.-t. Yih, N. A. Smith, L. Zettlemoyer, and T. Yu. One embedder, any task: Instruction-finetuned text embeddings. *arXiv preprint arXiv:2212.09741*, 2023.

[151] Y. Tong, D. Li, S. Wang, Y. Wang, F. Teng, and J. Shang. Can llms learn from previous mistakes? investigating llms' errors to boost for reasoning. *arXiv preprint arXiv:2403.20046*, 2024.

[152] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[153] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen. Multi-agent collaboration mechanisms: A survey of llms, 2025.

[154] J. Tu, J. Fan, N. Tang, P. Wang, C. Chai, G. Li, R. Fan, and X. Du. Unicorn: A unified multi-tasking model for supporting matching tasks in data integration. In *Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data*, 2023.

[155] UCP Working Group. Unified commerce protocol (UCP) specification v2026-01-11. `https://docs.unifiedcommerce.org/`, 2026. Accessed: 2026-02-15.

[156] W. M. P. van der Aalst, A. Adriansyah, and B. F. van Dongen. Replaying history on process models for conformance checking and performance analysis. *WIREs Data Mining and Knowledge Discovery*, 2(2):182–192, 2012.

[157] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[158] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[159] T. Wang, X. Chen, H. Lin, X. Chen, X. Han, L. Sun, H. Wang, and Z. Zeng. Match, compare, or select? an investigation of large language models for entity matching. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 96–109, 2025.

[160] Z. Wang, Z. Hu, F. Li, S.-B. Ho, and E. Cambria. Learning-based stock trending prediction by incorporating technical indicators and social media sentiment. *Cognitive Computation*, 15(3):1092–1102, 2023.

[161] Z. Wang, D. Huang, J. Cui, X. Zhang, S.-B. Ho, and E. Cambria. A review of chinese sentiment analysis: Subjects, methods, and trends. *Artificial Intelligence Review*, 58 (75), 2025.

[162] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2022.

[163] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[164] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations*, 2020.

[165] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[166] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[167] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL `https://arxiv.org/abs/2412.15115`.

[168] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL `https://arxiv.org/abs/2505.09388`.

[169] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 201–206, 2016.

[170] J. Zeng, H. Liu, Z. Dai, X. Tang, C. Luo, S. Varshney, Z. Li, and Q. He. Cite before you speak: Enhancing context-response grounding in e-commerce conversational llm-agents, 2025.

[171] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*, 2020.

[172] W. Zhang, Y. Deng, B. Liu, S. J. Pan, and L. Bing. Sentiment analysis in the era of large language models: A reality check. *arXiv preprint arXiv:2305.15005*, 2024.

[173] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, et al. Siren's song in the ai ocean: A survey on hallucination in large language models. *Computational Linguistics*, pages 1–45, 2025.

[174] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[175] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2024.

[176] X. Zheng, Y. Zheng, L. Wei, X. Shen, et al. Towards efficient llm serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2024.

Appendix A

Supplementary Materials

This appendix provides detailed specifications, configurations, and reference materials supporting the experimental work presented in Chapters 3–5. Section A.1 catalogs all evaluated models; Section A.2 details hardware platforms; Section A.3 presents experimental configurations; Section A.4 illustrates multi-agent error patterns; Section A.5 documents prompt templates; and Section A.6 consolidates the seven novel evaluation metrics introduced throughout this dissertation.

A.1 Complete Model Specifications

Table A.1 provides detailed specifications for all models evaluated in this dissertation, organized by model family. Models are grouped into proprietary alternatives (used as comparison baselines), reasoning-specialized architectures (central to Chapter 5), and general-purpose open-source models (evaluated across all three research pillars).

Table A.1: Complete Model Specifications

Model	Parameters	Context	Architecture	Notes
Proprietary Models (Comparison Baselines)				
GPT-4	Undisclosed	128K	MoE (speculated)	OpenAI; RAG baseline
GPT-4o	Undisclosed	128K	Multimodal	OpenAI; sentiment/reasoning
GPT-4o-mini	Undisclosed	128K	Multimodal	OpenAI; cost-optimized
GPT-4.1	Undisclosed	1M	Multimodal	OpenAI, Apr 2025; agentic
GPT-4.1-nano	Undisclosed	1M	Multimodal	OpenAI; edge annotation
GPT-4.1-mini	Undisclosed	1M	Multimodal	OpenAI; LAJ framework
Claude Sonnet 4	Undisclosed	200K	Decoder-only	Anthropic; agentic eval
DeepSeek Family				

Continued on next page

Table A.1 – continued from previous page

Model	Parameters	Context	Architecture	Notes
DeepSeek-V3	671B	128K	MoE	Base model for DSR1
DeepSeek-R1 (Full)	671B	128K	MoE	RL-trained reasoning
DeepSeek-R1:8B	8B	64K	Decoder-only	Distilled from Llama3.1-8B
DeepSeek-R1:14B	14B	64K	Decoder-only	Distilled from Qwen2.5-14B
DeepSeek-R1:32B	32B	64K	Decoder-only	Distilled from Qwen2.5-32B
DeepSeek-R1:70B	70B	64K	Decoder-only	Distilled from Llama3.3-70B
		Qwen Family		
Qwen2-7B	7B	128K	Decoder-only	Translation experiments
Qwen2-72B	72B	128K	Decoder-only	Translation; fine-tuning
Qwen2.5-0.5B	0.5B	128K	Decoder-only	Edge deployment
Qwen2.5-1.5B	1.5B	128K	Decoder-only	Edge deployment
Qwen2.5-3B	3B	128K	Decoder-only	Edge deployment
Qwen2.5-7B	7B	128K	Decoder-only	18T tokens; RAG/edge
Qwen2.5-14B	14B	128K	Decoder-only	Base for DSR1-14B
Qwen2.5-32B	32B	128K	Decoder-only	Base for DSR1-32B; agentic
Qwen2.5-72B	72B	128K	Decoder-only	18T tokens; reasoning
Qwen2.5-Plus	API	128K	Decoder-only	Soccer betting sentiment
Qwen3-0.6B	0.6B	32K	Decoder-only	T/N modes
Qwen3-1.7B	1.7B	32K	Decoder-only	T/N modes
Qwen3-4B	4B	32K	Decoder-only	T/N modes
Qwen3-8B	8B	32K	Decoder-only	T/N modes; best Qwen3
Qwen3-14B	14B	32K	Decoder-only	T/N modes
Qwen3-30B	30B	32K	Decoder-only	T/N modes
Qwen3-32B	32B	32K	Decoder-only	T/N modes; annotation
		Llama Family		
Llama-2-7B	7B	4K	Decoder-only	2T tokens
Llama-2-13B	13B	4K	Decoder-only	2T tokens
Llama-2-70B	70B	4K	Decoder-only	2T tokens
Llama-3-8B	8B	8K	Decoder-only	15T tokens
Llama-3-70B	70B	8K	Decoder-only	15T tokens

Continued on next page

Table A.1 – continued from previous page

Model	Parameters	Context	Architecture	Notes
Llama-3.1-8B	8B	128K	Decoder-only	Base for DSR1-8B; edge
Llama-3.1-70B	70B	128K	Decoder-only	15T+ tokens; fine-tuning
Llama-3.2-1B	1B	128K	Decoder-only	Edge-optimized
Llama-3.2-3B	3B	128K	Decoder-only	Edge-optimized
Llama-3.2-vision:11B	11B	128K	Multimodal	Vision-language; edge
Llama-3.2-vision:90B	90B	128K	Multimodal	Vision-language
Llama-3.3-70B	70B	128K	Decoder-only	Base for DSR1-70B
Reasoning-Specialized Models				
Granite3.3-2B	2B	128K	Decoder-only	IBM; T/N modes
Granite3.3-8B	8B	128K	Decoder-only	IBM; T/N modes
Magistral-24B	24B	128K	Decoder-only	Mistral; RL reasoning
GPT-OSS:20B (low)	20B	32K	Decoder-only	Minimal reasoning (RI<0.5)
GPT-OSS:20B (medium)	20B	32K	Decoder-only	Balanced reasoning
GPT-OSS:20B (high)	20B	32K	Decoder-only	Intensive reasoning (RI>0.85)
GPT-OSS:120B (low)	120B	32K	Decoder-only	Minimal reasoning
GPT-OSS:120B (medium)	120B	32K	Decoder-only	Balanced; best F1 (86.2%)
GPT-OSS:120B (high)	120B	32K	Decoder-only	Intensive reasoning
Other Open-Source Models				
Orca-2-7B	7B	4K	Decoder-only	Synthetic training
Orca-2-13B	13B	4K	Decoder-only	Synthetic; RAG champion
Mistral-7B	7B	32K	Decoder-only	Sliding window attn.
Phi-3.5-mini	3.8B	128K	Decoder-only	Synthetic training
Gemma-2B	2B	8K	Decoder-only	Google; 2T tokens
Gemma-7B	7B	8K	Decoder-only	Google; 6T tokens
InternLM2.5-7B	7B	32K	Decoder-only	Shanghai AI Lab
InternLM2.5-7B-1M	7B	1M	Decoder-only	Extended context; reasoning
InternLM2.5-20B	20B	32K	Decoder-only	Shanghai AI Lab
Functionary-small	8B	32K	Function-calling	Tool-optimized; agentic
Functionary-medium	70B	32K	Function-calling	Tool-optimized; agentic

Notes:

- T/N modes = Thinking/Non-thinking modes with runtime-activated reasoning (Sec-

tion 5.3.3)

- MoE = Mixture of Experts architecture
- RI = Reasoning Intensity metric introduced in Section 5.2.2
- Context lengths represent maximum supported; actual usage may vary by deployment configuration
- Models marked "edge" were evaluated across consumer hardware platforms (Section 5.2.1)
- Models marked "agentic" were evaluated in multi-agent workflows (Chapter 4)

A.2 Hardware Specifications

Table A.2 details the six primary hardware platforms used for evaluation in this dissertation, spanning enterprise GPUs, consumer hardware, and edge devices. These platforms correspond to the edge deployment evaluation in Section 5.2.1.

A.2.1 Platform Selection Rationale

The hardware platforms were selected to represent the full spectrum of enterprise deployment scenarios addressed in Chapter 5:

- **H100:** Cloud/data center deployments requiring maximum throughput; primary platform for 504-configuration task complexity evaluation (Section 5.3.3)
- **RTX A6000 Ada:** Professional workstations for development and medium-scale inference; energy efficiency evaluation (Figure 5.1)
- **RTX 4090 Desktop:** Cost-effective on-premises deployment with strong performance; multi-agent invoice reconciliation (Chapter 4), achieving 99.90% success rate with qwen2.5:32b at 523 Joules per task
- **RTX 4090 Laptop:** Mobile professional use cases; 16GB VRAM constraint defines viable reasoning models (qwen3:8b optimal at 84.8% F1, 13GB VRAM)
- **Apple Silicon:** macOS-native deployment and unified memory advantages; competitive edge performance
- **Jetson AGX Orin:** Edge deployment for privacy-sensitive and latency-critical applications

A.2.2 Key Hardware Findings Summary

The following hardware-specific findings from Section 5.2.1 inform deployment decisions:

1. **WSL Performance:** Windows Subsystem for Linux delivers up to 21× throughput improvement over native Windows for smaller models (qwen2.5:0.5b: 16,933 vs. 816 tokens/s on RTX A6000)
2. **VRAM Constraints:** Consumer 16GB GPUs support models up to 14B parameters for reasoning tasks; 24GB enables 32B deployment

Table A.2: Hardware Platforms for Model Evaluation

Platform	**Specifications**
	Enterprise GPU
NVIDIA H100	96GB HBM3 VRAM, 16,896 CUDA cores, 528 Tensor cores, PCIe 5.0, 3.9 TB/s bandwidth, Ubuntu 24.04.1 LTS, CUDA 12.4, Ollama 0.11.4
	Professional Workstation
NVIDIA RTX A6000 Ada	48GB GDDR6 ECC VRAM, 18,176 CUDA cores, 568 Tensor cores, AMD Threadripper PRO 5975WX (32 cores), 256GB DDR4-3200 ECC, Ubuntu 22.04 LTS / Windows 11 Pro + WSL2
	Consumer Desktop
NVIDIA RTX 4090	24GB GDDR6X VRAM, 16,384 CUDA cores, 512 Tensor cores, Intel Core i9-13900K (24 cores), 64GB DDR5-5600, Ubuntu 22.04 LTS via WSL2, CUDA 12.1, Ollama 0.5.4–0.6.8
	Consumer Laptop
NVIDIA RTX 4090 Laptop	16GB GDDR6 VRAM, 9,728 CUDA cores, 304 Tensor cores, Intel Core i9-14900HX, 64GB DDR5, Windows 11 Pro / WSL2 Ubuntu 22.04 (**21× speedup over native Windows for small models**)
	Apple Silicon
M3 Max MacBook Pro	96GB Unified Memory, 16-core CPU, 40-core GPU, 16-core Neural Engine, macOS Sequoia 15.4, MLX 0.12.0, Ollama 0.5.4
M4 Max MacBook Pro	48GB Unified Memory, 16-core CPU, 40-core GPU, 16-core Neural Engine, macOS Sequoia 15.1, Ollama 0.5.4
	Edge Device
NVIDIA Jetson AGX Orin	64GB LPDDR5 Memory, 2,048 CUDA cores, 64 Tensor cores, 12-core Arm Cortex-A78AE CPU, up to 275 TOPS INT8, JetPack 6.0, Ubuntu 20.04

3. **Cross-Platform Stability:** Larger models exhibit increased consistency (F1 std: 0.07–0.23%) compared to small models (0.26–0.90%)
4. **Energy Efficiency:** Small models (1.5B–3B) achieve highest performance-to-power ratios (0.161–0.163 %/J); large models consume 1,283–1,876 Joules per inference

A.3 Detailed Experimental Configurations

A.3.1 RAG Experiments Configuration (Chapter 3)

```
# Embedding Model Configuration
embedding_model: hkunlp/instructor-large
embedding_dim: 768
max_seq_length: 512

# Retrieval Configuration
vector_store: FAISS
index_type: IVF_FLAT
num_clusters: 100
top_k_retrieval: 10

# Chunking Configuration
chunk_size: 500  # tokens
chunk_overlap: 50  # tokens
separator: "\n\n"

# Generation Configuration
temperature: 0.7
max_new_tokens: 1024
top_p: 0.95
repetition_penalty: 1.0-1.2  # varied in experiments

# RAP Metric Configuration
rap_exponent: 3  # cubic penalty validated via ablation
performance_metric: RAGAS_composite  # or task-specific F1

# Hardware
gpu: NVIDIA RTX 4090 (24GB) / A6000 (48GB)
precision: float16
```

A.3.2 Fine-Tuning Configuration (Chapters 3, 5)

```
# LoRA Configuration
lora_r: 64
lora_alpha: 128
lora_dropout: 0.1
target_modules: ["q_proj", "v_proj", "k_proj", "o_proj"]

# Training Configuration
learning_rate: 2e-4
```

```
batch_size: 8
gradient_accumulation_steps: 4
num_epochs: 3  # with early stopping at 0.2 epoch granularity
warmup_ratio: 0.1
weight_decay: 0.01
optimizer: AdamW

# QLoRA Specific (for models >= 20B)
quantization_bits: 4
bnb_4bit_compute_dtype: float16
bnb_4bit_quant_type: nf4
bnb_4bit_use_double_quant: True

# Early Stopping (per Section 5.3.1.5)
# Larger models peak earlier: 0.8 epochs (72B) vs 1.0 epochs (7B)
# VCR improves from 1.17% to 100% within 0.2 epochs
```

A.3.3 Multi-Agent System Configuration (Chapter 4)

```
# LangGraph Configuration
graph_type: StateGraph
checkpoint_store: MemorySaver
recursion_limit: 25

# Agent Configuration
max_iterations: 10
tool_timeout: 30  # seconds
retry_attempts: 3

# Database Configuration
db_type: SQLite
connection_mode: in-memory

# Email Agent Configuration
imap_server: imap.gmail.com
smtp_server: smtp.gmail.com
fetch_limit: 50

# Ollama Configuration (Edge Deployment)
ollama_version: 0.6.8
quantization: Q4_K_M
context_length: 8192

# HMASP Four-Level Hierarchy
hierarchy_levels: [orchestrator, domain_supervisors,
                   specialized_agents, tools]
shared_state_variables: true
decoupled_message_states: true
structured_handoff_protocols: true
```

```
# Success Metrics
target_ASR: 0.93  # minimum for production-ready
target_TSR: 0.97  # task success rate
ECR_retry_budget: 0.15  # 15% overhead for reliability failures
```

A.3.4 Task Complexity Experiments Configuration (Section 5.3.3)

The following configuration was used for the 504-configuration evaluation of reasoning effectiveness across sentiment analysis tasks.

```
# Model Families Evaluated (7 families)
deepseek_r1: [671B-full, 8B, 14B, 32B, 70B]
deepseek_v3: [671B]  # Base model comparison
llama_bases: [3.1-8B, 3.3-70B]
qwen25_bases: [14B, 32B]
qwen3: [4B, 8B, 14B, 30B, 32B]  # T/N modes
granite33: [2B, 8B]  # T/N modes
magistral: [24B]  # T/N modes

# Datasets (3 complexity levels)
imdb:
  task: binary_sentiment
  classes: [Positive, Negative]
  test_size: 1000
  complexity: simple
  expected_reasoning_benefit: negative  # -4.8 pp avg

amazon_reviews:
  task: 5class_sentiment
  classes: [Strongly Negative, Negative, Neutral,
            Positive, Strongly Positive]
  test_size: 1838  # human-annotated
  complexity: moderate
  expected_reasoning_benefit: mixed  # -3.6 pp avg

goemotions:
  task: 27class_emotion
  classes: [admiration, amusement, anger, annoyance,
            approval, caring, confusion, curiosity,
            desire, disappointment, disapproval, disgust,
            embarrassment, excitement, fear, gratitude,
            grief, joy, love, nervousness, optimism,
            pride, realization, relief, remorse,
            sadness, surprise]
  test_size: 1000
  complexity: high
  expected_reasoning_benefit: positive  # +2.0 pp avg
  note: single-label subset only
```

```
# Few-Shot Configuration (7 levels)
shot_levels: [0, 5, 10, 20, 30, 40, 50]
sampling_strategy: stratified
random_seed: 42

# Inference Configuration
ollama_version: 0.6.8
quantization: Q4_K_M  # MXFP4 for GPT-OSS
context_length: 8192
temperature: 0.0  # deterministic
max_tokens: 256

# Hardware
primary: NVIDIA H100 (96GB)
validation: NVIDIA RTX 4090 (24GB)
cross_platform: [M3 Max, RTX 4090 Laptop]  # temporal robustness

# Metrics Computed
performance: [F1, Accuracy, Weighted-F1]
efficiency: [mean_latency_per_sample, VRAM_usage, speed_ratio]
reasoning: [MRT, MOT, RI]  # Mean Reasoning/Output Tokens

# Pareto Frontier Analysis
pareto_axes: [F1_score, log_latency]
frontier_identification: non-dominated_sorting
```

Configuration Notes:

- Total configurations: 7 model families × 3 datasets × 7 shot levels × T/N variants = 504
- DeepSeek-R1 (full) and DeepSeek-V3 accessed via official API; all others via Ollama
- Thinking (T) and Non-thinking (N) modes evaluated separately for Qwen3, Granite3.3, and Magistral
- Stratified sampling ensures balanced class representation in few-shot exemplars
- Failure rates by dataset: IMDB 100%, Amazon 80%, GoEmotions 50% (reasoning underperforms)

A.3.5 Reasoning Deployment Experiments Configuration (Section 5.2.2)

```
# Model Families (17 variants)
deepseek_r1: [8B, 14B, 32B, 70B]
qwen3: [0.6B, 1.7B, 4B, 8B, 14B, 30B, 32B]
gpt_oss: [20B-low, 20B-medium, 20B-high,
          120B-low, 120B-medium, 120B-high]

# Dataset
amazon_reviews_5class:
  train_split: 70%
```

```
  test_split: 30%
  total_samples: 1838

# Temporal Robustness Evaluation
testset2:
  samples: 574
  collection_date: mid-2025
  annotation: manual

# Quantization
deepseek_qwen: Q4_K_M
gpt_oss: MXFP4

# Ollama Configuration
version: 0.11.4
context_length: 8192

# Reasoning Intensity (RI) Regimes
minimal: RI < 0.5      # GPT-OSS low configurations
balanced: 0.5 <= RI <= 0.85  # optimal regime
intensive: RI > 0.85   # DeepSeek-R1 models

# Consumer Hardware Constraint
rtx_4090_laptop_vram: 16GB
viable_models: [qwen3:8b (13GB), qwen3:4b (11GB),
                qwen3:1.7b (6.8GB)]
optimal_choice: qwen3:8b  # 84.8% F1, 8.35s inference
```

A.3.6 Soccer Betting Experiments Configuration (Section 5.4.2)

```
# Data Sources
match_statistics: API-Football
seasons: 2015-16 to 2023-24 (9 seasons)
betting_odds: Football-Data.co.uk (BetFair 1X2)
commentaries: BBC Sport (Python Selenium scraping)

# Data Partitioning
feature_selection: 2015-19
training: 2015-22
testing: 2022-23
betting_simulation: 2023-24

# LLM Sentiment Analysis
models: [DeepSeek-R1, GPT-4o-mini, GPT-4.1, Qwen2.5-Plus]
prompting: zero-shot
sentiment_scale: 1-5 (Strongly Negative to Strongly Positive)
feature_computation: home_sentiment - away_sentiment

# Machine Learning Models
classifiers: [LogisticRegression, SVC, RandomForest, XGBoost]
```

```
tuning_strategies: [accuracy-tuned, F1-tuned]
hyperparameter_search: GridSearchCV
cross_validation: 5-fold

# Betting Simulation
initial_bankroll: $10,000
kelly_fraction: 0.125  # one-eighth Kelly
bet_condition: positive_expected_value

# Key Results Reference
best_return: 279.48%  # Random Forest + GPT-4.1 (F1-tuned)
second_best: 247.97%  # XGBoost Baseline (F1-tuned)
third_best: 174.01%   # Logistic Regression + DeepSeek-R1 (F1-tuned)
```

A.4 Invoice Reconciliation Error Examples

The following examples illustrate the three outcome categories tracked by the multi-agent invoice reconciliation system (Chapter 4). These examples demonstrate the diagnostic value of the structured error taxonomy and ASR metric introduced in Section 4.4.

A.4.1 SUCCESS Example

```
Input: Reconcile invoice INV-2024-001 for customer ABC Corp
       with recent email communications.

Agent Trace:
1. Data Engineering Agent: Retrieved invoice details
   - Invoice ID: INV-2024-001
   - Amount: $15,000
   - Status: Unpaid
   - Customer: ABC Corp
   - ASR Step 1: PASS (tool initialization)
   - ASR Step 2: PASS (parameter handling)
   - ASR Step 3: PASS (query execution)

2. Email Agent: Processed 3 relevant emails
   - Email 1: Payment confirmation mention
   - Email 2: Wire transfer receipt attachment
   - Attachment OCR: Amount $15,000, Ref: INV-2024-001
   - ASR Step 4: PASS (email retrieval)
   - ASR Step 5: PASS (OCR extraction)

3. Reconciliation Agent: Match confirmed
   - Invoice amount matches payment
   - Reference numbers align
   - Status updated to: Paid
   - ASR Step 6: PASS (matching logic)
   - ASR Step 7: PASS (status update)
```

```
Output:
{
  "status": "SUCCESS",
  "invoice_id": "INV-2024-001",
  "amount": 15000,
  "payment_matched": true,
  "reconciliation_date": "2024-11-15T10:30:00Z",
  "asr_score": 1.0,  # 7/7 steps successful
  "energy_consumed_joules": 523
}
```

A.4.2 NO INVOICE Error Example

```
Input: Reconcile invoice INV-2024-999 for customer XYZ Inc.

Agent Trace:
1. Data Engineering Agent: Query executed
   - SQL: SELECT * FROM invoices WHERE id = 'INV-2024-999'
   - Result: No rows returned
   - ASR Step 1: PASS (tool initialization)
   - ASR Step 2: PASS (parameter handling)
   - ASR Step 3: PASS (query execution - valid empty result)

2. Email Agent: Searched for related emails
   - Found 0 matching emails
   - ASR Step 4: PASS (search completed)

3. Reconciliation Agent: Unable to proceed
   - No invoice data available
   - No email evidence found
   - ASR Step 5: PASS (correct termination)

Output:
{
  "status": "NO_INVOICE",
  "error": "Invoice INV-2024-999 not found in database",
  "invoice_id": "INV-2024-999",
  "suggestion": "Verify invoice ID or check if invoice exists",
  "asr_score": 1.0,  # All executed steps successful
  "error_category": "Data Availability"  # Not a system error
}
```

A.4.3 ERROR Example (Tool Invocation Failure)

```
Input: Process payment reconciliation for all outstanding invoices

Agent Trace:
1. Data Engineering Agent: Attempted batch query
   - Error: Query timeout after 30 seconds
   - Reason: Too many records requested
```

```
   - ASR Step 1: PASS (tool initialization)
   - ASR Step 2: FAIL (parameter handling - unbounded query)
   - Error Category: "Parameter Validation" (from structured error taxonomy

2. System: Agent retry triggered (attempt 2/3)
   - Error persisted
   - ASR Step 3: FAIL (retry unsuccessful)

3. Reconciliation Agent: Task failed
   - Unable to retrieve invoice data
   - Workflow terminated
   - ASR Step 4: PASS (graceful termination)

Output:
{
  "status": "ERROR",
  "error_type": "QueryTimeout",
  "error_category": "Parameter Validation",
  "message": "Unable to process batch request",
  "suggestion": "Process invoices individually or in smaller batches",
  "asr_score": 0.5,  # 2/4 steps successful
  "retry_count": 2,
  "ecr_impact": "First-attempt failure; retry overhead incurred"
}
```

A.4.4 Error Taxonomy Reference

The structured error taxonomy introduced in Chapter 4 classifies failures as follows:

1. **Tool Initialization Errors:** Agent fails to properly initialize required tools
2. **Parameter Validation Errors:** Invalid or unbounded parameters passed to tools
3. **Parameter Type Errors:** Type mismatches in tool arguments
4. **Missing Parameter Errors:** Required parameters omitted
5. **Tool Execution Errors:** Runtime failures during tool execution
6. **Timeout Errors:** Operations exceeding time limits
7. **Result Parsing Errors:** Failures in interpreting tool outputs
8. **Result Interpretation Errors:** Incorrect semantic understanding of results
9. **Handoff Errors:** Failures in inter-agent communication
10. **State Management Errors:** Inconsistencies in shared state variables
11. **Retry Logic Errors:** Inappropriate retry behavior
12. **Graceful Degradation Errors:** Failures in error recovery paths

A.5 Prompt Templates

A.5.1 RAG System Prompt (Chapter 3)

```
You are a helpful AI assistant specialized in answering questions
about PCI DSS (Payment Card Industry Data Security Standard).

Instructions:
1. Answer questions based ONLY on the provided context
2. If the context doesn't contain relevant information, say so
3. Cite specific sections when possible
4. Be concise but thorough
5. Avoid repeating information unnecessarily

Context:
{retrieved_context}

Question: {user_question}

Answer:
```

A.5.2 Sentiment Analysis System Prompt (Chapter 5)

```
You are an advanced sentiment analysis assistant. Your task is to
analyze text and provide a sentiment rating along with a brief
explanation. The sentiment rating should be based on a {scale}-point
scale: {sentiments}. Always respond with a JSON object containing
the sentiment and the explanation.

Example format:
- Input: [example text]
- Output: {"sentiment": "[label]", "explanation": "brief reason"}

{few_shot_examples}

- Input: {input_text}
- Output:
```

A.5.3 Logical Reasoning System Prompt (Section 5.3.1)

```
You are an expert in logical reasoning, specifically trained to
solve Turtle Soup (lateral thinking) puzzles.

Task: Classify the given question-answer pair into one of five
categories:
-  (Yes): The answer confirms the question
-  (No): The answer negates the question
-  (Unimportant): The detail is not relevant to the puzzle
-  (Wrong Question): The question is incorrectly framed
-  (Correct Answer): The question reveals the solution
```

```
Puzzle Background:
{puzzle_story}

Truth (hidden from players):
{truth}

Question: {question}

Your classification (respond with only the Chinese category):
```

A.5.4 Multi-Agent Data Engineering Prompt (Chapter 4)

```
You are a Data Engineering Agent responsible for database operations
in an invoice reconciliation workflow.

Available Tools:
1. sql_query_builder: Constructs SQL queries from natural language
2. sql_executor: Executes validated SQL queries
3. result_formatter: Formats query results as natural language

Database Schema:
{schema_description}

Instructions:
1. Analyze the user's request
2. Construct appropriate SQL query with bounded results
3. Validate query against schema
4. Execute and format results
5. Return natural language summary
6. Store critical data in state variables (not in message text)

Current State Variables:
{state_variables}

User Request: {request}
```

A.5.5 Soccer Betting Sentiment Prompt (Section 5.4.2)

```
You are an advanced sentiment analysis assistant.
Your task is to analyze match commentaries in the English Premier
League. The sentiment rating on a certain team should be based on
a 5-point scale: Strongly Positive (5), Positive (4), Neutral (3),
Negative (2), or Strongly Negative (1).

I would like you to predict the sentiment for each team for the
upcoming match based on what happened in the previous match. For
the upcoming match on [match_date], [home_team] is the home team
and [away_team] is the away team. I have found the BBC commentaries
of both teams' previous matches.
```

```
Rate the sentiment of each team on an output on a scale of 1-5.
Reply in a JSON format with the following keys: home_team_sentiment,
away_team_sentiment. Do not reply with anything else.

Below are the articles:

The article for [home_team]'s previous match on
[home_team_last_match_date]: [home_team_commentary]

The article for [away_team]'s previous match on
[away_team_last_match_date]: [away_team_commentary]
```

A.6 Novel Metrics Summary

Table A.3 provides a consolidated reference for all seven novel metrics introduced in this dissertation. These metrics address evaluation gaps identified across the three research pillars, capturing operational dimensions invisible to traditional accuracy-focused assessment.

Metric Design Principles:

1. **Complementarity:** Each metric captures a distinct evaluation dimension not addressed by traditional accuracy measures
2. **Actionability:** Metrics provide concrete targets for optimization (e.g., RPP tuning for RAP, fine-tuning duration for VCR)
3. **Operational Relevance:** Metrics reflect real-world deployment concerns including reliability, efficiency, and cost
4. **Cross-Pillar Integration:** Metrics inform decisions across RAG systems (RAP), agentic workflows (ASR, ECR@1), and enterprise deployment (VCR, RI, TCR, CPS)

Abbreviations: RR = Repetition Ratio; MRT = Mean Reasoning Tokens; MOT = Mean Output Tokens; RPP = Repetition Penalty Parameter.

Table A.3: Summary of Novel Evaluation Metrics

Metric	Definition	Purpose	Chapter	Key Finding
RAP	$P \times (1 - \text{RR})^3$ where P = performance, RR = repetition ratio	Penalize repetition in RAG outputs while preserving task performance	Ch. 3	Up to 93.1% repetition reduction with <4% performance loss
VCR	$\frac{\text{Valid Classifications}}{\text{Total Outputs}}$	Measure output reliability for structured classification tasks	Ch. 5	Fine-tuning improves VCR from 1.17% to 100% in 0.2 epochs
ECR@1	$\frac{\text{First-Attempt Successes}}{\text{Total Evaluations}}$	Capture operational reliability and retry cost overhead	Ch. 4	85.4%–100% range across models; 175× cost variation
ASR	$\frac{\sum_{i=1}^{N} s_i}{N}$ where $s_i \in \{0, 1\}$ for atomic step i	Fine-grained agentic reliability via N-step decomposition	Ch. 4	qwen3:32b achieves 98% ASR matching GPT-4.1
RI	$\frac{\text{MRT}}{\text{MRT} + \text{MOT}}$ where MRT = mean reasoning tokens and MOT = mean output tokens	Quantify reasoning-explainability trade-offs	Ch. 5	Three regimes: Minimal (<0.5), Balanced (0.5–0.85), Intensive (>0.85)
TCR	$\frac{\text{Complete Translations}}{\text{Total Entries}}$	Measure translation reliability (no untranslated segments)	Ch. 5	Fine-tuning improves TCR to >98% across models
CPS	$\frac{\text{Source Characters}}{\text{Time (seconds)}}$	Measure translation efficiency	Ch. 5	2–3× speed improvement with smaller fine-tuned models